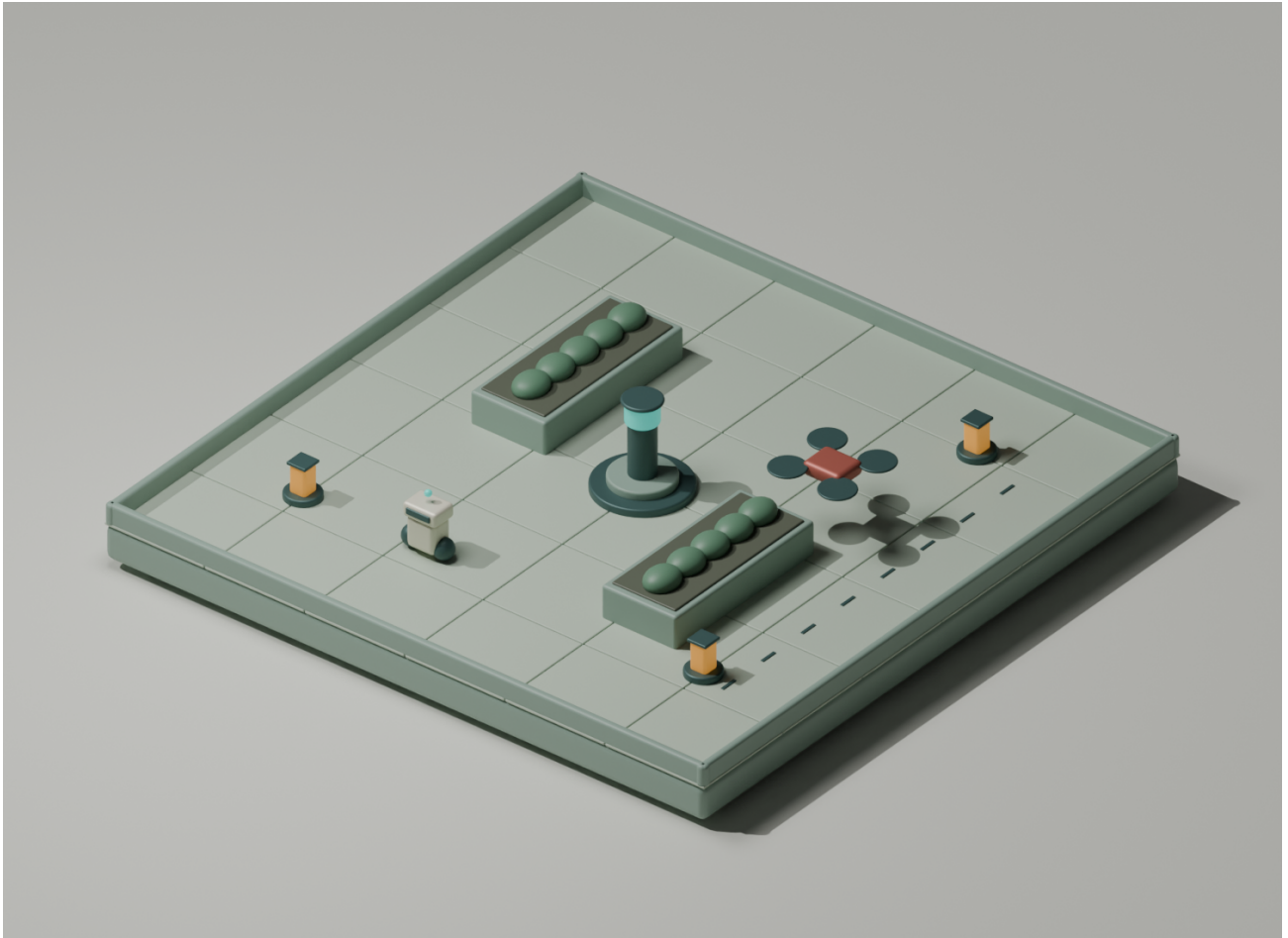


How to do better

From a rough idea to a playable 3D game.

A thoughtful way to work with AI.



Blender concept: Signal Garden, an original 24 by 24 arena with three energy cells and one beacon

14 September 2026 · English

Web edition · GitHub

Contents

Quick summary	3
Set up your environment	5
Copy-and-paste cookbook	11
The better loop	25
Six ways to build	28
Learn from the evidence	31
Choose the right workspace	35
Give the work a memory	38
Decide whether a skill adds value	41
Design a game worth finishing	48
Make a prompt that makes a prompt	53
The Astra master prompt	57
From empty folder to game	63
Verify the game you can play	66
Refine the idea and the process	69
Publish and close the loop	75
Think, decide, improve	79
Make a video with an agent	89
Review & corrections	104
Sources & reading notes	105

This is a book about making a game with GPT-6 Astra. **Signal Garden** is the worked design example; the prompts and milestones are original templates, not a claim that the game has already been built or benchmarked.

QUICK READ

Quick summary

The main points and the shortest path through the book.

- **Start small:** one playable loop with clear completion rules.
- **Add only useful guidance:** skills can help, but availability does not guarantee correct selection or better results.
- **Judge the result:** play it, record evidence, and improve one cause at a time.

Start here

New to the tools? Follow environment setup first.

Ready to work? Open the [copy-and-paste cookbook](#): one prompt per step, expected outputs, and links to the full explanation.

Want a complete planning document first? Try the optional [PLAN.md](#) generator and its self-review steps; the default is a separate small RTS exercise.

The concise steps

1. **Define the game.** Write the objective, controls, win/loss rules, and exclusions. The working loop → · [Full game brief](#) → · [Copyable prompt](#) →
2. **Choose a manageable approach.** Match the workflow and engine to your biggest unknown; test a tiny browser export. Compare approaches → · [Choose the stack](#) → · [Copyable prompt](#) →
3. **Check the evidence.** Match advice to the model and installed versions. Non-Astra benchmarks do not prove Astra gains. Research method → · [Copyable prompt](#) →
4. **Give the project a memory.** Save the brief, acceptance checks, commands, and progress in Git; review changes before committing. Repository setup → · [Copyable prompt](#) →
5. **Check tools and skills.** Reuse relevant available guidance; add a skill only for a demonstrated gap. MCP connects capabilities; hooks respond to events. Skill decisions → · [Decision and data flows](#) → · [Copyable prompt](#) →
6. **Give Astra a focused request.** Use the compact prompt with accurate project files, or the bootstrap for an empty folder. A meta-prompt is optional. Execution prompts → · [Prompt designer](#) → · [Copyable prompt](#) →
7. **Build in playable stages.** Setup → movement → win loop → loss/challenge → visual polish → release checks. M0-M5 plan → · [Copyable prompt](#) →
8. **Verify real play.** Test controls, collision, pause, win, loss, and restart. Observe a human playtest. Full verification guide → · [Copyable prompt](#) →

9. Fix, then distill. Save prompts and outcomes, repair the cause, and retain verified lessons. Compare P1/P2 from the same baseline before claiming a better prompt. Refinement method → · Copyable prompt →

10. Publish and check. Build for the chosen host and URL path, deploy when authorized, then verify the expected revision and a complete run at the public URL. GitHub Pages walkthrough → · Copyable prompt →

11. Optional: make a trailer. Use real capture for gameplay, label generated concepts, compose the clips, and review the final export. Video workflow → · Copyable prompt →

Signal Garden and these workflows are worked proposals, not measured game results. Source summaries → · Review and evidence limits →

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Main points and scope

Covers: Introduction

This summary condenses the existing book. Small scope and evidence-led iteration are our recommendations, informed by the Astra game case study and instruction guidance. Skill availability does not establish selection accuracy or measured game improvement.

OpenAI · Building games with Astra [04: reading note]
OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]
OpenAI · Build skills [08: reading note]
OpenAI · Desktop quickstart and Windows setup [56: reading note]
GitHub CLI · Browser login and Git authentication [61: reading note]
Vercel · Getting started, Git import and Vite [63: reading note]
Vercel · Hobby plan [64: reading note]

The short route through the full chapters

Covers: The concise steps

Each step links to the chapter containing its detailed rules, procedures, and evidence limits. The sequence, game, and optional trailer are worked proposals. Documentation supports the cited capabilities and deployment pattern; this summary adds no experiment or benchmark result.

OpenAI · Building games with Astra [04: reading note]
OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]
GitHub · Using custom workflows with GitHub Pages [13: reading note]
Remotion · Create videos with coding agents [25: reading note]
ComfyUI · LTX-2.5 video workflows [28: reading note]
OpenAI · Evaluation best practices [52: reading note]

Read the review method and corrections

CHAPTER 01

Set up your environment

From a folder on your computer to a game that other people can open.

This beginner route uses a local coding agent, Git, GitHub and a static TypeScript/Vite/Three.js game. Setup comes before the eleven cookbook prompts.

If tools or accounts already exist, check and reuse them. This chapter describes actions for your game project; reading it does not install software or grant access.

Installation and account availability were checked on 14 September 2026. [56] [58]

1 · Install the development tools

1. Install Git for Windows; the same site links macOS and Linux instructions. Git stores local history. [58]
2. Install a supported Node.js LTS release with npm compatible with the chosen Vite version. This is for building the game, not a requirement to install the desktop agent. [59]
3. Install GitHub CLI for account/repository operations. It is a different program from Git. Reopen the terminal and agent after installation so they see updated paths. [61]

In PowerShell on Windows, or a terminal on macOS/Linux, check the tools. Each command should print a version. A missing command means installation or PATH still needs attention.

```
git --version  
gh --version  
node --version  
npm --version
```

2 · Create a folder and open it in Codex

1. Create an independent folder such as Documents/signal-garden in your file manager. Avoid putting a new game inside this book's repository.
2. Follow the official desktop quickstart and Windows download guide. Current docs call the application the ChatGPT desktop app; select **Codex** for software development. Existing Codex app installations can use their project/folder controls.
3. Sign in with your ChatGPT account, open the game folder as a project, and start a local coding task there. Verify the absolute working path before editing. Choose GPT-6 Astra if offered in your model picker; account access and model options can vary.

The selected local environment must have the tools from step 1. A WSL or remote session has its own environment and may need separate tools and authentication. Keep the app's permission boundaries; use specific permissions when a necessary action requires them. [56]

3 · Optional: use Claude Code in the desktop app

Follow Claude's desktop quickstart, sign in, select **Code** → **Local** → **Select folder**, and choose the game folder. The Code tab requires eligible subscription access; a free chat account is not sufficient for this route.

Git is needed for worktree isolation and this book's version-control workflow. Older Windows app versions also required it before any local session.

Desktop includes Claude Code; a separate CLI install is only needed to run claude from a terminal. [57]

Use one agent at a time on the same checkout. To compare agents, use separate branches/worktrees with the same baseline and evaluation task.

Prompts can share project documents, but model selection and instruction-file discovery belong to each application; do not assume that every agent automatically reads every other agent's configuration. See tool and skill selection.

4 · Create GitHub access with HTTPS

1. Create a GitHub account, verify its email and set up account protection. Skip this if you already have an account. [60]

2. In your local terminal, run the login command below. Complete the browser login yourself. If asked to authenticate Git using GitHub credentials, choose Yes. [61]

```
gh auth login --hostname github.com --git-protocol https --web
gh auth status
```

Run the status check inside the agent's environment too. Codex sign-in is separate from GitHub login.

GitHub CLI authentication can also configure Git's HTTPS credentials, so **SSH is optional**. Keep passwords, tokens and private keys out of chat and commits.

GitHub CLI normally uses the OS credential store; its documented fallback is a plain-text file if no working credential store is available. [61]

5 · Optional: choose SSH instead

If you prefer SSH, follow GitHub's platform-specific key and ssh-agent guide. Check for existing keys first; never overwrite one accidentally.

A new key can be generated with the command below after replacing the email label. Add a passphrase and follow the guide for your OS. [62]

```
ssh-keygen -t ed25519 -C "your_email@example.com"
# After adding the public key to GitHub:
ssh -T git@github.com
```

Add only the .pub key under GitHub Settings → SSH and GPG keys. Before accepting a new host, compare its fingerprint with GitHub's linked official fingerprints.

The test should identify your username; success can still return exit code 1 because GitHub offers no interactive shell. Use an SSH remote such as `git@github.com:YOUR_ACCOUNT/signal-garden.git`.

GitHub CLI still needs its own login for API operations such as creating a repository. [62] [61]

6 · Check the project and set commit identity

Paste this read-only prompt in the selected project chat. It checks readiness before the cookbook starts creating files.

```
## Edit these parameters
PROJECT_FOLDER: current selected game folder
AGENT: Codex
MODEL: GPT-6 Astra
ENGINE: AUTO
OS: AUTO (Windows / macOS / Linux)
SHELL: AUTO (PowerShell / bash / zsh)

## Choices and consistency
- AGENT: choose Codex or Claude Code.
  Select MODEL in that application before sending; use an available Claude model with Claude Code.
  These lines do not switch the actual model or grant tools.
- Work only in PROJECT_FOLDER.
  Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.
- ENGINE: AUTO preserves an existing engine, otherwise compares THREE (TypeScript/Vite/Three.js) and GODOT
  (Godot/GDScript web export).
  Choose one; a changed value does not authorize porting existing work.
  KEEP uses the recorded project choice.

## Goal
Inspect this selected project folder without modifying it.

## Tasks and constraints
- Report its absolute path, existing files, Git root/branch/status, sanitized remote URLs, and whether Git and GitHub CLI are
  available.
  For THREE, check Node.js and npm; for GODOT, check the editor and web-export templates.
  For AUTO, report the existing engine or the available options without installing either.
- Check the runtime against this project's requirements.
- Report GitHub authentication status without displaying tokens, private keys, or environment values.
- Distinguish missing tools, missing login, and permission failures.
- If this is an empty folder, say so; do not invent package scripts.
- Use the actual OS and shell when their values are AUTO; otherwise verify the requested environment is available.
  Give the shortest setup steps for the selected engine.
- Do not install, initialize, commit, push, or deploy during this inspection.

## Completion check
An accurate environment report and the shortest remaining setup steps; no files or account settings changed.
```

After Git has been initialized for this project in cookbook step 4, set a missing author name/email locally with the following commands, replacing both values, then continue the commit step. Your GitHub email settings show a no-reply address if you prefer email privacy.

`user.name` and `user.email` label commits; they do not log you into GitHub. [58]

```
git config --local user.name "Your Name"
git config --local user.email "YOUR_VERIFIED_OR_GITHUB_NOREPLY_EMAIL"
```

Saving a file, committing, pushing, and deploying are four different results. To understand existing code, ask the agent to trace input → state → rendering with file references and Git history; see the repair recipe. A commit cannot prove the game works.

7 · Publish with GitHub Pages

Complete and test the game locally first. For this beginner GitHub Free route, use a public repository and publish only content intended to be public. GitHub Pages serves static files; it does not run your Vite dev server or a multiplayer backend. [46]

1. Use the Pages release prompt with your exact account and repository.
2. For a project URL `https://YOUR_ACCOUNT.github.io/signal-garden/`, set Vite base to `/signal-garden/`. Build and preview the output locally.
3. In the GitHub repository, select Settings → Pages → Source: GitHub Actions. The agent prepares the build/deploy workflow, commits and pushes the intended files. See the full Pages walkthrough.
4. Wait for the workflow for the expected commit, then verify the public URL, assets, win, loss and restart. A successful push alone is not a deployed game.

Vite's deployment guide documents the base path, build output and Actions route. `dist` is its default output; verify your project's actual configuration. [12] [13]

8 · Or publish with Vercel Hobby

Vercel Hobby is a free plan for **personal, non-commercial use**, subject to usage limits. It is not a blanket free tier for a business or commercial game. Check the current plan conditions before choosing it. [64]

1. Use the Vercel release prompt after filling both GitHub and Vercel destinations.
2. For a static Vite game served at the domain root, use base: `/'`, the project's build command (normally `npm run build`), and its output directory (normally `dist`). If using both hosts, configure each deployment's base explicitly.
3. In Vercel's New Project flow, connect GitHub, grant access to the selected repository and import it. Review the account/scope, root directory, Vite preset and build settings, then deploy. This integration needs its own repository access; SSH setup does not grant it.
4. Alternatively, install Vercel CLI with `npm install -g vercel` and run `vercel login`. In the game folder, review the project/scope prompts; use `vercel --prod` when ready to publish production. No extra skill or MCP server is required for the dashboard route.
5. Check the assigned `vercel.app` production URL while logged out, the deployed revision, assets and gameplay. Connected Git pushes can trigger later deployments; distinguish preview URLs from production.

The official getting-started guide covers both CLI and dashboard routes. Vercel's Git and Vite guides explain integration and framework behavior. A private source repository still cannot hide secrets embedded

in browser JavaScript. [63] [12]

[Continue to the copy-and-paste cookbook →](#) · [Back to Quick summary →](#)

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Setup scope and tools

Covers: Introduction · 1 · Install the development tools

Tools have distinct roles; this is a proposed beginner sequence using the linked official installation pages.

[OpenAI · Desktop quickstart and Windows setup \[56: reading note\]](#)

[Git and GitHub · Installation and commit identity \[58: reading note\]](#)

[Node.js · Downloads \[59: reading note\]](#)

[GitHub CLI · Browser login and Git authentication \[61: reading note\]](#)

Local agent access

Covers: 2 · Create a folder and open it in Codex · 3 · Optional: use Claude Code in the desktop app

Verified app setup and local folder selection; the one-agent-per-checkout rule is practical advice, not a product restriction.

[OpenAI · Desktop quickstart and Windows setup \[56: reading note\]](#)

[Anthropic · Claude Code desktop quickstart \[57: reading note\]](#)

[OpenAI · Custom instructions with AGENTS.md \[07: reading note\]](#)

Identity, authentication and preflight

Covers: 4 · Create GitHub access with HTTPS · 5 · Optional: choose SSH instead · 6 · Check the project and set commit identity

The guides support account, HTTPS and SSH mechanics. The read-only preflight and staged identity handoff are original prompts.

[Git and GitHub · Installation and commit identity \[58: reading note\]](#)

[GitHub · Creating an account \[60: reading note\]](#)

[GitHub CLI · Browser login and Git authentication \[61: reading note\]](#)

[GitHub · SSH key setup and connection test \[62: reading note\]](#)

[Git · Staging, differences, branches, and reverting \[39: reading note\]](#)

Hosting routes

Covers: 7 · Publish with GitHub Pages · 8 · Or publish with Vercel Hobby

Checked static output, Pages subpath, Vercel dashboard/CLI routes and Hobby restrictions. No game deployment was performed for this chapter.

Set up your environment

Vite · Deploying a Static Site [12: reading note]

GitHub · Using custom workflows with GitHub Pages [13: reading note]

GitHub · What is GitHub Pages? [46: reading note]

Vercel · Getting started, Git import and Vite [63: reading note]

Vercel · Hobby plan [64: reading note]

Read the review method and corrections

CHAPTER 02

Copy-and-paste cookbook

One worked prompt for every step in the Quick summary.

Start with environment setup. These original examples follow Signal Garden's game rules.

Paste **one prompt at a time** into the chat attached to your game folder, inspect the result, then continue. Keep your own agreed requirements when adapting the example.

The recipes are proposed workflows, not a benchmark or a promise that one prompt produces a finished game.

A chatbot without repository and execution tools can help draft a brief or prompt; it cannot inspect your local code, run checks or publish it. For those steps use a coding agent with the selected folder and relevant tools.

In Claude Code, use an available Claude model; writing "Astra" in a prompt does not change the model. Existing project files are the continuing source of truth. [56] [57]

Edit the short block, keep the detailed instructions

1. Copy the whole prompt into an unsent chat draft. Then edit the opening block; keep headings and line breaks.
2. **Edit the opening parameters.** Choose one agent and, where offered, one engine or video tool. Leave other defaults unchanged when they suit the project.
3. **Match the real environment.** Select the model in the app and open the intended folder. Typing a model or tool name does not activate it.
4. **Keep one source of rules.** Put design changes in RULE_CHANGES when drafting the brief; later recipes use the agreed project documents. A changed engine choice needs reconciliation with existing work.
5. **Check the result before continuing.** Compare it with the completion check. In a release prompt, PUBLISH: YES authorizes the named public release; use NO for local preparation.

What the RTS planning example teaches

The supplied RTS plan separates scope, grouped tasks, tuning values and completion checks. This book adopts that structure: **parameters** → **goal** → **tasks and constraints** → **completion check**.

A full game specification belongs in the project documents. A follow-up prompt should request the next useful outcome and reference those documents. Keep essential game rules explicit while letting the agent choose routine implementation details; more sections alone do not prove a better game. [17]

Optional: one prompt that creates PLAN.md

The idea is useful: ask for a planning document like the RTS example, then inspect it before asking an agent to build. The existing prompt designer creates an execution prompt; this recipe creates the plan itself.

The improvement is to separate a small playable V1 from later ambitions, label assumptions and tuning values, and connect each essential requirement to a check. A polished outline or the model's own approval does not establish feasibility or a fun game. Our format is an original proposal; the current Astra guidance supports focused instructions and clear completion boundaries, not a measured advantage for this template. [17] [52]

Try the defaults: copy the whole block into a fresh, unsent chat draft. It describes a small RTS and requires no access to the private reference file. Choose the model in the app, edit the opening values, and send. OUTPUT_MODE: CHAT works for a chatbot without file tools; save its Markdown as PLAN.md. A coding agent can use FILE in the intended project folder.

This is a separate RTS exercise. To use it for Signal Garden, replace GAME_IDEA, V1_SCOPE and the other relevant parameters with the agreed brief. Reconcile the accepted plan with existing project documents before continuing the recipes below.

Edit these parameters

PROJECT_NAME: Frontier Command

GAME_IDEA: A small 3D RTS: gather alloy, build a base, train units, and destroy one AI opponent's headquarters.

V1_SCOPE: One map, one shared faction, one resource, workers, one combat unit type, headquarters, one production building, win/loss/pause/restart.

EXCLUSIONS: Multiplayer, campaigns, accounts, paid services, multiple factions, fog of war, and large technology trees.

PLATFORM: DESKTOP_BROWSER

INPUT: MOUSE_KEYBOARD

ENGINE: AUTO

ASSET_TOOL: BLENDER

PROJECT_CONTEXT: EMPTY

REFERENCE: NONE

OUTPUT_LANGUAGE: English

OUTPUT_MODE: CHAT

OUTPUT_FILE: PLAN.md

Choices and consistency

- PLATFORM: DESKTOP_BROWSER or WINDOWS_DESKTOP.

- ENGINE: AUTO, GODOT, or THREE (TypeScript/Vite/Three.js).

Preserve an existing or explicit engine choice.

With AUTO and an empty project, recommend one route compatible with PLATFORM and explain the main tradeoff briefly.

Flag a conflicting engine/platform choice before dependent decisions; do not silently add a native wrapper or change the target.

- ASSET_TOOL: BLENDER or PRIMITIVES.

Plan placeholders first, and an import check before detailed Blender assets.

- PROJECT_CONTEXT: EMPTY, or relevant project paths/pasted documents.

REFERENCE: NONE, or an attached example/accessible file path.

The task must work without a reference; use a supplied reference for structure, not as authority over these requirements.

- Select the actual model in the app before sending; for example, GPT-6 Astra in Codex or an available Claude model in Claude Code.

Parameter text does not supply tools or change the model.

- OUTPUT_MODE: CHAT returns Markdown to save as OUTPUT_FILE.

FILE writes that document only if filesystem tools and the intended project folder are available.

Preserve an existing file by writing an unused candidate filename; otherwise return the document in chat and explain the limitation.

Goal

Produce the complete content of a practical PLAN.md for PROJECT_NAME.

The deliverable is the game-development plan itself, not another prompt and not an implemented game.

Tasks and constraints

- Use the parameters and relevant accessible project context.
Label unavailable files/tools and unknown installed versions; do not pretend to inspect them.
- Separate requirements, preferences, assumptions, tuning candidates and blocking questions.
Draft with reasonable labeled assumptions; ask only about conflicts that prevent a coherent plan.
- Keep V1_SCOPE small and complete.
Put suggested additions in a separate later backlog; do not import all features from a reference RTS.
- Organize the plan into numbered sections with short paragraphs, task bullets and useful tables.
Each implementation section needs its outcome, dependencies, grouped tasks and an observable completion check.
- Cover concept/scope; environment and first export check; map/camera/input; selection/commands/movement; economy/construction/production; combat/opponent AI; match state/UI; assets; integration/playtesting; release and milestones.
Adapt or omit genre-specific sections if GAME_IDEA changes.
- Define ownership of input, simulation state and presentation, with a small suggested folder structure for the selected engine.
Specify important behavior without prescribing every class, API or algorithm in advance.
- For the RTS default, resolve UI clicks versus world orders, destroyed command targets, blocked movement, resource spending/refunds, production queues and AI access to resources.
Define simultaneous match outcomes, pause/focus loss and a full restart.
- Give essential requirements stable IDs and map them to checks in an acceptance table.
For resource accounting, specify when costs are charged, whether cancellation refunds them, and how duplicate completion/refund is prevented.
- Put proposed numbers in one tuning table with units, rationale and a playtest method.
Mark them unvalidated; avoid conflicting duplicates and invented performance guarantees.
- Use outcome milestones in dependency order, starting with a minimal scene and export check, then one playable economy/combat loop.
Include risks, a small experiment for each major unknown, and a release route suited to PLATFORM.
- Verify version-sensitive technical claims with current primary documentation when browsing is available.
Link each such claim to its source and checked date, or mark it unverified.
Original design choices need rationale, not fabricated citations.
- Review the draft once for conflicting parameters, missing dependencies, untestable requirements, scope growth and unavailable tools.
Correct concrete defects and leave unresolved risks visible.
- This request authorizes planning and the selected document output only.
Do not install tools, implement gameplay, create assets, commit, push, deploy or purchase anything.

Completion check

Return one coherent Markdown plan, including its acceptance table, tuning table, milestones, sources/unknowns and a short review note.

End with the smallest implementation task the reader could authorize next.

Do not execute that task or report planned checks as passed.

Expected output: a complete Markdown plan with grouped implementation tasks, dependencies, a tuning table, requirement-to-check links, milestones, sources and unresolved risks. Planning stops before implementation.

Test the plan yourself

1. **Check consistency:** one engine and target, V1 exclusions respected, no reference-file dependency, and no claims that planned work already passed.

2. **Trace one requirement:** find the relevant task and check. For production, identify when the cost is charged, what cancellation refunds, and what prevents a second refund.

3. **Try a parameter change:** generate a separate candidate with `ASSET_TOOL: PRIMITIVES`. Blender must become optional; required imports and milestones must agree. Keep the old plan for comparison.

4. **Compare if useful:** in separate fresh chats, use the same project facts, model, tools and budget with this template and a short request such as “Write a development plan from these facts.” Judge contradictions, missing dependencies, scope growth and testable checks; repeat before claiming a reliable prompt advantage.

5. **Test feasibility next:** after accepting the plan, separately authorize its smallest scene/export experiment. Record what actually works and revise the affected assumptions. A document review cannot replace that experiment or human playtesting.

No independent model trial or game build was run to validate this new recipe. Use the refinement chapter to track prompt versions and measured outcomes separately. [52]

1 · Define the game

Use this first, with an empty game folder or an existing brief.

```
## Edit these parameters
PROJECT_FOLDER: current selected game folder
AGENT: Codex
MODEL: GPT-6 Astra
ENGINE: AUTO
PROJECT_NAME: Signal Garden
ASSET_TOOL: PRIMITIVES
RULE_CHANGES: none

## Choices and consistency
- AGENT: choose Codex or Claude Code.
  Select MODEL in that application before sending; use an available Claude model with Claude Code.
  These lines do not switch the actual model or grant tools.
- Work only in PROJECT_FOLDER.
  Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.
- ENGINE: AUTO preserves an existing engine, otherwise compares THREE (TypeScript/Vite/Three.js) and GODOT
  (Godot/GDScript web export).
  Choose one; a changed value does not authorize porting existing work.
  KEEP uses the recorded project choice.
- ASSET_TOOL: PRIMITIVES uses engine geometry; BLENDER uses an available Blender workflow after a small import check.
  Start with placeholders either way; do not install tools just because a name appears here.
- PROJECT_NAME names this example.
  RULE_CHANGES contains your changes to the defaults below.
  Reconcile changes with existing agreed documents before editing; save one authoritative set of rules.

## Goal
Draft or reconcile the game brief and its observable acceptance checks.

## Tasks and constraints
- In this game project, inspect existing instructions and documents first.
- Draft or reconcile docs/game-brief.md and docs/acceptance.md for PROJECT_NAME: one 24 x 24 metre rooftop arena, a fixed
  elevated camera, keyboard movement, three energy cells and a beacon.
- Use a 90-second round, three health points, one patrolling drone, and a one-second damage cooldown.
- E activates the beacon within two ground-plane units after all cells are collected.
- Loss takes priority if time or health reaches zero in the same step.
```

- Pause freezes simulation; blur pauses and clears input; resume is explicit; restart resets all state.
- Exclude multiplayer, backend, accounts and paid assets.
- Separate requirements from design assumptions.
- Resolve contradictions with existing user choices before changing them.
- Return the brief and observable checks, without implementing gameplay yet.

Completion check

A short brief and acceptance checklist with consistent rules.

Expected output: A short brief and acceptance checklist with consistent rules.

Full reading: Design a game worth finishing · The better loop. [17]

2 · Choose the approach

Use after defining scope. A spike answers one risky technical question.

Edit these parameters

PROJECT_FOLDER: current selected game folder

AGENT: Codex

MODEL: GPT-6 Astra

ENGINE: AUTO

Choices and consistency

- AGENT: choose Codex or Claude Code.

Select MODEL in that application before sending; use an available Claude model with Claude Code.

These lines do not switch the actual model or grant tools.

- Work only in PROJECT_FOLDER.

Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.

- ENGINE: AUTO preserves an existing engine, otherwise compares THREE (TypeScript/Vite/Three.js) and GODOT (Godot/GDScript web export).

Choose one; a changed value does not authorize porting existing work.

KEEP uses the recorded project choice.

- If ENGINE is explicit, evaluate that route against the brief instead of recommending a different engine.

Goal

Choose one workable engine and record the evidence for that choice.

Tasks and constraints

- Read the game brief and inspect available tools.
- Only when ENGINE is AUTO and no engine is already chosen, compare TypeScript/Vite/Three.js with Godot web export for this small keyboard browser game.
- Evaluate iteration speed, asset workflow, browser constraints, debugging and deployment for the chosen route or comparison.
- Keep an explicitly chosen engine.
- Otherwise recommend the smallest workable route; TypeScript/Vite/Three.js is the default for this example.
- Verify version-sensitive claims in official docs and state unknowns.
- Record the decision in docs/decisions.md.
- If a browser-export uncertainty could invalidate it, make only a disposable minimal scene in a clearly named spike folder and test its production export.
- Record commands and observed failures; do not describe an unrun test as passing.

Completion check

A recorded engine choice and export evidence if an experiment was needed.

Expected output: A recorded engine choice and export evidence if an experiment was needed.

Full reading: [Six ways to build](#) · [Choose the right workspace](#). [\[09\]](#) [\[11\]](#) [\[12\]](#)

3 · Check current evidence

Use before applying advice about versions, models or skills. Requires browsing for fresh verification.

Edit these parameters

PROJECT_FOLDER: current selected game folder

AGENT: Codex

MODEL: GPT-6 Astra

Choices and consistency

- AGENT: choose Codex or Claude Code.

Select MODEL in that application before sending; use an available Claude model with Claude Code.

These lines do not switch the actual model or grant tools.

- Work only in PROJECT_FOLDER.

Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.

Goal

Verify the advice that will guide this game project.

Tasks and constraints

- Review the advice we are about to use for this game.

- Search primary sources, prioritizing material published or updated since March 2026, while retaining necessary live documentation with its checked date.

- For each important decision, save the URL, relevant heading, main points, installed-version match, and limits in docs/research.md.

- Check claims about automatic skill selection and GPT-6 Astra specifically.

- Distinguish a documented capability, a study on another model, and our own proposed workflow.

- Do not infer Astra gains from another model's benchmark.

- If a YouTube transcript is unavailable, label it and use a verified written source instead.

- End with only the changes this evidence justifies.

Completion check

Source notes that distinguish checked facts from hypotheses.

Expected output: Source notes that distinguish checked facts from hypotheses.

Full reading: [Learn from the evidence](#). [\[17\]](#) [\[52\]](#)

4 · Save project memory

Use once the brief and approach agree. Git author identity must be configured before a commit.

Edit these parameters

PROJECT_FOLDER: current selected game folder

AGENT: Codex

MODEL: GPT-6 Astra

Choices and consistency

- AGENT: choose Codex or Claude Code.

Select MODEL in that application before sending; use an available Claude model with Claude Code.

These lines do not switch the actual model or grant tools.

- Work only in PROJECT_FOLDER.

Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.

Goal

Create accurate project memory and a reviewed local baseline.

Tasks and constraints

- Prepare this game's project memory.
- Inspect the Git root and current work first; never initialize a nested repository accidentally.
- For a new independent folder, initialize Git on main.
- Preserve existing history and unrelated files.
- Reconcile the brief, acceptance checks, decisions, and docs/progress.md; keep AGENTS.md short and limited to useful project conventions.
- Create a suitable .gitignore before staging: exclude secrets, local credentials, dependencies and generated output, but retain source and the lockfile.
- Read back the diff, stage only the intended files, and make a local baseline commit when the configured author identity is correct.
- If identity is missing, ask for it rather than inventing it.
- Report the commit and the next milestone.
- Do not create a remote or publish in this step.

Completion check

Reviewed project documents and a local baseline commit; nothing published.

Expected output: Reviewed project documents and a local baseline commit; nothing published.

Full reading: Give the work a memory. [07] [39] [58]

5 · Choose useful skills

Use for the next concrete task, not as a request to install every popular skill.

Edit these parameters

PROJECT_FOLDER: current selected game folder

AGENT: Codex

MODEL: GPT-6 Astra

Choices and consistency

- AGENT: choose Codex or Claude Code.
- Select MODEL in that application before sending; use an available Claude model with Claude Code.
- These lines do not switch the actual model or grant tools.
- Work only in PROJECT_FOLDER.
- Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.

Goal

Decide whether a skill adds value for the next concrete task.

Tasks and constraints

- Inspect the tools and skill catalog actually available in this session.
- For the next game milestone, identify whether any skill adds relevant checks or capabilities beyond our existing brief and tools.
- Prefer no extra installation unless a concrete gap exists.
- For a proposed GitHub skill, inspect its exact path, instructions, scripts, dependencies, license and revision before recommending it.
- Compare no skill, automatic selection and explicit invocation on the same small task if a comparison would change our choice.
- Record observed selection, task result and overhead.
- Do not install a bundle merely because it is popular, and do not claim that availability guarantees invocation.
- Save the decision and any untested hypothesis in docs/decisions.md.

Completion check

A justified choice: existing tools, available skill, or a specific missing capability.

Expected output: A justified choice: existing tools, available skill, or a specific missing capability.

Full reading: Decide whether a skill adds value · Think, decide, improve. [08] [17]

6 · Start a focused build

Send to the coding agent after steps 1-5. Select GPT-6 Astra in Codex if your account offers it.

```
## Edit these parameters
PROJECT_FOLDER: current selected game folder
AGENT: Codex
MODEL: GPT-6 Astra
ASSET_TOOL: PRIMITIVES

## Choices and consistency
- AGENT: choose Codex or Claude Code.
  Select MODEL in that application before sending; use an available Claude model with Claude Code.
  These lines do not switch the actual model or grant tools.
- Work only in PROJECT_FOLDER.
  Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.
- ASSET_TOOL: PRIMITIVES uses engine geometry; BLENDER uses an available Blender workflow after a small import check.
  Start with placeholders either way; do not install tools just because a name appears here.

## Goal
Implement and verify the first playable movement checkpoint.

## Tasks and constraints
- Use the approved docs/game-brief.md, docs/acceptance.md and docs/decisions.md to implement the first playable movement slice of the game in this selected folder.
- Inspect existing code before editing and keep the chosen stack.
- Build a floor, robot placeholder, fixed elevated camera, keyboard movement and collision boundaries.
- Use consistent units and normalize nonzero digital diagonal input.
- Handle pause, focus loss and explicit resume without stuck keys or a large resume time step.
- Establish working install, dev and build commands.
- Verify the build and real keyboard interaction if tools permit; report any checks you cannot run.
- Update docs/progress.md with what works and the next milestone.
- Stop at this movement checkpoint so I can inspect it; do not add unrelated features or publish.

## Completion check
A runnable movement slice and honest verification record.

The longer bootstrap is linked below.
```

Expected output: A runnable movement slice and honest verification record. The longer bootstrap is linked below.

Full reading: The Astra master prompt · Make a prompt that makes a prompt. [04] [17]

7 · Continue one milestone

Repeat this after inspecting each checkpoint. Read the progress file again in a new session.

Edit these parameters

PROJECT_FOLDER: current selected game folder

AGENT: Codex

MODEL: GPT-6 Astra

Choices and consistency

- AGENT: choose Codex or Claude Code.

Select MODEL in that application before sending; use an available Claude model with Claude Code.

These lines do not switch the actual model or grant tools.

- Work only in PROJECT_FOLDER.

Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.

Goal

Complete one next playable milestone from the current state.

Tasks and constraints

- Continue the game from its actual current state.

- Read docs/progress.md, the acceptance checks and the relevant code; verify the previous checkpoint before trusting the note.

- Implement the next incomplete playable milestone: win loop, then drone/health/time loss, then visual polish, then release readiness.

- For this run, finish one milestone and its affected checks.

- Preserve the agreed rules, including loss-before-activation, single collection, cooldown timing and complete restart.

- Use placeholder assets until mechanics pass.

- Record the commands, observed results, remaining failures and next step.

- Review and commit only this milestone's intended changes after checks pass.

- Leave publication to the release step.

Completion check

One additional playable milestone, its checks and a reviewed commit.

Expected output: One additional playable milestone, its checks and a reviewed commit.

Full reading: From empty folder to game. [04] [05]

8 · Verify actual play

Use on a production build. The human observation task is for you or another player.

Edit these parameters

PROJECT_FOLDER: current selected game folder

AGENT: Codex

MODEL: GPT-6 Astra

Choices and consistency

- AGENT: choose Codex or Claude Code.

Select MODEL in that application before sending; use an available Claude model with Claude Code.

These lines do not switch the actual model or grant tools.

- Work only in PROJECT_FOLDER.

Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.

Goal

Verify the production game through rules, real controls and human observation.

Tasks and constraints

- Verify the current game against docs/acceptance.md using the production build.

- Check movement, boundaries, diagonal speed, single collection, beacon range, damage cooldown, pause, focus loss/resume,

both loss causes, win and repeated restart.

- Separate logic tests from real browser input and visual inspection.
- Record browser, viewport, revision, steps, expected result and actual result in a new docs/playtests/ entry.
- Do not infer playability from a passing build.
- Give me a short first-time-player test: find the objective, attempt a round, restart, and describe confusing moments.
- Mark human observations as pending until I supply them.
- Rank concrete failures and propose the smallest next repair.

Completion check

Reproducible failures and an explicit list of checks still pending.

Expected output: Reproducible failures and an explicit list of checks still pending.

Full reading: Verify the game you can play. [43] [44] [45]

9 · Repair and improve the next prompt

Use with real failure evidence. Save outcomes with the prompts; do not paste the whole chat into the next instruction.

Edit these parameters

PROJECT_FOLDER: current selected game folder

AGENT: Codex

MODEL: GPT-6 Astra

Choices and consistency

- AGENT: choose Codex or Claude Code.

Select MODEL in that application before sending; use an available Claude model with Claude Code.

These lines do not switch the actual model or grant tools.

- Work only in PROJECT_FOLDER.

Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.

Goal

Repair one evidenced failure and prepare an honestly evaluated prompt revision.

Tasks and constraints

- Read the latest playtest evidence and reproduce the highest-impact failure.
- Trace it from input through state changes to rendering, citing actual files and relevant Git history.
- Fix the cause with the smallest suitable change, rerun affected checks and preserve a reviewed working checkpoint.
- Save the relevant prompts, model/settings, baseline commit, outcome and evidence links in docs/prompt-log.md; do not copy credentials or irrelevant chat.
- Distill verified lessons into the brief, checks or a compact candidate prompt P2.
- Keep prompt P1 and game versions separately identified.
- Explain that a repaired game V2 does not establish a better prompt.
- Propose a P1/P2 comparison from the same clean baseline, equal tool access and budget, with repeated trials and a held-out task; run it only within the agreed evaluation scope.

Completion check

A tested repair and a compact candidate prompt, with a fair comparison plan.

Expected output: A tested repair and a compact candidate prompt, with a fair comparison plan.

Full reading: Refine the idea and the process. [39] [52]

10 · Publish to one destination

Choose one prompt below and replace the account fields. These prompts explicitly authorize publication of this game's source and build. Check the destination before sending.

GitHub Pages · public static game

Edit these parameters

PROJECT_FOLDER: current selected game folder

AGENT: Codex

MODEL: GPT-6 Astra

GITHUB_OWNER: [YOUR_ACCOUNT]

REPOSITORY: signal-garden

PUBLISH: YES

Choices and consistency

- AGENT: choose Codex or Claude Code.

Select MODEL in that application before sending; use an available Claude model with Claude Code.

These lines do not switch the actual model or grant tools.

- Work only in PROJECT_FOLDER.

Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.

- PUBLISH: YES authorizes the public repository/source and deployment described below; NO means local release preparation only.

Resolve all destination placeholders before publication.

Derive the URL and base path from GITHUB_OWNER and REPOSITORY.

Goal

Prepare or publish this game on GitHub Pages, according to PUBLISH.

Tasks and constraints

- Use the GITHUB_OWNER and REPOSITORY values above; resolve placeholders before any external publication.

- Inspect the actual repository, branch, remotes, intended diff, excluded secrets and asset licenses.

Preserve unrelated work; do not replace a conflicting remote or force-push.

- Keep the existing engine.

For a Vite/Three.js project, verify the install/build commands and dist output.

For Godot, verify the installed version, web-export support and actual export directory; preserve the generated filenames.

Prove the static build works before publishing.

- For a project site, derive the URL as https://GITHUB_OWNER.github.io/REPOSITORY/ using the actual values.

Set Vite base to /REPOSITORY/ when using Vite; for another engine, verify its exported assets under that path.

- Prepare a GitHub Pages Actions workflow that builds the project and uploads only the verified static output directory, with the required Pages permissions.

- If PUBLISH is YES, I authorize creating the named public GitHub repository if absent, committing and pushing this game's reviewed intended files, and publicly deploying it to the selected host.

If PUBLISH is NO, stop after local preparation and report the remaining release steps.

- For YES, enable Pages from GitHub Actions if permitted and follow the workflow for the exact pushed commit.

- Open the resulting public URL, verify assets and a complete round through real controls if available, and identify any unperformed checks.

Completion check

PUBLISH=NO: a tested local release package and remaining steps.

PUBLISH=YES: the reviewed commit, matching successful deployment, public URL and actual verification record.

Report a concrete blocker if completion is unavailable.

Vercel Hobby · personal, non-commercial game

Edit these parameters

PROJECT_FOLDER: current selected game folder

AGENT: Codex

MODEL: GPT-6 Astra

GITHUB_OWNER: [YOUR_ACCOUNT]

REPOSITORY: signal-garden

PUBLISH: YES

VERCEL_SCOPE: [YOUR_VERCEL_SCOPE]

Choices and consistency

- AGENT: choose Codex or Claude Code.

Select MODEL in that application before sending; use an available Claude model with Claude Code.

These lines do not switch the actual model or grant tools.

- Work only in PROJECT_FOLDER.

Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.

- PUBLISH: YES authorizes the public repository/source and deployment described below; NO means local release preparation only.

Resolve all destination placeholders before publication.

Use GITHUB_OWNER and REPOSITORY for the source repository.

Use the selected Vercel project's actual domain and deployment path for the game URL and base path.

Goal

Prepare or publish this game on Vercel Hobby, according to PUBLISH.

Tasks and constraints

- Use the GITHUB_OWNER and REPOSITORY values above; resolve placeholders before any external publication.

- Inspect the actual repository, branch, remotes, intended diff, excluded secrets and asset licenses.

Preserve unrelated work; do not replace a conflicting remote or force-push.

- Keep the existing engine.

For a Vite/Three.js project, verify the install/build commands and dist output.

For Godot, verify the installed version, web-export support and actual export directory; preserve the generated filenames.

Prove the static build works before publishing.

- Resolve VERCEL_SCOPE before linking a project.

Check existing Vercel links and report a destination conflict without replacing them.

- Verify this personal, non-commercial project is eligible for Hobby under current limits; do not buy services or upgrade.

- Use Vite base / for a domain-root Vite deployment.

Verify the selected engine's actual build command and output directory; do not assume every engine uses dist.

Prepare the Vercel configuration.

- If PUBLISH is YES, I authorize creating the named public GitHub repository if absent, committing and pushing this game's reviewed intended files, and publicly deploying it to the selected host.

If PUBLISH is NO, stop after local preparation and report the remaining release steps.

- For YES, use an authenticated Vercel CLI or Git import in VERCEL_SCOPE.

Guide browser sign-in or repository access if needed; never ask for secrets in chat.

- Verify the deployed revision and unauthenticated public gameplay.

Report the actual assigned production URL and any checks not run.

Completion check

PUBLISH=NO: a tested local release package and remaining steps.

PUBLISH=YES: the reviewed commit, matching successful deployment, public URL and actual verification record.

Report a concrete blocker if completion is unavailable.

Expected output: A commit, a successful deployment of that revision, a public game URL and recorded gameplay checks.

Full reading: Set up your environment · Publish and close the loop. [12] [13] [40] [46] [63] [64]

11 · Optional: introduce the game

Use after real gameplay exists. A capture plan is the right output when footage or recording tools are missing.

```
## Edit these parameters
PROJECT_FOLDER: current selected game folder
AGENT: Codex
MODEL: GPT-6 Astra
VIDEO_TOOL: AUTO
DURATION_SECONDS: 20
PAID_BUDGET: 0

## Choices and consistency
- AGENT: choose Codex or Claude Code.
  Select MODEL in that application before sending; use an available Claude model with Claude Code.
  These lines do not switch the actual model or grant tools.
- Work only in PROJECT_FOLDER.
  Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.
- VIDEO_TOOL: AUTO selects a suitable installed editor; REMOTION requests an available Remotion workflow.
  If missing, report the gap and prepare the shot plan.
  Use DURATION_SECONDS for the total runtime; PAID_BUDGET is the maximum separately authorized generation spend.

## Goal
Prepare an introduction lasting DURATION_SECONDS seconds for the current game build.

## Tasks and constraints
- Inspect the real build and available capture/editing tools first.
- Use a simple sequence: title and objective, genuine movement/collection footage, drone challenge, beacon success, playable URL.
- Save a shot list, captions and an asset manifest with file paths, rights and build revision.
- If footage is missing, give a precise capture plan and mark those shots pending; do not fabricate gameplay.
- Prefer available tools for deterministic editing, such as an installed Remotion workflow.
- Use generated material only as clearly labeled concept imagery.
- Keep generation spend at zero unless separately authorized.
- Render a draft only when required media and tools are available; review text, pacing, audio and final URL.
- Report output files and pending work.
- Publishing the trailer is a separate action.

## Completion check
A DURATION_SECONDS-second shot plan, verified media list and, when feasible, a reviewed video draft.
```

Expected output: A shot plan for the selected duration (20 seconds by default), a verified media list and, when feasible, a reviewed video draft.

Full reading: Make a video with an agent. [26] [27] [47]

Back to concise steps →

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Original recipes with documented boundaries

Covers: Introduction · 1 · Define the game · 2 · Choose the approach · 3 · Check current evidence · 4 · Save project memory · 5 · Choose useful skills · 6 · Start a focused build · 7 · Continue one milestone · 8 · Verify actual play · 9 · Repair and improve the next prompt · 10 · Publish to one destination · 11 · Optional: introduce the game

The eleven steps and optional PLAN.md generator are original templates, not measured model results. The plan recipe adds parameter checks, requirement-to-check links and a reader-run comparison; no independent model trial or game build validates it. Citations support instruction guidance, evaluation and the documented tool/publication boundaries.

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

OpenAI · Building games with Astra [04: reading note]

Anthropic · Effective harnesses for long-running agents [05: reading note]

OpenAI · Custom instructions with AGENTS.md [07: reading note]

OpenAI · Build skills [08: reading note]

Godot · Your first 3D game [09: reading note]

Godot · Exporting for the Web [11: reading note]

Vite · Deploying a Static Site [12: reading note]

GitHub · Using custom workflows with GitHub Pages [13: reading note]

Remotion · Agent Skills [26: reading note]

Remotion · Documentation MCP deprecation [27: reading note]

Git · Staging, differences, branches, and reverting [39: reading note]

GitHub CLI · gh repo create [40: reading note]

MDN · requestAnimationFrame [43: reading note]

MDN · Autoplay guide for media and Web Audio APIs [44: reading note]

MDN · prefers-reduced-motion [45: reading note]

GitHub · What is GitHub Pages? [46: reading note]

Runway · Usage tiers and throttled tasks [47: reading note]

OpenAI · Evaluation best practices [52: reading note]

OpenAI · Desktop quickstart and Windows setup [56: reading note]

Anthropic · Claude Code desktop quickstart [57: reading note]

Git and GitHub · Installation and commit identity [58: reading note]

Vercel · Getting started, Git import and Vite [63: reading note]

Vercel · Hobby plan [64: reading note]

Read the review method and corrections

CHAPTER 03

The better loop

A better result begins with a smaller uncertainty.

You ask an AI agent to make a beautiful 3D game. It creates a landscape, a character, a menu, and several hundred lines of code.

The screenshot looks promising. Then you press a key: the camera points into a wall, the player slides through a collectible, and nothing explains how to win.

The request was easy to begin and difficult to finish because “good” never became something you could observe.

This book uses one original example, **Signal Garden**: guide a small maintenance robot around an abandoned rooftop garden, collect three energy cells, avoid a patrolling drone, and power a beacon before time runs out. The game is deliberately small.

It gives us enough movement, collision, feedback, atmosphere, and state transitions to practice making a complete experience.

01 / INTENT / Describe / What should a player experience?

02 / ACTION / Build / Make one part playable.

03 / EVIDENCE / Observe / Play it. Capture what happens.

04 / LEARNING / Refine / Change the weakest part.

Give every iteration a question

“Can a new player find the beacon?” is a useful question.

It suggests a route, a camera angle, a first playtest, and a way to judge the change. “Make the game more professional” leaves the agent to guess among textures, controls, menus, sound, or architecture.

The next step is often easier to choose when the current problem is named precisely.

For this book, better means increasing the quality of the player’s experience while keeping the cost of the next change manageable. The first half needs playtesting.

The second needs understandable files, repeatable checks, and recoverable commits. A beautiful frame and a successful build each answer only part of the question.

A model, an agent, and a workflow

- **GPT-6 Astra** is the requested model.

- An **agent** is the surrounding software that gives that model access to files, a terminal, documentation, and possibly a browser or game editor.
- A **workflow** is your agreement about what to build, what evidence to collect, and when to continue.

A pasted prompt cannot give an ordinary chat tools it does not have.

OpenAI's Astra guidance recommends making autonomy, instruction-file behavior, delegation, and test scope explicit. Use the model name shown by your agent host, and check that the tools needed for the task are available.

The game recipe here does not depend on a particular subscription tier or an invented model capability. [01]

The working principle

Write the smallest instruction that carries the intent, constraints, and evidence needed for the next useful decision.

How to use this book

1. Read the approach comparison before choosing an engine.
2. Set up the repository and fill the game brief.
3. Use a compact execution prompt when the brief is clear; use the optional meta-prompt to resolve missing design decisions.
4. Review available skills before adding project-specific instructions.
5. Build through useful checkpoints, keep the evidence, and revise the workflow when the model or the task changes.

The September 2026 revision checks the advice against current Astra guidance and research published or updated since March. Every source has a readable summary, date information, and limits in the reading notes. Older videos remain explicitly labeled background.

For a visual map, use the thinking, decision, and improvement diagrams. The video walkthrough extends the same method to a game trailer and explains how node workflows, MCP, skills, and hooks fit together.

Try this before you prompt

Write one sentence for the player's objective, one action that should feel satisfying, and one observable condition that means the first version is done. If those sentences are unclear, work on the game idea first.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

The worked example and iteration method

Covers: Introduction · Give every iteration a question · How to use this book · Try this before you prompt

The broken-game opening is an illustrative scenario. Signal Garden, the four-step loop, and the exercises are our teaching design. The sources motivate inspectable results and focused guidance; they do not validate this particular game.

[OpenAI · Building games with Astra \[04: reading note\]](#)

[OpenAI · Rethinking skills and prompts for GPT-6 Astra \[17: reading note\]](#)

Model, agent, and working agreements

Covers: A model, an agent, and a workflow

Astra guidance covers initiative, instructions, delegation, and verification. MCP architecture distinguishes the host and exposed tools; a prompt alone cannot supply missing tool access.

[OpenAI · GPT-6 Astra prompting best practices \[01: reading note\]](#)

[MCP · Architecture, specification 2026-07-28 \[23: reading note\]](#)

[Read the review method and corrections](#)

CHAPTER 04

Six ways to build

Choose an approach for the uncertainty you have.

There is no single best prompting style for every stage of a game. A fast sketch helps you discover a direction.

A specification helps you preserve a direction. A test helps you check a claim.

Confusion begins when one of those activities is expected to do all three.

The comparison below is an editorial assessment for a solo developer making a small browser game. It is **not a measured ranking of models**. Setup effort and review effort depend on the project and the tools the agent can actually use.

Approach	Best use	Main weakness	Refinement
1. One-shot prototype	Discover an attractive visual or a promising mechanic quickly.	A broad request can hide incomplete controls and game states.	Limit it to one arena and one loop. Treat it as a sketch until you play it.
2. Specification first	Preserve a clear idea across many changes.	Documentation can grow before the game has proved enjoyable.	Write a one-page brief and a short acceptance list; validate with a playable slice.
3. Tutorial guided	Learn an unfamiliar engine using a known example.	Old APIs, incompatible assets, or copied structure can mislead.	Match versions, reproduce the small lesson, then adapt one mechanic.
4. Incremental agent loop	Build a complete game over several sessions.	Local improvements can drift from the whole experience.	Keep a runnable baseline, milestone evidence, and a short handoff note.
5. Visual playtest loop	Improve camera, readability, animation, and feedback.	A good screenshot does not prove behavior over time.	Pair images with actual input sequences and state observations.
6. Parallel specialists	Independent research, art exploration, or review of stable interfaces.	Conflicting edits and integration can consume the time saved.	Assign separate files or read-only work; one owner integrates and tests.

What the sources establish

GitHub's Spec Kit separates specification, planning, tasks, and implementation. That is a useful model for keeping intent visible. It does not require a tiny game to adopt every template or tool in the kit. [06]

Anthropic reports that long-running web-app agents benefited from incremental work, progress records, Git history, and browser verification. Its report does not establish that multiple agents always beat a single agent, or that its findings automatically transfer to every game. [05]

Its March 2026 follow-up revisits the surrounding workflow as models improve. Treat elaborate planning structures as experiments to justify, not prerequisites. Our milestone list names visible outcomes; it does not require a separate agent or a rigid planning ceremony for each one. [21]

OpenAI's Astra game case study combines a player-experience brief, concept references, inspectable state, repeatable scenes, and tests using real controls. It provides an implementation example, rather than a controlled comparison of these six approaches. [04]

The recommended combination

For Signal Garden, begin with a small specification, use a tutorial only where engine knowledge is missing, and make the first complete loop with one agent. Add visual review immediately after movement works. Introduce parallel work only when there is a task that can be integrated without changing the game's shared assumptions.

A useful delegation might be: "Review the collision module without editing it, and report reproducible failures." A poor split is: "One agent builds movement while another rewrites the camera and a third redesigns the coordinate system." Those tasks depend on one another even if their file names differ.

Decision rule

If you do not know whether the idea is fun, prototype it. If you know what should happen but the agent keeps drifting, improve the specification. If you do not know whether it works, improve the evidence.

Choose a first experiment

Write the biggest unknown in your game. Pick one row of the comparison to address it. Name the artifact that will let you decide what to do next: a playable build, a control test, a reference image, or a one-page decision.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Comparison and route selection

Covers: Introduction · The recommended combination · Choose a first experiment

The six-way comparison, integration advice, and recommended combination are editorial judgments for this small game. They are not a measured ranking of prompting styles or single versus multiple agents.

OpenAI · Building games with Astra [04: reading note]

GitHub · Spec Kit [06: reading note]

Anthropic · Harness design for long-running application development [21: reading note]

What the implementation accounts establish

Covers: What the sources establish

Spec Kit documents a workflow. The Astra game article and Anthropic accounts describe implementations. The March follow-up simplifies the workflow for a newer model; none supplies a controlled comparison of all six approaches.

OpenAI · Building games with Astra [04: reading note]

Anthropic · Effective harnesses for long-running agents [05: reading note]

GitHub · Spec Kit [06: reading note]

Anthropic · Harness design for long-running application development [21: reading note]

Read the review method and corrections

CHAPTER 05

Learn from the evidence

A transcript becomes useful when it changes a decision.

Searching for “best AI game prompt” produces demonstrations, tool promotions, speculative claims, and genuine engineering advice side by side. Read each source for a specific purpose.

Use engine documentation to establish API behavior. Use a creator’s demonstration to learn a process.

Use your own build to find out whether that process works under your constraints.

Start with the current evidence

This revision prioritizes material published or updated from **1 March through 14 September 2026**. The two September OpenAI articles directly address Astra; the newer skill studies test other models and therefore inform hypotheses, not claims of Astra performance.

Live official documentation is retained even when it exposes no publication date. A page checked today is not necessarily an article written today.

For the skills question, begin with the Astra instruction audit, then compare SWE-Skills-Bench with SkillsBench’s July update. Read SkillsInjector for selection and SkillAlchemy for evidence-led generation. Each entry below has main-item summaries and tested-model limitations.

Earlier idea	September 2026 assessment	What changes in this book
Break long work into recoverable pieces.	Still a useful project practice; the amount of external scaffolding should be reassessed as models improve. [21]	Use outcome checkpoints, and let implementation order adapt to dependencies.
Give the agent more context.	Useful context remains important; requiring every document for every edit is excessive for Astra. [17]	Point to documents by purpose and avoid duplicating the full brief.
Create a skill to improve the project.	Conditional. Existing skills may suffice; studies differ by skill quality, task, and model. [18] [22]	Custom skill creation becomes optional, with a comparison against the current setup.
Verify the generated result.	Still supported by current Astra game practice. [04]	Keep behavior and visual evidence; use focused checks rather than repetitive blanket demands.
Generate a detailed prompt before coding.	No reviewed experiment establishes that a meta-prompt always outperforms a direct request on Astra.	Use it to resolve uncertainty; offer a compact prompt when the brief already exists.

Earlier idea	September 2026 assessment	What changes in this book
Follow tutorial export instructions.	Version and hosting constraints remain real despite better models. [11] [12]	Check current engine documentation and test the actual deployment path.

Why keep any older videos?

The 2025 videos below are **historical exceptions**. The verification passage in Software Is Changing (Again) was checked again; the other transcript could not be retrieved in this review.

Neither is current product documentation or an Astra benchmark. Recent video searches did not yield a verified replacement transcript suitable for these notes.

Transcript notebook: Software Is Changing (Again)

In his Y Combinator keynote, Andrej Karpathy discusses the loop between AI generation and human verification. He argues that concrete requests and auditable intermediate artifacts make that loop more useful, and notes that a huge code diff still leaves substantial review work.

These are conceptual observations, not measured results for game production. [02]

Our application: ask for one runnable game milestone with a short list of behaviors to inspect. Put the build link, relevant change, and evidence together.

The person reviewing the game should be able to answer a question such as “Does losing all health always show a restart screen?” without first reconstructing the agent’s entire conversation.

Transcript notebook: How I use LLMs

Retrieval limitation: the previous edition’s note used a public transcript mirror. That mirror timed out during this review, and an alternative returned a video landing page without transcript text.

The former passage summary is therefore withdrawn from this edition’s verified evidence. The original video remains a historical reading lead. [03]

Our application: attach the actual error, installed engine version, current scene structure, and a relevant documentation excerpt to a debugging request. “The player is broken” supplies almost no useful working information.

“In Godot version X, this CharacterBody3D falls through the floor in this minimal scene” gives the agent an investigation it can carry out.

The recommendation to supply relevant current evidence is supported independently by Astra’s current guidance on loading task-relevant context. It does not depend on the unavailable transcript. [17]

Transcript provenance

Source 02 uses a public transcript mirror of the original speech; automatic captions may contain errors. No stable passage timestamps were available, so the note gives a searchable term.

Source 03 is a reading lead with unavailable transcript evidence. A reachable page or video title does not verify its spoken content.

A game tutorial is a separate kind of evidence

GDQuest's written Godot 4 course and Godot's own first-3D-game tutorial provide a sequence for learning game structure. Use them to reduce unfamiliarity with the engine. The GDQuest YouTube transcript could not be retrieved reliably during research, so only its accessible written course is used here. [09] [10]

1. Do not carry an entire tutorial into a new design unchanged.
2. Extract one concept, such as movement relative to the camera.
3. Build a minimal reproduction with the installed engine version.
4. Then explain how the concept maps to your game, including what must change.

A successful exercise is evidence that you understand one part; it does not validate the whole game.

A repeatable research procedure

1. **Start with a question.** For example: "Can the chosen Godot export run on GitHub Pages without custom response headers?"
2. **Find the primary answer.** Search official documentation for the installed version; open the page and read the constraints.
3. **Find an applied example.** Prefer a creator's video with an accessible transcript and companion repository.
4. **Record provenance.** Save the URL, title, retrieval date, version, and a passage locator. Mark missing captions or an unavailable repository explicitly.
5. **Separate observation from inference.** "The tutorial uses this node" is an observation. "This will simplify our camera system" is a hypothesis.
6. **Run one small experiment.** Keep the result and the code commit that produced it.

Question: Will the beacon remain visible behind the planter?

Source: [official API link or creator's original tutorial]

Version / date: [installed version] / [date reviewed]

Observed idea: [brief paraphrase]

Our hypothesis: [how it may help this game]

Experiment: [small reproducible change]

Outcome: [measured result, or NOT TESTED]

Commit: [actual revision]

Two accounts of the same video are one underlying source. A successful edited demonstration does not reveal the unsuccessful runs, total effort, or hidden manual work. Keep the useful method, and test the outcome yourself.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Current evidence and older material

Covers: Introduction · Start with the current evidence · Why keep any older videos?

Dates and tested models determine relevance. Newer non-Astra results justify experiments, not claims about Astra game quality. Live reference pages are distinguished from dated publications.

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

SkillsBench · Agent leaderboard, v1.1 [18: reading note]

SkillsInjector · Dynamic Skill Context Construction for LLM Agents [19: reading note]

SkillAlchemy · Open-World Agent Skill Creation [20: reading note]

Anthropic · Harness design for long-running application development [21: reading note]

SWE-Skills-Bench · Do Agent Skills Actually Help in Real-World Software Engineering? [22: reading note]

Verified keynote passage

Covers: Transcript notebook: Software Is Changing (Again)

The mirrored keynote passage discusses verification workload and inspectable outputs. The runnable-milestone application is our interpretation, with current game practice supplied by the Astra case study.

Andrej Karpathy · Software Is Changing (Again) [02: reading note]

OpenAI · Building games with Astra [04: reading note]

Unavailable second transcript

Covers: Transcript notebook: How I use LLMs

The prior transcript passage could not be reverified and is withdrawn from the evidence. Current context advice is supported independently by source 17. No timecodes or video-only claims are invented.

Andrej Karpathy · How I use LLMs [03: reading note]

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

Learning Godot and recording evidence

Covers: A game tutorial is a separate kind of evidence · A repeatable research procedure

The written tutorials establish the learning sequence; current export docs establish deployment constraints. The research worksheet and experiment procedure are original tools for checking transfer to your project.

Godot · Your first 3D game [09: reading note]

GDQuest · Your first 3D game with Godot 4 [10: reading note]

Godot · Exporting for the Web [11: reading note]

Read the review method and corrections

CHAPTER 06

Choose the right workspace

Pick the stack your agent can build and you can judge.

The game's delivery platform should influence the engine choice early. For a small game intended to live at a GitHub Pages URL, a browser-native stack offers a direct path from code changes to a playable link.

For an existing Godot or Unity project, preserving the engine may be more useful than switching to match a fashionable workflow.

Stack	What you get	What you still own	Choose it when...
TypeScript + Vite + Three.js	A code-oriented 3D rendering workflow and a small static-site deployment path. [12] [14]	Gameplay architecture, movement, collisions, input, UI, and asset management.	Your game is small, browser delivery matters, and your agent can edit and run code.
Babylon.js	A browser 3D engine with a broad feature set, including physics integration and inspection tools. [15]	Game design, integration choices, compatibility checks, and performance tuning.	You want more engine facilities within a JavaScript workflow.
Godot + GDScript	Scenes, nodes, a visual editor, and an established beginner learning path. [09]	Editor access or clear scene-file conventions; a tested web export configuration.	You prefer an engine workflow or plan native game releases too.
Unity	An editor-centered game project that can target supported web browsers. [16]	The installed editor and modules, asset import workflow, and web-build testing.	You already know Unity or need an existing project and its assets.

These are practical selection criteria, not benchmark results or a complete feature comparison. Test the cheapest risky part before committing to the stack. If the agent cannot reliably inspect the editor, begin with a tiny scene and an export test to discover that limitation immediately.

Our default: Three.js, TypeScript, and Vite

Signal Garden uses a fixed, slightly elevated camera, a flat arena, simple geometric obstacles, and a small number of moving objects. That scope makes a browser codebase a reasonable teaching choice.

Start with simple collision shapes and a readable state machine. If the design later depends on slopes, rigid-body interactions, or complex character movement, evaluate an appropriate physics solution before improvising one.

Use compatible stable package versions available when you start, commit the lockfile, and record the versions in the README. Use `npm ci` for a clean install from a matching committed lockfile; it fails on a package/lock mismatch rather than silently updating the lock.

Avoid copying an old tutorial's versions without checking compatibility. Demonstrate the chosen renderer on the target browsers. [42]

If you choose Godot instead

For a straightforward Pages release, begin by testing a GDScript project with Compatibility rendering and a single-thread web export. Godot's documentation states that web exports use WebGL 2, that Godot 4 C# projects currently cannot export to web, and that threaded exports require cross-origin isolation.

These are version-sensitive constraints. [11]

Prove an empty scene's exported build loads at the final project URL before building a full game. Keep the export files together.

The HTML entry can be named `index.html`; keep the other generated files' names unchanged. Godot also documents a PWA service-worker workaround for isolation, so the single-thread recommendation is a simple starting route, not proof that every threaded Pages deployment is impossible. [11]

Export naming: choose `index.html` in the export dialog itself. Do not rename the generated HTML or companion files afterward; their references depend on the original export name. [11]

The 30-minute feasibility check

As a timeboxed exercise, create a floor, one object, and a camera. Make the object move, build the output, and serve it under a repository subpath.

Record what failed. The timebox is a planning suggestion, not a promised completion time.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Stack features and teaching choice

Covers: Introduction · Our default: Three.js, TypeScript, and Vite

Official sources establish rendering, engine, hosting, and lockfile behavior. Choosing a browser codebase and simple collision for Signal Garden is an editorial scope decision, not a performance benchmark.

OpenAI · Building games with Astra [04: reading note]

Vite · Deploying a Static Site [12: reading note]

Three.js · README [14: reading note]

Babylon.js · Specifications [15: reading note]

Unity · Web browser compatibility [16: reading note]

npm · `npm ci` [42: reading note]

Godot export constraints

Covers: If you choose Godot instead

Checked WebGL 2 Compatibility rendering, Godot 4 C# restrictions, threading/isolation, PWA workaround, and the HTML entry-name exception. Test the installed version and actual host.

Godot · Exporting for the Web [11: reading note]

Feasibility exercise

Covers: The 30-minute feasibility check

A tiny export tests the documented deployment constraints in your environment. Thirty minutes is a suggested investigation budget, not a supported completion estimate.

Godot · Exporting for the Web [11: reading note]

Vite · Deploying a Static Site [12: reading note]

Read the review method and corrections

CHAPTER 07

Give the work a memory

The repository is the handoff between yesterday and tomorrow.

- Git records local history.
- GitHub holds a remote copy, review conversations, issues, and automation.
- GitHub Pages serves the published game or book.

Keeping those roles separate makes debugging easier: a local commit is not a push, a push is not a successful deployment, and a deployment is not proof that the game is playable.

Step 1: establish a clean starting point

In a new empty game folder, initialize Git on main. If the folder is already a repository, inspect its status and preserve the current work instead.

Install Git and GitHub CLI, and authenticate GitHub CLI before creating a remote. The following commands are an example for a new Signal Garden project; replace the account name first. [39] [40]

```
git init -b main
git status --short
# Create the README, .gitignore, game brief, and project files.
# Review ignore rules and all scaffolded files before staging.
git status --short
git add --all
git diff --cached --stat
git diff --cached
git commit -m "chore: establish the Signal Garden baseline"
gh repo create YOUR_ACCOUNT/signal-garden --public --source=. --remote=origin
git push -u origin main
```

In this new project, stage the complete scaffold, including index.html, configuration files, and required assets. The two diff commands show its file summary and actual staged content.

Set ignore rules for dependencies, local environment files, and build output as appropriate before staging. In an existing repository, stage only the intended paths so unrelated work stays separate.

A public repository and Pages site publish the selected files; keep credentials and unlicensed assets out of the commit. [39]

Step 2: keep the important decisions in files

```
signal-garden/
AGENTS.md # Optional short project conventions
.agents/skills/signal-garden-round-review/
SKILL.md # Optional; only if it adds proven value
docs/
game-brief.md # Player promise and constraints
acceptance.md # Observable completion conditions
decisions.md # Choices and reasons
progress.md # Current state and next step
playtests/ # Human observations and evidence links
src/
game/ # State, input, simulation
render/ # Scene, camera, visual feedback
ui/ # Instructions, HUD, result screens
public/assets/
credits.md # Asset provenance and license notes
tests/ # Focused behavior checks
.github/workflows/pages.yml
package.json
package-lock.json
```

This structure is a starting proposal, not a demand to create empty abstractions. A tiny build can begin with fewer modules. Separate systems when their responsibilities become clear, especially if simulation logic is difficult to test because it is mixed with rendering code.

Step 3: make a checkpoint per meaningful change

A useful commit says what behavior now works: feat: collect cells and power the beacon is better than update. Save a known working baseline before trying an uncertain idea.

This example improves readability while preserving Signal Garden's fixed camera. The switch reference documents creating its branch.

Create the named playtest file with your observations, and adapt the paths to the actual project. [39]

```
git switch -c experiment/camera-readability
# Adjust the fixed camera framing, then playtest the obstructed route.
git diff
git add src/render/camera.ts docs/playtests/camera-readability.md
git diff --cached
git commit -m "fix: keep the robot visible behind the north planter"
git push -u origin experiment/camera-readability
# Write docs/pr-camera.md with the problem, change, and checks first.
gh pr create --base main --head experiment/camera-readability --title "Improve fixed camera readability" --body-file docs/pr-camera.md
```

The `--body-file` option reads an existing file; it does not create the description for you. The PR should state the player-visible problem, change, and checks.

For a solo experiment, a branch and reviewed commit may be enough. Use a PR when review or CI gates add value. [41]

Step 4: leave a short continuation note

At the end of a session, update progress.md with the current commit, working behavior, known failure, launch command, and one next milestone. Anthropic's long-running-agent report uses progress artifacts and Git history to reduce the need for a new session to reconstruct previous work. [05]

When a bad change has already been shared, git revert creates a new commit that undoes it while preserving history. Inspect dependencies between commits and resolve any conflicts before completing the revert.

For an uncommitted experiment, show the exact diff and restore only the intended changes; unrelated work may be present. [39]

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Git, remote repository, and Pages setup

Covers: Introduction · Step 1: establish a clean starting point

Git references support initialization, complete staging, and reviewing staged content. GitHub CLI documents remote creation. Pages is a separate static publication step. Example commands require actual scaffold files and authenticated tools.

Git · Staging, differences, branches, and reverting [39: reading note]

GitHub CLI · gh repo create [40: reading note]

GitHub · What is GitHub Pages? [46: reading note]

Repository layout and handoff

Covers: Step 2: keep the important decisions in files · Step 4: leave a short continuation note

Scoped instructions and skill discovery have documented locations; the rest of the directory layout is a proposal. Progress artifacts come from the long-running-agent account. Revert records an undo, but conflicts and dependent changes still need review.

Anthropic · Effective harnesses for long-running agents [05: reading note]

OpenAI · Custom instructions with AGENTS.md [07: reading note]

OpenAI · Build skills [08: reading note]

Git · Staging, differences, branches, and reverting [39: reading note]

The fixed-camera experiment and PR

Covers: Step 3: make a checkpoint per meaningful change

Branch and PR options are documented. The paths and camera change are illustrative and must match actual files. The body file must be written before gh pr create reads it.

Git · Staging, differences, branches, and reverting [39: reading note]

GitHub CLI · gh pr create [41: reading note]

Read the review method and corrections

CHAPTER 08

Decide whether a skill adds value

Start with what the agent already has. Add what the task is missing.

The claim that project skills can be redundant is **partly supported**. A skill that repeats general coding advice may contribute little, while a specific workflow can supply knowledge the agent lacks.

Automatic skill selection is a capability to use and evaluate; it is not evidence that the agent always selects correctly or produces a better game.

For visual guidance, see the decision graph for choosing tools, skills, and hooks. The video chapter adds a current example: Remotion has deprecated its older documentation MCP in favor of skills, while Comfy exposes MCP tools that execute media workflows. [27] [37]

Astra update · 11 September 2026

OpenAI recommends reassessing accumulated instructions for Astra. Overspecified procedures and overlapping skill descriptions can impede useful work.

The practical change here is to make custom skills optional, with a specific reason to exist. This is guidance, not a measured Astra skill benchmark. [17]

Four different things people call “adding a skill”

1. **Make a skill available.** Install or expose it to the agent’s host. A file elsewhere on the internet is not automatically available.
2. **Let the agent select it.** Codex supports implicit matching between a task and an available skill’s description. Explicit selection is also supported. [08]
3. **Write a new skill.** Package a missing reusable procedure, references, or scripts. This is a separate decision from invoking an existing one.
4. **Write repository instructions.** AGENTS.md supplies scoped conventions; it is not an on-demand skill selector. [07]

Check the claims, one by one

Implicit selection also depends on host configuration. In Codex, `allow_implicit_invocation: false` disables automatic invocation for that skill.

A crowded initial catalog can shorten or omit entries under its context budget. Check availability, eligibility, and what was actually invoked before attributing a result to skill quality. [08]

Claim	Evidence-based verdict	Decision for Signal Garden
Every project needs a custom skill.	Not established. Software-task results show that many tested skills added no benefit. [22]	Begin with the brief, working tools, and existing available skills.
The agent can call the correct skill itself.	Implicit selection is documented; reliable selection on every task is not guaranteed. [08]	Check whether the relevant skill was available and actually used when diagnosing a failure.
Automatically calling a skill is always better than mentioning it.	No direct Astra comparison was found in the reviewed sources.	Try explicit selection when the right available skill was missed; compare outcomes.
Astra makes all skills obsolete.	Unsupported. Current Astra guidance still identifies uses for specialized workflows. [17]	Keep unique project knowledge or useful scripts when they earn their maintenance cost.
More skills mean more capability.	Selection and context interactions matter; more is not consistently better. [19]	Remove duplicate or overly broad optional entries and evaluate again.
AI-generated skills never help.	Too broad. SkillAlchemy reports gains from an evidence-led generation method on other models. [20]	Validate a generated procedure before adopting it; do not assume it improves Astra.

Why the studies seem to disagree

SWE-Skills-Bench found little average benefit from public software skills using Haiku 4.5. **SkillsBench** reports larger gains for curated skills across a broader task mix.

Those are different interventions, tasks, and agent configurations. They can both be true.

Neither answers whether this particular game-review skill helps Astra. [22] [18]

Our recommendation: do not create a new skill merely because a tutorial includes that step.

1. First let Astra attempt a bounded task with the information already available.
2. If it misses an existing procedure, inspect selection.
3. If it lacks a recurring piece of project knowledge, supply that knowledge.

A prompt correction, a documentation update, or a small script may be sufficient.

A compact AGENTS.md for the example game

- The task prompt names the immediate outcome.
- The game brief owns the game rules.
- AGENTS.md can map recurring decisions to the relevant documents and commands.

Keeping each fact in one place makes later changes easier to maintain.

Signal Garden - project instructions

Project references: docs/game-brief.md defines mechanics and scope; docs/acceptance.md defines observable behavior; docs/progress.md records the current milestone and known issues.

Consult the documents relevant to the task.

Preserve existing user work.

Use TypeScript, Vite, and Three.js with the committed lockfile.

Keep Y as up, movement on X/Z, and speed in world units per second.

Prefer a small clear module over a new abstraction without a present need.

Commands after scaffolding: npm run dev, npm run build, npm test.

Keep these scripts accurate in package.json and the README.

Keep simulation state separate from scene objects where that makes behavior easier to inspect.

Put tunable gameplay constants in one named configuration module.

Verify the behavior affected by a change using the documented checks and available tools.

Report checks that were not run.

Do not weaken acceptance conditions to hide a failure.

Use original or appropriately licensed assets and record attribution in public/assets/credits.md.

Do not commit credentials or local environment files.

At a completed milestone, update docs/progress.md with the revision, evidence, known issues, and next step.

Follow the user's existing authorization for commits, pushes, and publication; repository instructions do not grant new external permissions.

Adapt this sample only after its commands and paths exist. It points to documents by purpose instead of demanding that every edit reread every file. A typo correction and a change to round-state logic need different context.

An optional skill with project knowledge

The example below contributes Signal Garden's unusual boundary cases. It is useful only if this information is repeatedly missed or if the packaged procedure makes review more reliable. If the acceptance document and an available review skill already cover these cases, leave this sample uninstalled.

```
---
name: signal-garden-round-review
description: Check Signal Garden round-state regressions after changing beacon activation, damage, timers, pause, or restart.
---
```

Signal Garden round review

Optional example: install only after the referenced game documents exist and these checks add value beyond available review tools.

This is not a generic game-development skill.

Use docs/acceptance.md as the authority for current rules and test commands.

The cases below describe this book's initial design; update them when the game design changes.

Relevant boundary cases:

- If time expires or health reaches zero in the same simulation step as beacon activation, loss wins.
- Activation requires a fresh E press within two units after all three cells have been collected.
- Drone contact removes one health point, with a one-second damage cooldown.
- Pause freezes both the round timer and simulation.

Window blur clears held input and pauses an active round; returning focus requires explicit Resume.

- Restart from paused, won, or lost resets cells, health, timer, robot position, drone progress, cooldowns, effects, and held input, then enters playing for a fresh round.

Select the cases affected by the change.

Use deterministic checks for timing boundaries and actual player controls for the affected journey when browser tools are available.

Setup helpers may prepare a scene but cannot serve as proof that its controls work.

Output: revision, affected case, reproduction steps, expected/observed result, and evidence.

Mark unavailable checks as unrun.

Fix in-scope failures and recheck what the fix affects; finish when those checks pass.

Update docs/progress.md when this completes a milestone.

To try it, save the sample as `.agents/skills/signal-garden-round-review/SKILL.md` in the game repository and verify that your host lists it. Keep the referenced acceptance document current. A skill supplies guidance, not browser access; actual tools must also be available.

A small experiment for your Astra setup

This is a proposed experiment, **not a result measured for this book**. Use the same starting revision, Astra configuration, tool access, game brief, acceptance checks, and effort budget.

Keep required policies in place. Try these conditions in separate clean runs:

1. **Baseline:** current available skills, with no extra project skill and no forced skill selection.
2. **Selection check:** explicitly select the existing relevant skill. This tests whether routing contributed to the problem.
3. **Content check:** keep that same existing skill selected and add one explicitly selected, narrow custom skill. Compare against the existing-skill condition to test the additional information.

For each run, record skill availability and invocation if observable, accepted behaviors, regressions, manual interventions, elapsed time, and token cost when available. Score the same playable routes without

knowing the condition if possible.

Repeat several runs; retain the extra instructions only if their observed benefit justifies their cost. Once content helps, separately test whether its description allows reliable implicit selection.

An instruction audit

For an optional instruction, ask: which observed failure does it prevent, which fact does it add, and where is the evidence? Shorten, update, or remove it when those answers disappear. If an instruction causes a pause or conflict, identify its exact file and rule before changing it.

GitHub examples: choose by activity

These are **source-inspected candidates**, not installed or benchmarked winners. Match scope, dependencies, instructions, and actual results; stars do not measure suitability.

Activity / source	Useful contribution	Decision and limitation
Browser game: OpenAI web-game-foundations [49]	Simulation/render separation and engine choices.	Useful at project setup; too much structure for a tiny isolated repair. Preserve shared references.
Game introduction: remotion-dev/skills [50]	Markup, preview, captions, and export.	A strong first candidate for editing genuine game capture; still inspect the complete export.
Unclear video story: community Video Director [51]	Audience and creative framing.	Optional. Its mandatory interview can obstruct an already sufficient brief.
Generated video: Comfy-Org/comfy-skills [55]	Cloud commands and live template discovery.	Its cloud plugin targets Claude Code. The old skills folder is frozen; use Codex's documented MCP setup instead of copying that folder.

1. **A practical trial:** inspect the repository at a recorded revision, read the selected skill and its dependencies, then use the supported installer in the intended game or trailer project.

2. Remotion documents npx skills add remotion-dev/skills.

3. Check the installer's selected host, scope, and skills before applying it.

4. Verify that the host lists the skill and invoke one relevant entry for one bounded job.

5. Record what was actually loaded and the output.

6. Keep it only if that trial helps.

A skill file that references `../..references/` needs the corresponding directory structure; copying one Markdown file is incomplete.

For the first trailer, a focused request is enough: "Using the available Remotion composition guidance, assemble my three approved capture files into the ten-second storyboard, add readable captions, preview, render, and report any missing inputs." See the complete video workflow.

These examples have not been executed for the book.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Availability, invocation, and repository instructions

Covers: Introduction · Four different things people call “adding a skill” · A compact AGENTS.md for the example game

Codex supports eligible implicit matching and explicit selection, with a budgeted catalog. Repository conventions and skills have different roles. Remotion replaces a documentation integration; Comfy exposes action tools. These do not prove universal redundancy.

OpenAI · Custom instructions with AGENTS.md [07: reading note]

OpenAI · Build skills [08: reading note]

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

Remotion · Documentation MCP deprecation [27: reading note]

Comfy · MCP server for local and cloud workflows [37: reading note]

Evidence for conditional skill value

Covers: Check the claims, one by one · Why the studies seem to disagree

Benchmarks differ in tasks, models, skill construction, and discovery. Their findings cannot be pooled into an Astra performance claim. OpenAI’s Astra instruction audit is first-party guidance, not an independent gain measurement.

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

SkillsBench · Agent leaderboard, v1.1 [18: reading note]

SkillsInjector · Dynamic Skill Context Construction for LLM Agents [19: reading note]

SkillAlchemy · Open-World Agent Skill Creation [20: reading note]

Anthropic · Harness design for long-running application development [21: reading note]

SWE-Skills-Bench · Do Agent Skills Actually Help in Real-World Software Engineering? [22: reading note]

Optional game review skill and local comparison

Covers: An optional skill with project knowledge · A small experiment for your Astra setup

The skill’s round rules belong to the original game brief. The proposed comparison tests whether guidance adds value in your actual setup; it has not been run for this book.

OpenAI · Build skills [08: reading note]

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

SkillsBench · Agent leaderboard, v1.1 [18: reading note]

SWE-Skills-Bench · Do Agent Skills Actually Help in Real-World Software Engineering? [22: reading note]

Inspected GitHub candidates

Covers: GitHub examples: choose by activity

Reviewed the linked source instructions and repository layout. Suitability judgments are our assessment, not executed benchmarks or installation claims.

OpenAI · Web game foundations skill [49: reading note]

Remotion · Maintained GitHub skill collection [50: reading note]

Community example · Remotion Video Director [51: reading note]

Comfy-Org · Current skill and plugin repository [55: reading note]

Read the review method and corrections

CHAPTER 09

Design a game worth finishing

A small world can still have a clear identity.

Signal Garden is an original teaching design. You are a maintenance robot on a forgotten rooftop at dusk.

Three dormant cells can bring a garden's beacon back to life. A patrolling drone and a short countdown make the route a decision.

The desired feeling is calm exploration interrupted by readable moments of pressure.

The following values are **initial design targets**. They have not been measured in a completed game. They exist to make the brief concrete enough to build and then revise.

Design dimension	First-version decision	Observable check
Player objective	Collect three cells, return to the beacon, and activate it.	A first-time player can explain the objective after reading the start screen.
World	One flat 24 × 24 unit arena, with 1 unit treated as 1 metre; simple planters form two routes.	All cells and the beacon are reachable without crossing a collider.
Controls	WASD or arrows to move; E near the beacon; Escape to pause; a visible restart button.	Diagonal input is normalized; the restart resets all state.
Camera	Fixed elevated view with a stable orientation; no pointer lock.	Player, hazards, and objective remain legible during a full run.
Challenge	One drone follows a visible route. Contact removes one of three health points, with 1 second of invulnerability.	Standing in contact cannot remove all health in a single frame.
Round	90-second timer. Win by activating the charged beacon; lose at zero health or time.	Win and loss cannot both occur. Pausing freezes simulation and timer.
Visual language	Muted green stone, warm amber cells, a teal beacon, and a distinct drone silhouette.	Players can distinguish objects by shape and labels as well as color.
First release	Desktop browser with keyboard input, responsive menus, static GitHub Pages hosting.	The production URL loads, plays, ends, and restarts. Touch gameplay is a later scope decision.

Define the core loop in verbs

Observe → **choose a route** → **move** → **collect** → **evade** → **return** → **activate**. Each verb should have a visible consequence. A cell disappears once, the counter increases once, and the beacon's appearance changes when it is ready.

A hit has clear feedback and a short recovery period. Losing gives an immediate route back into play.

Decide the edge cases before they become bugs

- For the first implementation, check loss conditions before allowing beacon activation in each simulation step.
- If the timer reaches zero in that step, the result is a loss.
- E should act on a fresh key press, and activation requires the player to be within two units of the beacon.
- After a terminal result, movement, collection, damage, and the timer stop until restart.
- Restart is available from paused, won, and lost.
- It resets the player, cells, health, timer, drone progress, cooldowns, effects, and held input, then starts a fresh playing round.
- Losing window focus clears input and pauses an active round; returning focus requires an explicit Resume action.

These are our chosen rules, shared by the prompts and acceptance checks.

Pick a single convention for coordinates and units. In the proposed Three.js game, use Y as up and move on the X/Z plane.

Keep character speed in units per second. Decide these once in the brief so movement, camera, collision, and tests do not silently use different assumptions.

Give visual references a job

Collect or create references you have permission to use. Describe what each reference contributes: palette, camera height, silhouette, lighting, or UI density. Treat a reference as a design constraint, not an instruction to copy another game's characters, brand, or proprietary assets.

For Signal Garden, ask for a greybox image first, then a visual pass that keeps the same camera and geometry. Comparing like-for-like views makes the effect of lighting and material changes easier to judge. A more elaborate scene is not automatically a better scene.

Write the exclusions

For the first release, exclude multiplayer, procedural worlds, inventory systems, dialogue trees, and multiple levels. Add one only when it strengthens a playtested core loop enough to justify the extra work.

Check the mathematics behind the rules

For keyboard input (x,z), each component is -1, 0, or 1. Its length is $\sqrt{x^2 + z^2}$.

A diagonal (1,1) has length $\sqrt{2}$, so unnormalized diagonal motion is about 41% faster. Divide a nonzero vector by its length, then multiply by speed in metres/second and simulation time in seconds.

Zero input must stay zero. This is an elementary derivation, not a benchmark.

```
n = Math.hypot(x, z)
dx = n === 0 ? 0 : (x / n) * speed * dtSeconds
dz = n === 0 ? 0 : (z / n) * speed * dtSeconds
// At 4 m/s and 0.25 s: total displacement = 1 m.
// Diagonal components are each about 0.7071 m.
```

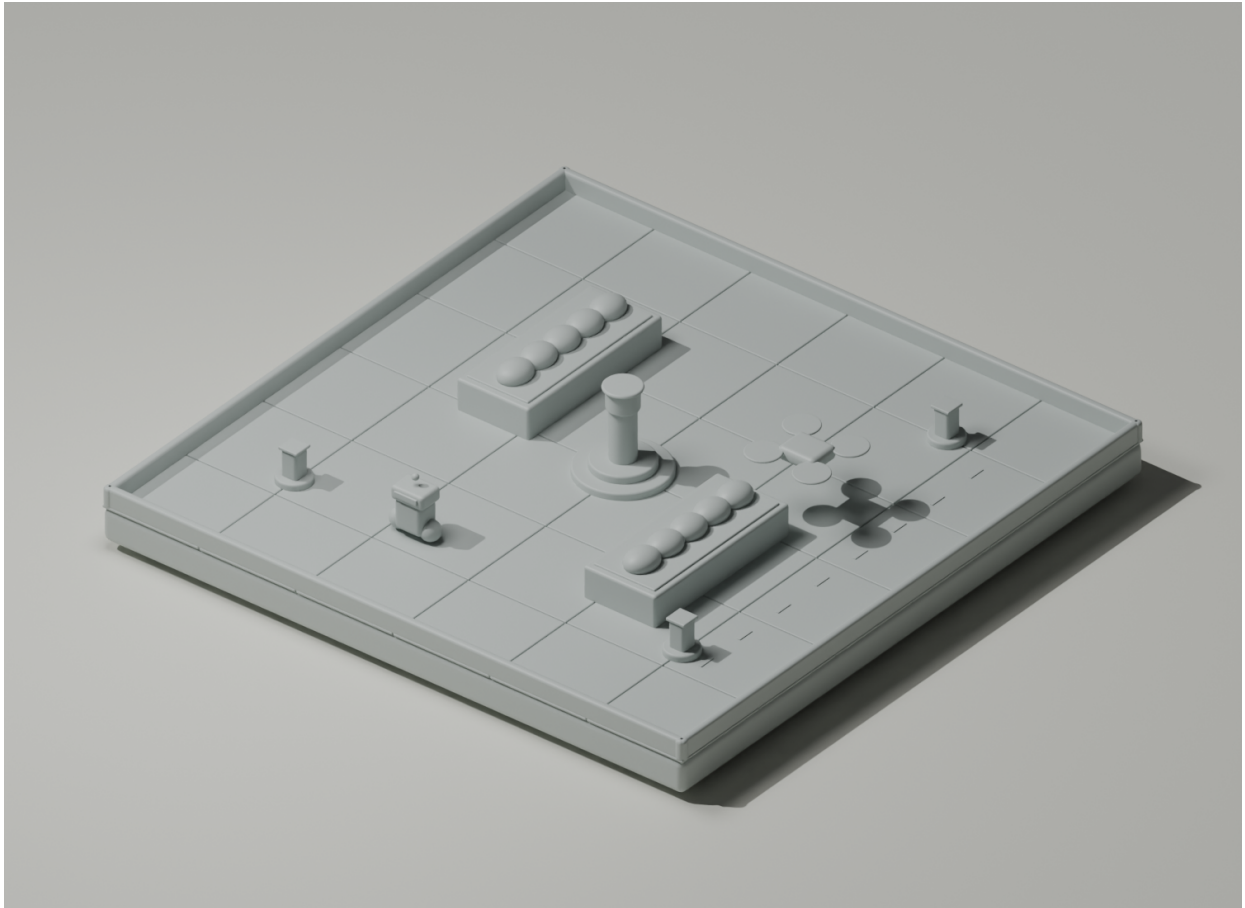
This formula is for digital keyboard input. A future analog stick should preserve partial deflection, typically clamping length to at most 1 after a dead zone instead of normalizing every nonzero input.

Collision resolution is a separate step: a large displacement can cross a thin obstacle. Use suitable bounded simulation steps or swept collision checks and test the relevant speeds; elapsed-time scaling alone does not prevent tunneling.

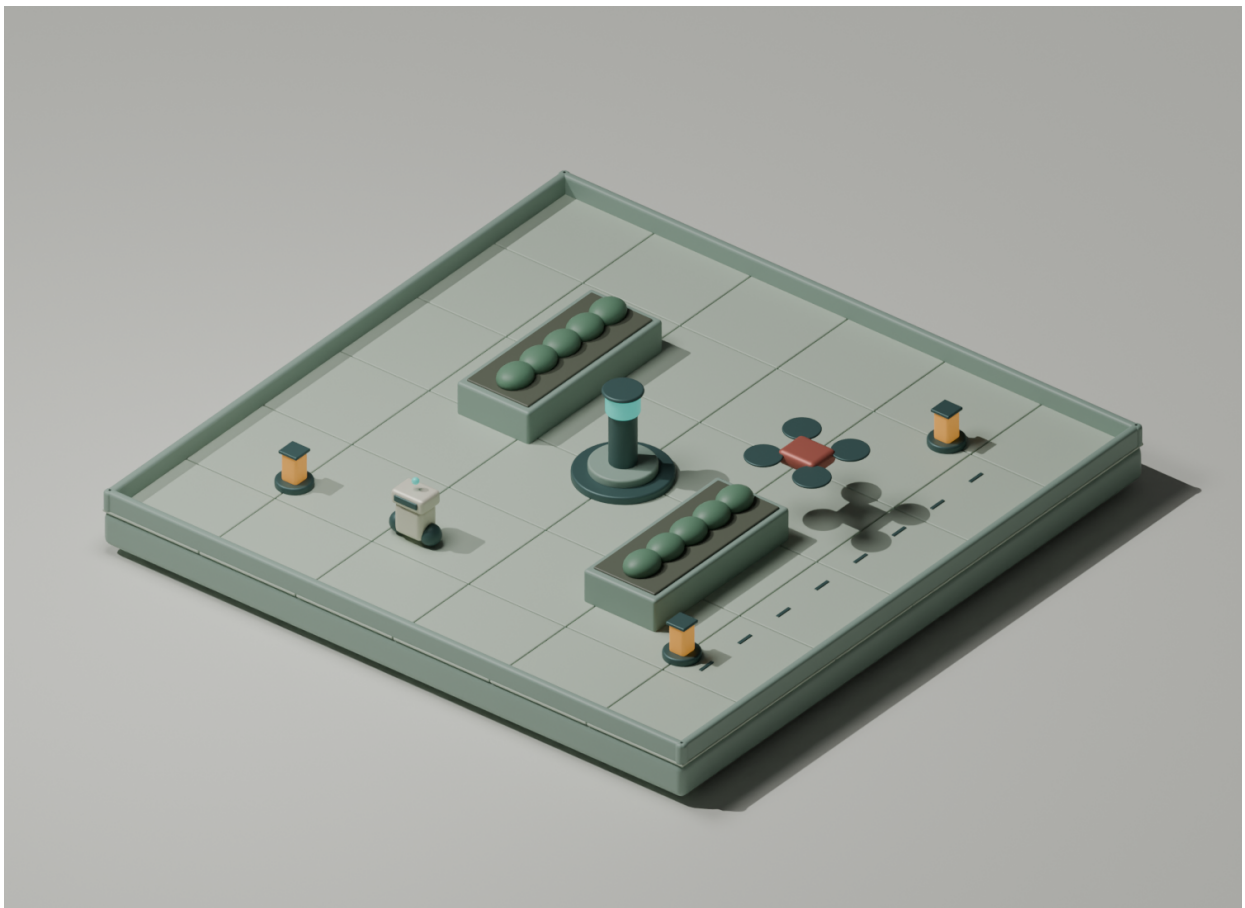
1. For this design, “within two units” means ground-plane **centre-to-centre distance ≤ 2** ; the equivalent squared-distance check is $dx^2 + dz^2 \leq 4$.
2. Use simulation time for damage cooldown and the round timer so pause freezes both.
3. Process damage and elapsed time, settle loss, and only then allow activation if still playing.
4. A single phase value plus centralized transition checks prevents contradictory terminal screens.

These are explicit implementation rules for our original design.

Blender study: compare the same scene



A · Geometry first. Check routes, scale, and silhouettes.



B · Material study. Camera, geometry and lights are unchanged; only object materials differ.

These original stills were rendered in Blender 4.5.3 LTS using the shared tool from the neighboring 3d_astra project. They are **concept illustrations**, not screenshots of a completed game.

The neutral lighting makes the material comparison readable; final dusk lighting remains a separate experiment. The dark floor dashes indicate an illustrative patrol guide, not an implemented route.

Rebuild with node scripts/render-illustrations.mjs --blender PATH_TO_BLENDER. The script maps game coordinates (x,y,z) to Blender (x,-z,y), preserving our Y-up convention.

Download the editable Blender scene or read the generation script. Blender documents background rendering and argument order. [54]

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Player promise, mechanics, and boundaries

Covers: Introduction · Define the core loop in verbs · Decide the edge cases before they become bugs · Write the exclusions

All game values, rules, loss precedence, pause/restart transitions, and exclusions are authored design targets. No external source establishes that three cells, 90 seconds, or this arena will be enjoyable. Verify implementation against this brief, then evaluate it through playtesting.

Original design: no external source establishes these targets. Evaluate them in the proposed game.

Using visual references

Covers: Give visual references a job

The Astra case study uses player experience and visual references. Assigning each reference a specific job and comparing a consistent camera view are our practical recommendations, not measured effects.

OpenAI · Building games with Astra [04: reading note]

Units, distance and transition mathematics

Covers: Check the mathematics behind the rules

Elementary vector derivation and explicit design conventions. Numerical checks verify the examples, not a completed game.

Original design: no external source establishes these targets. Evaluate them in the proposed game.

Reproducible Blender illustrations

Covers: Blender study: compare the same scene

Two actual renders share scene geometry, camera and lighting. The script and editable scene are supplied; gameplay is not implemented by these images.

Blender · Command line rendering, 4.5 manual [54: reading note]

Read the review method and corrections

CHAPTER 10

Make a prompt that makes a prompt

Ask for a better brief before asking for more code.

A meta-prompt is an instruction for drafting another prompt. Its value is the design work it forces into the open: audience, controls, constraints, missing information, completion conditions, and tradeoffs.

It cannot guarantee that the generated game will be good. You still need to review the resulting specification and play the implementation.

This step is optional. If the brief already expresses the game clearly, go directly to the compact execution prompt. Use the prompt designer when it resolves a real uncertainty, such as conflicting mechanics or an unsuitable export target. Creating another prompt is not itself evidence of improvement.

If you want a full development document like the RTS example, use the PLAN.md generator. It produces the plan directly; the designer below produces an execution prompt. Choose the deliverable you need instead of automatically chaining both.

Build the input packet

Give the prompt designer the one-page game brief, your target platform, installed tools, available assets, and any existing repository instructions. If the folder is empty, say so.

Include a simple reference image or sketch if it clarifies the camera or visual style. Label preferences as preferences and hard constraints as constraints.

For Signal Garden, the important facts are a small desktop browser game, keyboard input, a fixed camera, one arena, three collectibles, one hazard, a complete win/loss/restart loop, and static hosting. “Make something amazing” adds less useful direction than any of those facts.

The prompt designer

Design an execution prompt for GPT-6 Astra

You are a game-design and coding-workflow editor.

Produce a practical execution prompt for GPT-6 Astra running inside a coding agent.

Your task in this conversation is prompt design, not game implementation.

INPUT BRIEF

Use the game brief below or the attached docs/game-brief.md.

If details are missing, use the Signal Garden defaults and label them as assumptions:

- Player: a maintenance robot restoring a rooftop beacon.
- Loop: collect three energy cells, avoid one patrolling drone, return to the beacon, activate it.
- Scope: one flat arena, desktop browser, keyboard controls, one complete round with win, loss, pause, and restart.
- Stack preference: TypeScript, Vite, Three.js; GitHub Pages publication after the destination is explicitly authorized.
- Art: original geometric assets, muted green stone, amber cells, teal beacon.
Readable silhouettes and camera first.
- Constraints: no backend, accounts, multiplayer, paid APIs, or unlicensed assets.
Keep the first release small.

My project-specific changes, existing files, reference images, installed tools, target hardware, and available budget:
[Add these here, or retain the defaults and mark unknowns.]

PROCESS

1.
Separate hard requirements, preferences, assumptions, and genuinely blocking unknowns.
Ask at most three focused questions only if a wrong assumption would materially change the outcome.
Continue independent prompt drafting where possible.
2.
Compare only approaches that could change a decision for this brief.
Candidates include a one-shot prototype, specification first, tutorial guided, incremental implementation, visual playtesting, and parallel specialists.
Explain the relevant tradeoffs concisely; do not invent benchmark results or completion-time guarantees.
3.
Recommend a minimal combination and a stack the available tools can actually operate.
Preserve an explicit engine choice.
Check primary documentation before making version-sensitive technical claims; disclose unavailable browsing.
4.
Draft an execution prompt with the player promise, controls, camera, world rules, edge cases, asset policy, verification, and completion conditions.
If these already live in accurate repository documents, reference them rather than duplicating them.
Use milestones as outcome checkpoints, not a mandatory itinerary for every edit.
Prefer available skills; propose a new one only for a specific recurring gap.
5.
Include an instruction to inspect existing files before editing, preserve user work, make reasonable routine decisions, and continue through authorized milestones.
Treat external publication as a separate permission boundary unless already authorized.
6.
Specify only useful evidence: focused logic checks, real-control interaction checks, visual inspection, and human playtesting where judgment is required.
Do not claim that a passing build proves the game works.
7.
Review the draft for contradictions, requirements without an observable check, hidden extra scope, unavailable tools, and vague

adjectives.

Revise once to fix those specific issues.

RETURN

A.

Assumptions and any blocking questions.

B.

A concise comparison and recommendation.

C.

The final copyable execution prompt, with unresolved fields clearly marked.

D.

A short acceptance checklist and a first playtest plan.

E.

A compact explanation of the defects corrected in the review pass.

Keep the final execution prompt self-contained for an empty folder, or explicitly dependent on named existing documents.

Use direct instructions and concrete player behavior.

Do not substitute another model for GPT-6 Astra, ask for private chain-of-thought, or present a self-assigned quality score as evidence of game quality.

Return the final prompt with an editable parameter block first, then a goal, grouped tasks/constraints, and observable completion checks.

List alternative engines/tools as choices, select only one, and make the detailed instructions conditional on that selection.

Keep tuning values separate from requirements, and check that changing a parameter cannot leave a conflicting hardcoded value in the body.

Review the result before executing it

Check whether the generated prompt carries the game's actual identity. If the output could describe any generic action game, the input probably needs a clearer player experience.

Then check whether it introduces hidden scope: user accounts, online leaderboards, paid services, generated assets, multiplayer, or a large framework you did not request.

Question	Weak answer	Useful answer
What is the game?	A beautiful 3D adventure.	A robot restores a rooftop beacon by collecting three cells while avoiding a drone.
What should the agent do now?	Plan carefully.	Inspect the workspace, record assumptions, then build and verify the first complete loop.
How will we judge it?	Make it polished and bug-free.	Exercise start, movement, collection, pause, win, loss, and restart; inspect camera and readable feedback.
When does it stop?	When it looks good.	When the scoped acceptance checks pass, known issues are stated, and the authorized deliverables exist.

Use one refinement pass with a purpose

Ask a reviewer to find contradictions, unavailable tools, requirements with no observable test, and unrequested complexity. Revise those specific defects. Repeatedly asking a model to “make this prompt even better” can produce a longer prompt without a better design.

Version the input brief and the resulting prompt together. If you later change the controls or release platform, update the source brief first, then regenerate the affected parts. Keep a short change note so you can connect an implementation difference to an instruction difference.

A useful prompt review

Underline every adjective in your generated prompt. For “smooth,” “clear,” “fast,” or “beautiful,” add either an example to compare against or a concrete observation to collect. Keep subjective judgments visible rather than pretending every quality can be reduced to a number.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Input packet and prompt designer

Covers: Introduction · Build the input packet · The prompt designer

The downloadable meta-prompt is authored for Signal Garden. Current guidance motivates explicit intent and relevant context. No reviewed experiment proves a meta-prompt beats a direct request on Astra.

OpenAI · GPT-6 Astra prompting best practices [01: reading note]

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

OpenAI · Building games with Astra [04: reading note]

Reviewing and refining a generated prompt

Covers: Review the result before executing it · Use one refinement pass with a purpose · A useful prompt review

The review questions, one-pass exercise, and versioning procedure are proposed safeguards against contradictions and excess process. Source accounts motivate revisiting scaffolding; they do not prescribe this exact checklist.

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

Anthropic · Harness design for long-running application development [21: reading note]

Read the review method and corrections

CHAPTER 11

The Astra master prompt

Use a compact request when the repository already holds the detail.

This is an original worked execution prompt for the Signal Garden brief. It is a manually authored example of the output the previous chapter asks for, **not a transcript of an experiment or a benchmarked best prompt**. Use it in a coding-agent session with the intended game folder open and GPT-6 Astra selected.

Before pasting, confirm the agent can edit the folder, run the package manager, and inspect a browser if you expect automated playtesting. The prompt allows reasonable local implementation choices and asks the agent to continue through the scoped milestones.

Publication is handled separately in the deployment chapter so the destination can be explicit.

Option A · the brief already exists

This is the default to try when docs/game-brief.md and docs/acceptance.md accurately describe the game. It keeps the request short by referring to those files. It has not been benchmarked against the longer version; compare them using the refinement experiment.

```
## Edit these parameters
BRIEF_PATH: docs/game-brief.md
ACCEPTANCE_PATH: docs/acceptance.md
PROGRESS_PATH: docs/progress.md
```

Use the document paths in the parameter block throughout this request.

Implement Signal Garden from the repository brief

Use GPT-6 Astra in this game repository.

Build the playable desktop-browser release specified in BRIEF_PATH and satisfy ACCEPTANCE_PATH.

The brief owns the mechanics; PROGRESS_PATH records completed work and remaining issues.

If a required document is missing, identify it before inventing a replacement design.

Inspect the current implementation and preserve user work.

Make routine technical choices within the brief and continue through the authorized scope.

Use available skills when relevant; do not create or install a custom skill just to start this task.

Keep questions limited to material ambiguities or missing authorization.

Complete the gameplay, review the running result through real controls when tools permit, and collect evidence for the affected acceptance conditions.

Resolve in-scope failures and report anything untested.

Treat visual judgment and human playtesting separately from automated correctness.

Deliver the working game, reproducible build, brief progress note, and release instructions.

Local commits are authorized.

Follow any existing authorization for pushing and publication; otherwise prepare the release for review with the intended destination stated.

Report the delivered revision, verification results, and remaining limitations.

Option B · start from an empty folder

The longer bootstrap prompt below includes the rules because the agent has no project brief yet. Its length comes mainly from the game specification. After the agent records those rules in files, use focused requests rather than repeatedly pasting the complete bootstrap text.

Edit these parameters

PROJECT_FOLDER: current selected game folder

PROJECT_NAME: Signal Garden

RULE_CHANGES: none

Apply PROJECT_NAME as the game name and RULE_CHANGES as proposed changes to the defaults below.

Reconcile existing agreed requirements before changing them.

Work in PROJECT_FOLDER; keep the established engine and available tools.

The Three.js-specific commands below apply to the Three.js route; use the compact repository prompt after agreeing another engine.

Build PROJECT_NAME from scratch with GPT-6 Astra

Work in the current project folder as a game developer.

Inspect the folder, Git status, applicable instructions, and available tools first.

Preserve existing user work.

If this is an empty folder, create the complete project described below.

Deliver a small, coherent, playable desktop-browser 3D game.

Implement it and run the appropriate checks; continue through the milestones instead of ending with a plan.

Make reasonable routine implementation choices and record assumptions.

Ask a focused question only if missing information would materially change the result or requires authorization that is not already present.

PLAYER EXPERIENCE

PROJECT_NAME is a quiet rooftop at dusk.

The player is a maintenance robot restoring a beacon.

Collect three energy cells, avoid a patrolling drone, and return to activate the beacon before the round ends.

The world should feel abandoned but recoverable, with clear routes and warm light from restored machinery.

SCOPE AND RULES

- One flat 24 by 24 unit arena.

Treat one unit as one metre; Y is up, with movement on X/Z.

- Use a fixed elevated camera with a stable orientation and no pointer lock.

Keep the player and routes readable.

- WASD or arrow keys move.

Normalize diagonal input.

E activates the beacon on a fresh press within two units, only after all three cells are collected.

Escape pauses/resumes.

Provide visible start, pause, resume, restart, and mute controls.

- The player begins with three health points and 90 seconds.

One drone follows a deterministic visible patrol.

Contact causes one damage, followed by one second of invulnerability.

- Each cell can be collected once.

Display the cell count, health, remaining time, and beacon readiness.

- Use title, playing, paused, won, and lost states.

Pause freezes simulation and timer.

Losing all health or time ends the round.

Evaluate loss conditions before beacon activation within a simulation step so simultaneous expiry and activation result in loss.

- After win or loss, freeze gameplay.

Restart is available from paused, won, and lost: reset player position, cells, health, timer, drone route, cooldowns, effects, and input state, then enter playing for a fresh round.

Window blur releases held inputs and pauses an active round; returning focus requires explicit Resume.

- Exclude multiplayer, accounts, procedural worlds, inventory, dialogue, and extra levels from this release.

TECHNICAL DIRECTION

Prefer TypeScript, Vite, and Three.js unless the existing project explicitly selects another engine.

Use available stable packages and commit the lockfile.

Record the actual versions.

Check official documentation for APIs that are uncertain or version-sensitive.

Keep a small readable structure with separable input, simulation/state, rendering/camera, and UI responsibilities.

Keep tunable values in one configuration module.

Use simple collision shapes appropriate to a flat arena; explain a physics dependency if one becomes necessary.

Use elapsed time correctly and prevent large resume-time steps.

Create accurate npm scripts for dev, build, and focused tests.

Create docs/game-brief.md, docs/acceptance.md, docs/progress.md, and a concise README with setup, controls, target platform, and limitations.

Add appropriate ignore rules.

VISUAL AND AUDIO DIRECTION

Start with readable greybox geometry.

Then use an original geometric robot, muted green stone and planters, amber energy cells, a teal beacon, and a drone with a distinct silhouette.

Keep routes visible; avoid excessive bloom, particles, or camera shake.

Shape and text should reinforce color-coded signals.

Respect reduced-motion preferences for decorative motion and hit effects.

Use original or appropriately licensed assets.

Record sources and attribution.

Add modest feedback for collection, damage, readiness, victory, and loss.

Begin audio only after user interaction and provide mute.

Keep menus responsive and keyboard accessible.

This first version targets desktop keyboard play; do not claim mobile gameplay is supported unless implemented and tested.

OUTCOME CHECKPOINTS

Use these to track completion.

Adapt the implementation order when dependencies or the existing project justify it.

M0: Scaffold, document setup, build successfully, and display a minimal scene.

M1: Make movement, boundaries, camera, and pause reliable.

M2: Implement cells, beacon activation, HUD, timer display, win, and complete restart from pause or win.

Loss remains for M3.

M3: Implement drone, damage cooldown, timer loss, and health loss.

M4: Apply coherent art and sound; inspect readability and feedback.

M5: Test production output under /signal-garden/, fix important failures, and prepare the release workflow.

At each milestone, keep the project runnable, update the progress note, and make a local commit after reviewing the diff and running the relevant checks.

Local commits are authorized by this prompt.

Treat a milestone as a checkpoint and continue to the next one without an unnecessary permission pause.

VERIFICATION

Run focused checks for single collection, normalized movement, damage cooldown, pause, terminal-state exclusivity, boundary timing, and complete restart.

Use real controls in the running browser for the main interaction paths when browser tools are available.

Inspect screenshots for camera and UI quality, and observable state for the rules.

Deterministic setup helpers may prepare test scenes but must not substitute for journey tests through player input.

Check start, collection, a win, both loss conditions, pause/resume, and repeated restart.

Build and test the production output at its repository subpath.

Check asset loads and runtime errors.

Name the actual browser and device used.

If measuring performance, record the device, viewport, scene, warmup, and sample duration; do not invent a frame-rate result.

If a tool is unavailable, complete independent checks, state what was not run, and provide the shortest remaining manual verification sequence.

Do not mark untested conditions passed.

After a fix, rerun checks justified by the changed behavior; avoid redundant broad testing once the relevant checks pass.

WORKING AGREEMENTS

Keep communication concise and connect changes to player-visible outcomes.

If instructions conflict, identify the relevant file and rule and resolve them according to the applicable instruction hierarchy.

Use available skills when relevant; no new custom skill is required.

If the session permits parallel specialists, use them only for independent tasks with clear ownership and one integration owner.

Otherwise continue with one agent.

COMPLETION

Deliver the runnable local game, its source and lockfile, accurate setup instructions, acceptance results, asset credits, reviewed local commits, and a prepared GitHub Pages workflow.

Report what is complete, what was actually tested, and what needs human playtesting.

Publish only if this session already authorizes the specific account/repository and public release; otherwise finish the reviewable release preparation and request that final authorization.

Why these parts are present

- The player promise gives the work direction.
- The coordinate convention prevents integration mistakes.
- The terminal-state rules settle edge cases.
- The milestones make partial progress recoverable.
- The evidence requirements make completion reviewable.
- The exclusions keep the first game small enough to finish.

The model-specific portion is short: state how much initiative is expected, how to handle instruction conflicts, when delegation helps, and what test scope is appropriate. Those concerns are reflected in OpenAI's current Astra guidance. The game rules and target values are our design choices. [01]

Steer the active task with evidence

Do not restart from scratch after every disappointing result. A focused correction should name the current revision, the observed problem, the desired behavior, and the acceptable scope of change. Keep the overall objective intact while correcting the specific failure.

Repair one observed game problem

Continue the existing game and preserve the overall brief.

Revision / build: [actual commit or build URL]

Observed problem: [describe what happened]

Reproduction: [starting state and exact input sequence]

Expected behavior: [what the player should see or experience]

Evidence: [screenshot, state snapshot, error, or playtest note]

Allowed scope: [systems that may change and constraints to preserve]

Reproduce the issue before changing code.

Inspect the smallest relevant path, explain the likely cause briefly, and implement a targeted fix.

Avoid unrelated features or rewrites.

Verify the failing sequence and any affected neighboring behavior.

Show the evidence for the result and identify checks that were not run.

Update the progress note and follow this session's commit authorization.

If the agent cannot reproduce the problem, ask it to inspect the simplest relevant scene or add a narrowly scoped observation point. If it proposes another broad rewrite, require an explanation of why a local change cannot solve the problem and a recovery point before proceeding.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Compact and bootstrap execution prompts

Covers: Introduction · Option A · the brief already exists · Option B · start from an empty folder

Both templates are manually authored and unbenchmarked. The compact prompt relies on existing accurate project files; the bootstrap carries the original game rules because those files do not yet exist.

OpenAI · GPT-6 Astra prompting best practices [01: reading note]

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

Working agreements and evidence-based repair

Covers: Why these parts are present · Steer the active task with evidence

Astra documentation supports initiative and proportionate testing; the game case study supports observable state and real controls. The coordinate rules, milestone order, and repair template remain project choices.

OpenAI · GPT-6 Astra prompting best practices [01: reading note]

OpenAI · Building games with Astra [04: reading note]

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

Read the review method and corrections

CHAPTER 12

From empty folder to game

Finish the loop before expanding the world.

The following sequence is a proposed implementation plan. Each milestone names a runnable outcome and evidence; adapt the order when dependencies or an existing project justify it. Passing a milestone is a checkpoint, not a request for another approval when the remaining work is already authorized.

M0 · Make the project reproducible

Inspect the folder and any instructions. Scaffold the minimal application, record the exact runtime and package versions, and establish the build command.

Add the brief, acceptance list, and launch instructions. Start the application and verify the simplest scene loads.

If package installation or browser access is blocked, report the precise limitation and complete independent work.

Evidence: a fresh setup can follow the README; the build exits successfully; a floor, camera, and player placeholder appear. Do not spend this milestone on a final title screen or an elaborate component library.

M1 · Make movement legible

Implement keyboard input, a flat arena, collision boundaries, and the fixed elevated camera. Normalize diagonal motion.

Browser animation callbacks usually follow display refresh and pause in background tabs: use elapsed time, clear held input on blur, pause the round, and require explicit resume without a large time step. [43]

Evidence: real keyboard input moves the player in the intended directions; boundaries stop the player; diagonal speed matches straight-line speed; the camera shows the playable space. Pause or tab switching does not cause the player to teleport through geometry on return.

M2 · Close the game loop

Add the three cells, beacon activation, counter, timer, and win screen. Keep one authoritative game-state value, such as title, playing, paused, won, or lost. Centralize transitions so UI, audio, and simulation do not each invent a separate idea of whether the round has ended.

Evidence: each cell increments the counter once; the beacon requires all three; activation at the wrong distance does nothing; winning stops the timer and movement. Restart from pause or win resets the round into playing. This is a playable win-loop slice with placeholder art; challenge and both loss conditions remain for M3.

M3 · Add one readable source of pressure

Add the patrolling drone, health, invulnerability window, and time-based loss. Show a readable route and a visible hit response.

Begin with deterministic patrol behavior so a bug can be reproduced. Introduce randomness only if it benefits the design, and keep a seed for test runs if you do.

Evidence: collision causes the intended damage; sustained overlap respects the cooldown; zero health loses exactly once; zero time loses; pause freezes patrol and cooldown progression as defined; loss then restart produces a clean round.

M4 · Give the world its identity

Replace placeholders with a coherent robot silhouette, low geometric planters, a distinct drone, warm cell highlights, and a readable beacon. Initialize or resume audio after user interaction, handle playback being blocked, and provide mute.

Use the device's reduced-motion preference to reduce optional motion; a WebGL scene must apply that preference in its animation code as well as its CSS interface. [44] [45]

Evidence: the robot is visible against the floor; the hazard is distinct from collectibles; the HUD remains legible at the target window sizes; sound and pause controls work; reduced-motion preferences avoid unnecessary shake and flashing. Recheck the core loop after asset changes.

M5 · Prove the release path

Build production output, serve it under /signal-garden/, and use the exported game for a complete run. Record a human playtest and fix the most consequential issue. Prepare the public repository and Pages workflow, then publish when the destination and publication are authorized.

Evidence: the final URL serves the expected revision, asset requests succeed, a win and a loss are possible, and restart works repeatedly. The README states controls, build instructions, asset credits, tested browsers, and remaining limitations.

When a milestone fails

Stop expanding the feature set. Reproduce the failure, identify the smallest causal change, fix it, and rerun the affected checks. Update the milestone status honestly, then continue.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Reproducible setup and stable motion

Covers: Introduction · M0 · Make the project reproducible · M1 · Make movement legible

A committed lockfile and elapsed-time animation have documented behavior. M0-M1 define proposed checkpoints, including explicit pause/resume policy for background time.

[OpenAI · Building games with Astra \[04: reading note\]](#)

[npm · npm ci \[42: reading note\]](#)

[MDN · requestAnimationFrame \[43: reading note\]](#)

Playable win slice, then complete loss rules

Covers: M2 · Close the game loop · M3 · Add one readable source of pressure

M2 is deliberately incomplete: it establishes the win loop, timer display, and restart. M3 adds drone/health pressure and both loss conditions. Compare the final implementation with the game brief's single authoritative rules.

Original design: no external source establishes these targets. Evaluate them in the proposed game.

Art, audio, motion, and release

Covers: M4 · Give the world its identity · M5 · Prove the release path

Browser docs support autoplay handling and reduced-motion detection; Pages/Vite docs support the release path. Visual targets, human playtesting, and milestone acceptance are our design recommendations.

[OpenAI · Building games with Astra \[04: reading note\]](#)

[MDN · Autoplay guide for media and Web Audio APIs \[44: reading note\]](#)

[MDN · prefers-reduced-motion \[45: reading note\]](#)

[Vite · Deploying a Static Site \[12: reading note\]](#)

[GitHub · Using custom workflows with GitHub Pages \[13: reading note\]](#)

Read the review method and corrections

CHAPTER 13

Verify the game you can play

A build log cannot tell you whether movement feels right.

Use three layers of evidence.

- **Logic checks** cover rules that should be deterministic.
- **Interaction checks** exercise the game through its controls.
- **Human playtests** judge clarity, feel, challenge, and appeal.

Each layer answers questions the others cannot.

Keep the critical acceptance list short

Behavior	Test action	Expected evidence
Movement	Move straight, diagonally, and against each boundary.	Consistent speed and collision; no escape or camera obstruction.
Collection	Enter and leave each cell trigger several times.	Exactly one count increment per cell.
Damage	Touch the drone, remain in contact, then move away.	Damage observes the 1-second invulnerability policy.
Pause	Pause while the drone is moving; resume after a wait.	Timer and simulation remain frozen, then continue without a jump.
Win	Collect all cells; activate within range.	One win transition, frozen gameplay, working restart.
Loss	Reach zero health; separately let time expire.	One loss transition in each case, then a clean restart.
Boundary timing	Attempt activation on the step the timer reaches zero.	The brief's loss-before-activation rule holds.
Restart	Restart after win, after loss, and after pausing.	Player, cells, health, timer, patrol, cooldowns, effects, and held input reset; phase becomes playing.
Production	Open the deployed repository URL and play a round.	No missing assets or runtime errors; controls work in the published build.

Make invisible state inspectable

In a test-enabled build, expose a read-only state snapshot containing the round state, player position, remaining cells, timer, health, and drone position. Use deterministic scene fixtures to reproduce a boundary condition. Avoid a hidden test shortcut that marks a condition passed without exercising the gameplay rule.

OpenAI's Astra game case study distinguishes named scene setup from real-control journey tests. Both are useful: a fixture makes a failure cheap to reproduce, while an input sequence checks whether a player can actually reach that state. [04]

```
// Proposed shape of a read-only test snapshot, not shipped game code.
{
  phase: "playing",
  player: { x: 2.0, z: -3.0, health: 3 },
  cellsCollected: 1,
  secondsRemaining: 62.4,
  beaconReady: false,
  drone: { x: -4.0, z: 1.0 }
}
```

Define a performance observation

Pick a named device, browser, window size, and quality setting. For an initial experiment, aim for a 95th-percentile frame time under 33 ms during a 60-second normal-play sample after warmup.

This is a proposed target of roughly a 30-fps frame budget, not a measured result or a universal requirement. Record loading stalls separately; a smooth average can hide disruptive pauses.

State what the measurement means. Intervals between requestAnimationFrame callbacks describe main-thread callback cadence; they are not a GPU render-time measurement.

Keep the tab foregrounded, record the sampling method, and treat background pauses separately. This distinction matters when comparing two runs. [43]

If performance misses the target, collect evidence before reducing visual quality. Compare draw calls, geometry, shadows, resolution, and update work.

Change one likely bottleneck, then repeat the same sample. Do not claim an optimization worked because the code became shorter.

Measurement limit: callback intervals measure main-thread callback cadence. They do not establish that every rendered frame was displayed.

State the percentile estimator (for example, sorted sample at index $\text{ceil}(0.95 \times N) - 1$), sample count, and long stalls as well as the target. The percentile convention is our proposed reporting choice.

Run a small human playtest

Give someone the build and ask them to play without coaching. Observe whether they identify the objective, understand the controls, notice damage, and discover restart.

Then ask where they felt uncertain and what they wanted to try next. A small playtest reveals specific problems; it does not establish a population-level success rate.

Build / commit:

Device, browser, window size:

New or returning player:

Could explain the objective? [observation]

First confusing moment: [what happened]

Win/loss/restart observed: [which, with evidence]

Control feel: [player's own description]

Highest-impact issue:

Next change and the observation it should improve:

Separate “I ran it” from “I tested it.” A credible report names the actions, the result, and the revision. If the agent cannot inspect the game, preserve that limitation in the report instead of turning an unrun check green.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Rules, real controls, and visible state

Covers: Introduction · Keep the critical acceptance list short · Make invisible state inspectable

The Astra example distinguishes deterministic setup from real-control journeys. The table and snapshot are original acceptance tools for the game brief's design, not evidence of a game already tested.

OpenAI · Building games with Astra [04: reading note]

Performance observation

Covers: Define a performance observation

MDN establishes callback timing and background behavior. The 95th percentile, 33 ms target, and 60-second sample are proposed choices. Callback intervals must not be reported as GPU duration.

MDN · requestAnimationFrame [43: reading note]

Human playtest worksheet

Covers: Run a small human playtest

The worksheet asks for observed experience and evidence. Implementation accounts motivate evaluation, but a small playtest does not establish population success rates or an objective fun score.

OpenAI · Building games with Astra [04: reading note]

Anthropic · Harness design for long-running application development [21: reading note]

Read the review method and corrections

CHAPTER 14

Refine the idea and the process

Improve the weakest link, then keep what you learned.

Use the Mermaid improvement loop and decision record to make the cause, experiment, and result visible.

When a first build disappoints, resist adding five new features. Decide what kind of failure you have.

- A wrong mechanic needs a clearer rule.
- A correctly implemented but dull mechanic needs a design experiment.
- A broken control needs a reproduction.
- A forgotten convention needs better repository context.

A worked prompt refinement

The examples below are illustrative revisions, not measured test results.

Version	Instruction	Why it improves the next decision
P0 · vague	Make a good 3D game with a robot.	Almost every design and completion criterion is unspecified.
P1 · scoped	Make a browser game in one rooftop arena. A robot collects three cells and returns to a beacon.	Defines the world, objective, and minimum complete loop.
P2 · testable	Use WASD/arrows, a fixed elevated camera, one drone, three health points, and a 90-second round. Verify win, loss, pause, and restart.	Provides concrete rules and visible acceptance conditions.
P3 · situated	At this revision, the north planter hides the robot. Keep the current rules. Adjust camera or occlusion treatment and compare the same route before and after.	Uses evidence from the current build to bound the change.

P0-P2 add specification detail; P3 is a different repair task at an existing revision, not a universally better bootstrap prompt.

Use a rubric to expose disagreements

Score each dimension from 0 to 2: **0** means missing or broken, **1** means present but inconsistent, and **2** means demonstrated under the stated test. Add a note and evidence for each score. The score is a conversation aid, not an objective measure of fun.

- **Clarity:** the player understands the goal and current state.
- **Control:** movement responds consistently and the camera supports it.
- **Completion:** the core loop includes win, loss, pause, and reliable restart.
- **Feedback:** collection, damage, and beacon readiness are perceivable.
- **Challenge:** the route and hazard produce a readable decision.
- **Coherence:** visuals, sound, and UI support the same mood.
- **Reliability:** the deployed build performs acceptably on the named target.
- **Maintainability:** a new session can reproduce the build and find the next change.

Compare approaches with a fair experiment

If you want to know whether your meta-prompt helps, compare it against a direct prompt from the same clean starting revision. Keep the game brief, model, tools, target platform, and effort budget the same. Record which prompt was used and the actual cost, elapsed time, manual interventions, accepted milestones, and regressions.

Judge the playable outputs without looking at which prompt produced them if possible. Repeat more than once before trusting a difference; agent runs can vary.

Do not use a single lucky output to declare a universal winner. If one approach uses extra agents or a much larger budget, report that difference instead of attributing the result entirely to wording.

Keep only improvements that still help

- A repeated project convention may belong in AGENTS.md.
- A recurring procedure may merit a skill when existing tools and documentation do not already cover it.
- Keep a one-off fix in the task's notes.
- Update the game brief when playtesting changes the design.
- Avoid copying the same rule into several layers by default.

For example, loss-before-activation at the same simulation step belongs in the authoritative acceptance document. A review skill can point to it if the agent repeatedly misses that boundary.

“Move this planter two units east” belongs in the current design change. Reevaluate optional guidance after changing the model: a workaround that helped yesterday may be unnecessary today.

The retrospective

After a release, write three sentences: which instruction helped most, which failure cost the most rework, and which single change to the workflow you will test next time. Connect each sentence to an actual observation.

Improve game versions and prompt versions separately

Your proposed loop is useful: build V1, fix it, learn from the prompts, then attempt V2. The correction is to retain **prompts plus observed outcomes**, not concatenate the conversation.

V2 also inherits better code, assets, and human decisions. Its improvement alone cannot establish that prompt P2 beats P1.

A prompt revision changes supplied instructions; it does not update Astra's model weights.

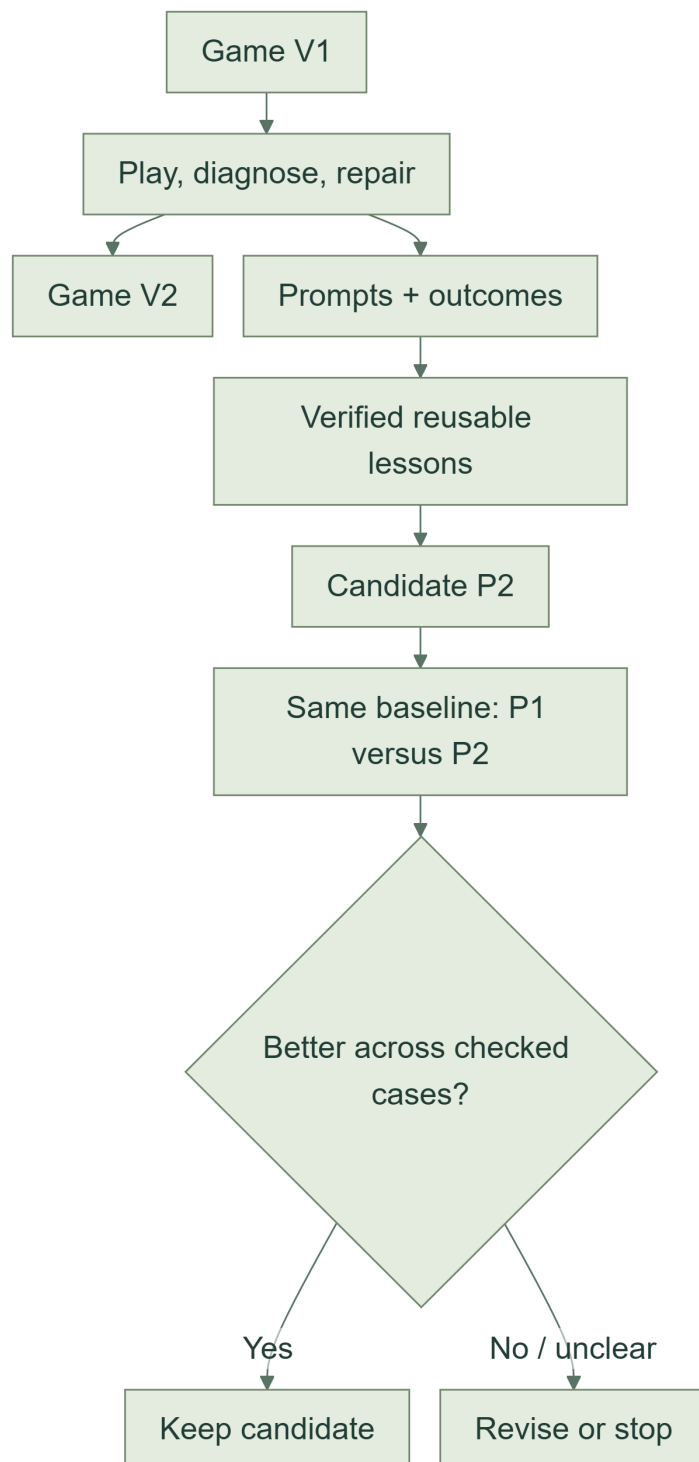
1. **Save a baseline.** Record commit, prompt version, model settings, tools and skills, and acceptance conditions.
2. **Build and diagnose.** Keep failures, successful repairs, evidence, and intervention count.
3. **Extract verified lessons.** Separate facts from guesses; drop failed advice and obsolete requirements.
4. **Store each lesson once.** Rules go in the brief; regressions in tests; conventions in AGENTS.md; a repeatedly useful procedure may become a skill.
5. **Draft P2.** Reference those files and state the next outcome. A shorter prompt can be better.
6. **Compare when the question matters.** Give P1 and P2 the same starting commit, files, model, tools, and budget in separate clean runs. Repeat promising comparisons and include a scenario not used to revise P2. Keep main-game development incremental.

For example, a failed restart should produce a regression check and an acceptance rule, then P2 can say "Implement the next milestone and preserve the acceptance rules." If P2 receives corrected documents that P1 did not, you have tested an improved instruction package, not wording alone.

Decide which comparison you want before running it. This is our experimental design, informed by task-specific evaluation guidance. [52]

MERMAID · EDITABLE FLOW Two related improvement loops

Keep the game moving forward; test prompt revisions separately against the same baseline.



[Download .mmd](#)

Run ID / baseline commit / prompt version:
 Model settings / tools / available and invoked skills:
 Task / acceptance checks / time and cost budget:
 Observed failure / verified repair / remaining uncertainty:
 Lesson to retain / authoritative file / supporting evidence:
 Held-out scenario / regressions / decision: keep, revise, or stop

A hypothetical change from 6/10 to 8/10 successful runs is **20 percentage points**, or about 33% relative improvement. It is not “20% better,” and ten runs do not establish a general success rate. Report counts, failed cases, costs, and uncertainty; avoid ranking prompts by a single attractive output.

The dataset-backed OpenAI prompt optimizer illustrates feedback-based revision, but its documentation now schedules legacy Evals read-only access for 31 October 2026 and shutdown for 30 November 2026. The repository-based method above needs neither service.

Optimization can regress on some inputs; verify the candidate before adopting it. [53]

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Failure classification, prompt revisions, and rubric

Covers: Introduction · A worked prompt refinement · Use a rubric to expose disagreements

The examples and 0-2 rubric are authored comparison aids. Human judgments should remain visible; no measured experiment produced the P0-P3 sequence.

OpenAI · Building games with Astra [04: reading note]

Anthropic · Harness design for long-running application development [21: reading note]

Fair comparisons and retiring redundant guidance

Covers: Compare approaches with a fair experiment · Keep only improvements that still help · The retrospective

Varying model/task findings justify controlling the starting revision, tools, and budget. Repetition and transparent intervention records improve the comparison; the book reports no new Astra benchmark.

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

SkillsBench · Agent leaderboard, v1.1 [18: reading note]

SkillsInjector · Dynamic Skill Context Construction for LLM Agents [19: reading note]

SkillAlchemy · Open-World Agent Skill Creation [20: reading note]

Anthropic · Harness design for long-running application development [21: reading note]

SWE-Skills-Bench · Do Agent Skills Actually Help in Real-World Software Engineering? [22: reading note]

Prompt learning versus game improvement

Covers: Improve game versions and prompt versions separately

Separates confounded game progress from controlled prompt comparisons. Counts and percentage calculations are illustrative; no comparison was executed. The optimizer retirement notice is product-specific.

Refine the idea and the process

OpenAI · Evaluation best practices [52: reading note]

OpenAI · Prompt optimizer and retirement notice [53: reading note]

Read the review method and corrections

CHAPTER 15

Publish and close the loop

A release is a working URL and a recoverable revision.

For the proposed Vite game, GitHub Pages serves built static files. It provides no private server runtime for this game's API calls.

A key placed in published JavaScript is delivered to visitors, so keep service credentials in a separate backend or local tool when needed. Signal Garden's first release needs neither accounts nor an online service. [46]

1 · Configure the repository path

A project called signal-garden normally appears at https://YOUR_ACCOUNT.github.io/signal-garden/. In Vite, set the base path accordingly.

A user site at the domain root uses a different base. The repository name's case and asset filenames must be correct on the deployed host. [12]

```
// vite.config.ts - example for the game repository
import { defineConfig } from 'vite';

export default defineConfig({
  base: '/signal-garden/',
});
```

Test the production build under this subpath before publication. Prefer imported assets or base-aware URLs. A path that works at `localhost/` may fail when the application lives one directory below the domain root.

2 · Build, upload, deploy

In the repository's Pages settings, choose GitHub Actions as the source. A Pages workflow builds the site, uploads its public artifact, and deploys it with the appropriate Pages and identity-token permissions. Use the `github-pages` environment and the URL returned by the deployment step. [13]

This book's actual workflow is a small static-site example. A Vite game should install from its lockfile and run its own checks before uploading dist. Pin reviewed action versions or commit SHAs, and check current official examples when adapting the workflow.

3 · Make publication explicit

After the game works locally, replace the account and repository placeholders in this release prompt. Its publication permission applies only after you actually send it with the intended destination.

Edit these parameters

PROJECT_FOLDER: current selected game folder

AGENT: Codex

MODEL: GPT-6 Astra

GITHUB_OWNER: [YOUR_ACCOUNT]

REPOSITORY: signal-garden

PUBLISH: YES

Choices and consistency

- AGENT: choose Codex or Claude Code.

Select MODEL in that application before sending; use an available Claude model with Claude Code.

These lines do not switch the actual model or grant tools.

- Work only in PROJECT_FOLDER.

Inspect the actual tools and relevant project instructions; resolve a material mismatch before dependent work.

- PUBLISH: YES authorizes the public repository/source and deployment described below; NO means local release preparation only.

Resolve all destination placeholders before publication.

Derive the URL and base path from GITHUB_OWNER and REPOSITORY.

Goal

Prepare or publish this game on GitHub Pages, according to PUBLISH.

Tasks and constraints

- Use the GITHUB_OWNER and REPOSITORY values above; resolve placeholders before any external publication.

- Inspect the actual repository, branch, remotes, intended diff, excluded secrets and asset licenses.

Preserve unrelated work; do not replace a conflicting remote or force-push.

- Keep the existing engine.

For a Vite/Three.js project, verify the install/build commands and dist output.

For Godot, verify the installed version, web-export support and actual export directory; preserve the generated filenames.

Prove the static build works before publishing.

- For a project site, derive the URL as https://GITHUB_OWNER.github.io/REPOSITORY/ using the actual values.

Set Vite base to /REPOSITORY/ when using Vite; for another engine, verify its exported assets under that path.

- Prepare a GitHub Pages Actions workflow that builds the project and uploads only the verified static output directory, with the required Pages permissions.

- If PUBLISH is YES, I authorize creating the named public GitHub repository if absent, committing and pushing this game's reviewed intended files, and publicly deploying it to the selected host.

If PUBLISH is NO, stop after local preparation and report the remaining release steps.

- For YES, enable Pages from GitHub Actions if permitted and follow the workflow for the exact pushed commit.

- Open the resulting public URL, verify assets and a complete round through real controls if available, and identify any unperformed checks.

Completion check

PUBLISH=NO: a tested local release package and remaining steps.

PUBLISH=YES: the reviewed commit, matching successful deployment, public URL and actual verification record.

Report a concrete blocker if completion is unavailable.

4 · Verify the public result

1. Confirm the push reached the intended remote and branch.
2. Inspect the workflow run for that commit. A queued run is not a successful deployment.
3. Open the deployment URL and check the production asset requests.
4. Play a win path and a loss path, then restart. Repeat a pause/resume check.

5. Record the deployed commit, controls, target browser, and any remaining limitations in the release note.

If deployment fails, inspect its logs and fix the specific cause. For a missing asset, check the base path and filename case.

For a Godot export, check the renderer, exported filenames, and browser requirements. For a permissions error, inspect the Pages source, workflow permissions, and repository settings before changing unrelated code.

5 · Keep a path back

Record the last verified release revision. If a new release breaks the game, revert the responsible change in a new commit and allow the workflow to publish the repaired state.

Check the resulting live URL again. A rollback command is only a step toward restoration.

The complete workflow

Choose a player experience → compare relevant approaches → verify current sources → establish the repository and brief → use available skills when relevant → choose a compact or bootstrap prompt → build playable checkpoints → test the real game → publish → retain only workflow changes that help.

The durable result is more than a game. It is a process that makes your next attempt easier to understand, easier to review, and easier to improve.

The next two chapters map that process as editable decision and data flows, then apply it to creating a short game trailer.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Static hosting, repository base, and workflow

Covers: Introduction · 1 · Configure the repository path · 2 · Build, upload, deploy

Checked Pages' static role, Vite project base, lockfile install, and artifact/deploy requirements. The linked book workflow is the actual implementation; the game snippet is an example.

GitHub · What is GitHub Pages? [46: reading note]

Vite · Deploying a Static Site [12: reading note]

GitHub · Using custom workflows with GitHub Pages [13: reading note]

npm · npm ci [42: reading note]

Publication and public verification

Covers: 3 · Make publication explicit · 4 · Verify the public result

The release prompt and full-play sequence are original verification procedures. Successful CI alone does not demonstrate gameplay; the expected revision and its assets must be checked at the public URL.

GitHub · Using custom workflows with GitHub Pages [13: reading note]

Godot · Exporting for the Web [11: reading note]

Vite · Deploying a Static Site [12: reading note]

Recovery

Covers: 5 · Keep a path back

Git revert records an undo and the workflow publishes another artifact. Restoration still requires a successful deployment and a check of the repaired public result.

Git · Staging, differences, branches, and reverting [39: reading note]

GitHub · Using custom workflows with GitHub Pages [13: reading note]

Read the review method and corrections

CHAPTER 16

Think, decide, improve

Make the next decision visible.

A useful thinking flow shows the outcome, the uncertainty, the action, and the evidence that changes the plan. These are **observable working decisions**, not a reconstruction of a model's private reasoning.

The diagrams below are original proposals for this book. Use them to discuss work with Astra and to review what happened afterward.

Read and reuse the diagrams

Rectangles name actions or artifacts; diamonds name decisions; labeled arrows show the condition or information that moves work forward. A loop means that new evidence may change the next action.

It does not mean “repeat forever.” Set a completion condition and an effort limit for an experiment.

These are real **Mermaid** diagrams: text definitions rendered as SVG. Each includes a readable description, a copy button, the editable .mmd source, and an SVG download after rendering.

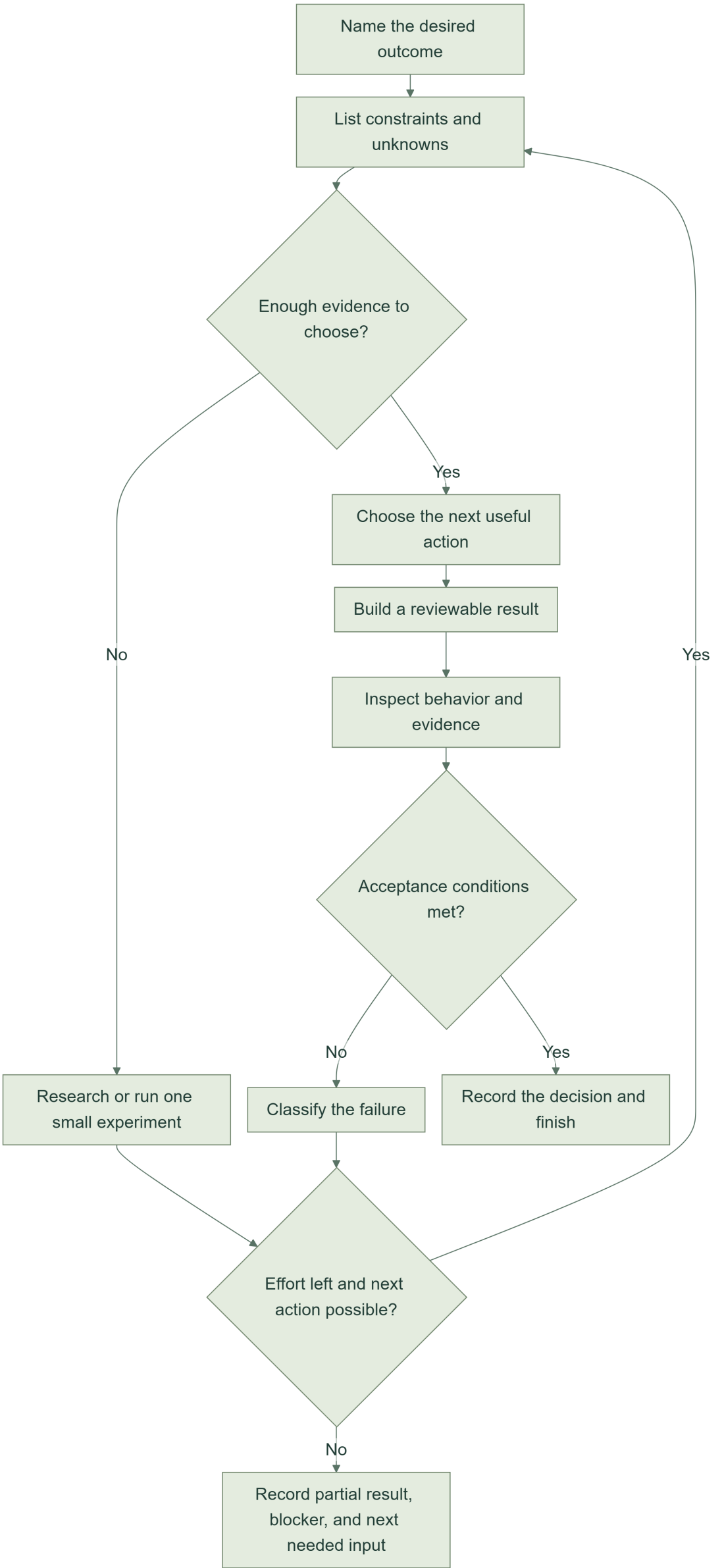
Scroll across wide graphs, or select **Fit diagram** for an overview. A diagram explains a process; it does not execute it.

Mermaid documents how to embed and render definitions. [36]

1 · The thinking and decision loop

MERMAID · EDITABLE FLOW The thinking and decision loop

Name the outcome, investigate the important unknown, build a small result, and judge it using evidence. Record the decision when the acceptance conditions are met. If effort is exhausted or progress needs unavailable input, record the partial result and blocker.



Download .mmd

Signal Garden example: “The player loses sight of the robot behind the north planter” is a useful observation. First reproduce that route.

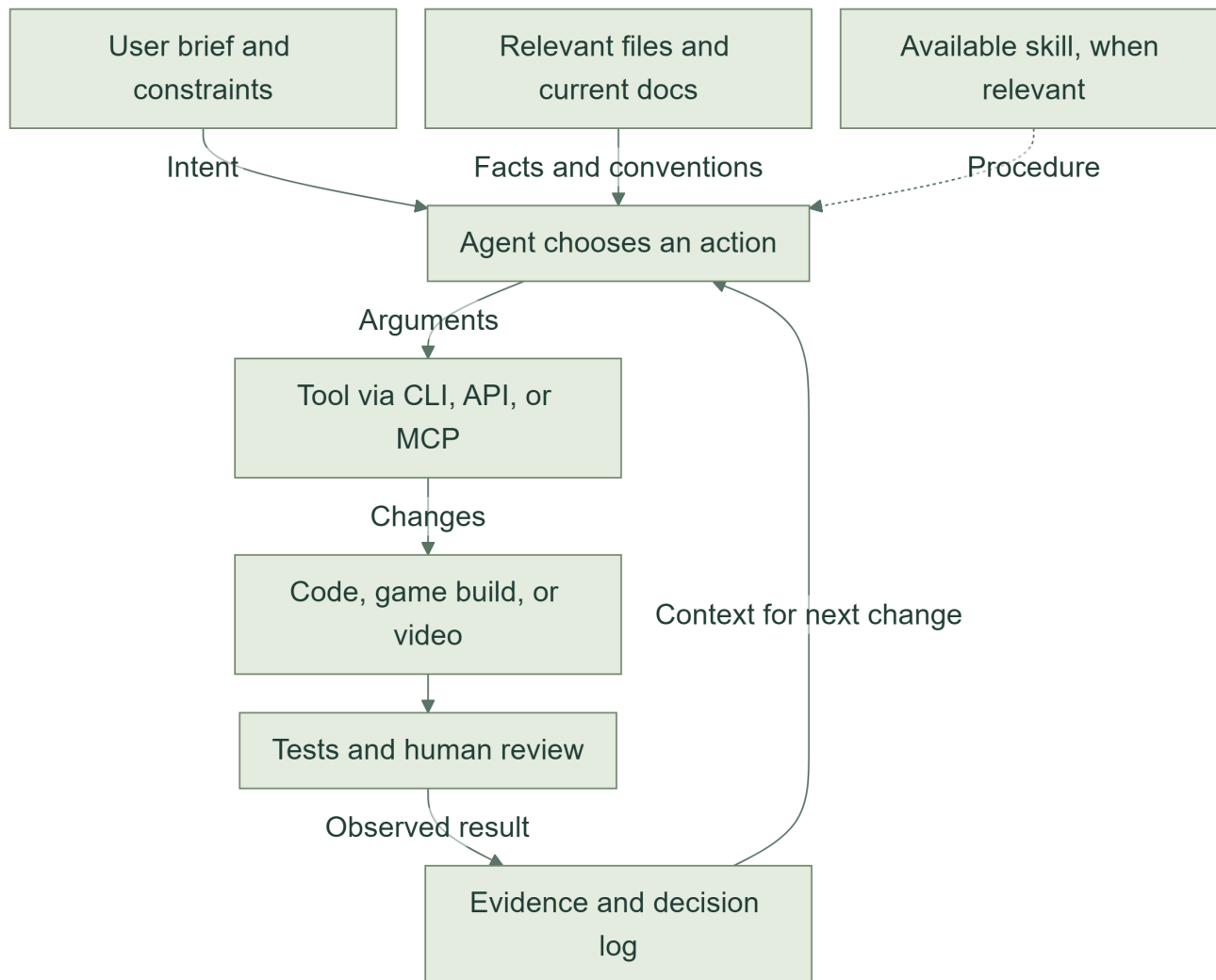
Compare a camera adjustment with a small occlusion treatment, then choose the smallest change that keeps the robot readable. Stop when the named route and nearby routes work; adding enemies does not resolve this uncertainty.

Ask Astra for a concise decision record: what changed, what evidence supports it, which uncertainty remains, and what happens next. A lengthy explanation is not a substitute for a playable result. The process can be lightweight for a small edit and more explicit when a decision affects the whole game.

2 · The data flow: prompts, context, skills, and tools

MERMAID · EDITABLE FLOW What information goes where

The brief supplies intent; relevant files and documentation supply facts; an optional skill supplies a procedure. Tools change artifacts, and review returns evidence to the next decision.



[Download .mmd](#)

- The brief supplies the desired experience.
- Repository files supply actual rules and implementation.
- A relevant skill may supply a reusable procedure.
- A tool performs an action.
- Results return as evidence for another decision.

This separation helps diagnose failures: a better prompt cannot grant missing browser access, and adding an API connection does not settle an ambiguous game rule.

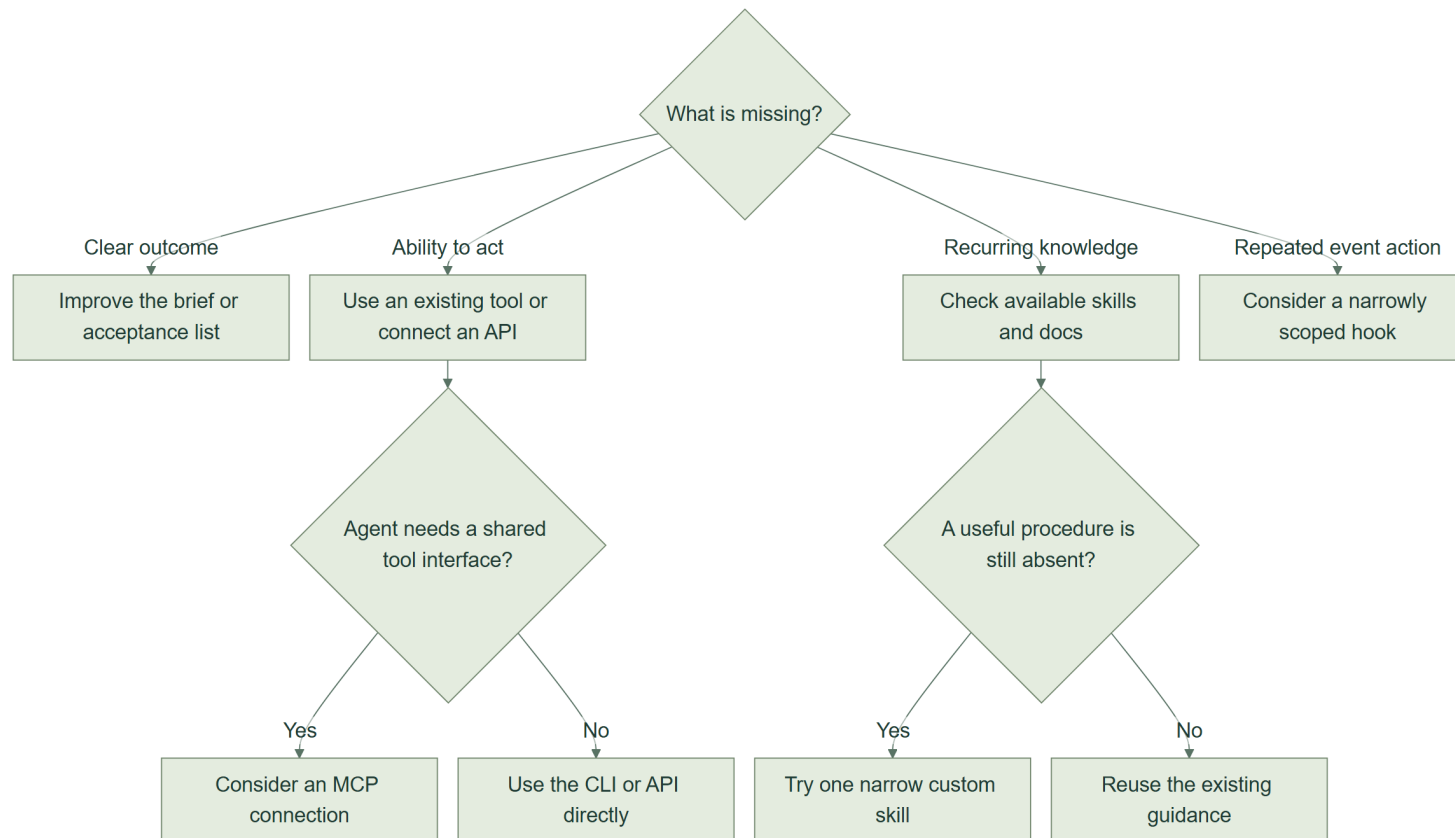
Layer	What it contributes	Example
Prompt / brief	Intent and acceptance conditions	Collect three cells, then activate the beacon.
Repository context	Current facts and conventions	The implemented state machine and its acceptance document.
Skill	A discoverable procedure or supporting resource	An available game-review workflow.
Tool / API / CLI	The ability to inspect or act	Run the game, capture a frame, or render a video.
MCP	A protocol connecting an agent application to exposed tools and context	A server that lets the agent submit a ComfyUI workflow.
Hook	An action triggered at a supported agent lifecycle event	Validate a manifest after a relevant file edit.
Workflow engine	Coordination across steps, jobs, and waiting states	n8n submits a task, waits, checks status, and stores the result.

MCP standardizes context exchange; it does not prescribe the agent's planning strategy or make a server's tools automatically available everywhere. Hooks depend on the host's supported events and configuration. [23] [24]

3 · Decide what to add

MERMAID · EDITABLE FLOW Choose the help you actually need

Clarify an unclear outcome, connect a missing tool, reuse existing procedural guidance, and consider a hook only for a repeatable event action.



[Download .mmd](#)

Before writing a new skill, inspect the agent's actual capabilities. Is the existing skill available?

Was it selected? Does it contain the missing knowledge?

The skills chapter explains a controlled comparison. Astra's current guidance supports reassessing overlapping instructions, while leaving room for specialized procedures. [17]

For video work, this question becomes concrete.

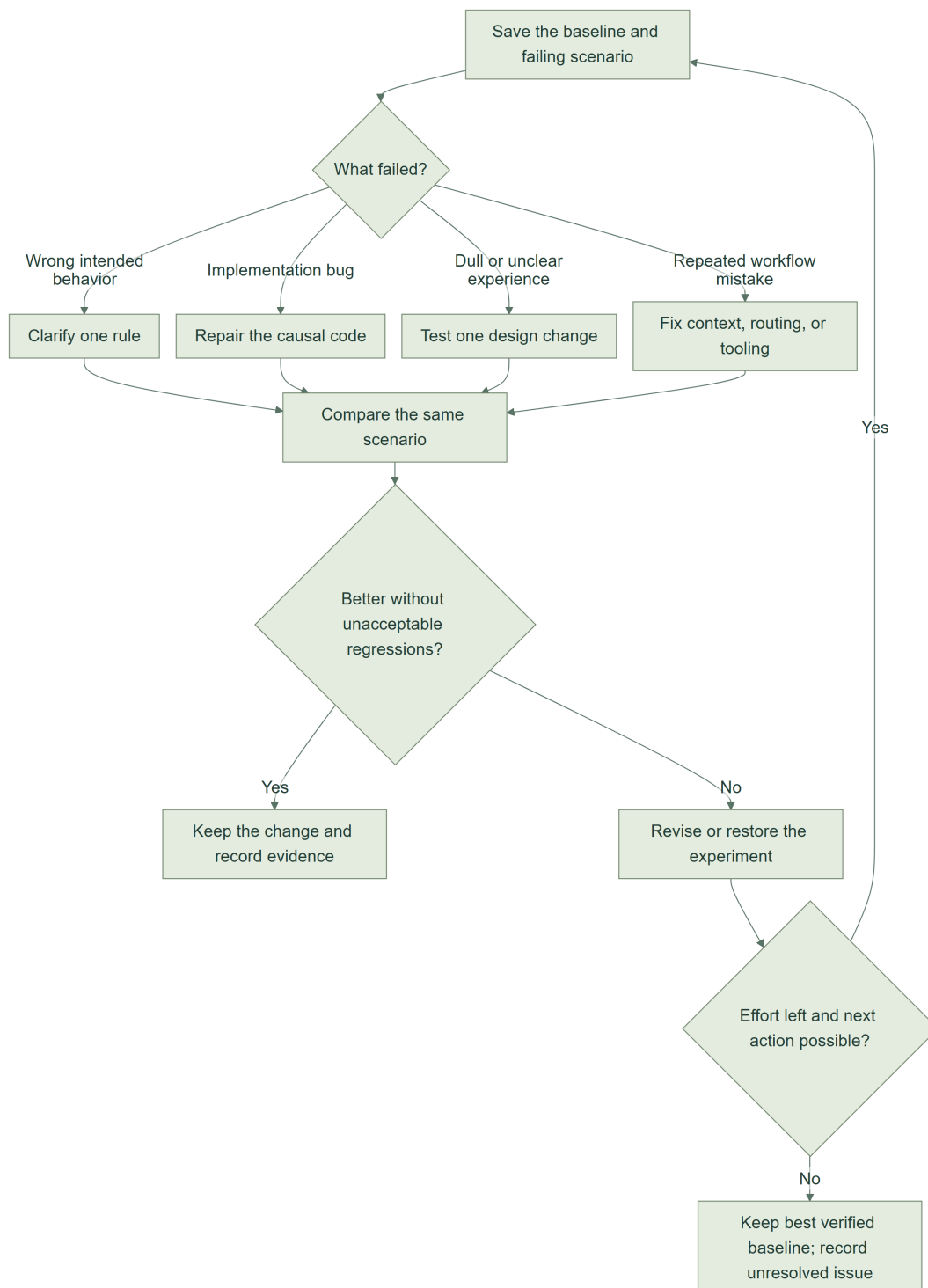
- A Remotion skill can teach composition conventions.
- A ComfyUI MCP tool can submit a workflow.
- A hook can check a changed file.

They may work together because they solve different problems; installing all three is not a prerequisite.

4 · Improve the cause of the failure

MERMAID · EDITABLE FLOW Improve one cause at a time

Preserve a baseline and failing scenario. Change the rule, code, design, or workflow responsible. Compare the same scenario and retain only improvements that survive review. Stop at the effort limit and preserve the best verified baseline.



Download .mmd

Keep the baseline revision and a short reproduction before experimenting.

- If the intended rule is wrong, revise the brief.
- If the code violates a sound rule, repair the code.
- If the implemented experience is dull, test a design hypothesis.
- If an agent repeatedly misses a procedure, investigate context and selection before adding instructions.

A decision record you can reuse

- **Outcome:** what should the player or viewer experience?
- **Evidence:** which route, screenshot, clip, or check exposes the problem?
- **Change:** what single cause are we testing?
- **Comparison:** what improved, and what regressed, under the same scenario?
- **Decision:** retain, revise, or restore; link the artifact and revision.

This record is also a good prompt input. Give Astra the observed problem and the review conditions, then let it choose appropriate implementation details.

Save a reusable procedure only after repeated evidence shows that it helps. Next, apply the same approach to an AI-assisted game trailer.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Process diagrams and bounded decisions

Covers: Introduction · Read and reuse the diagrams · 1 · The thinking and decision loop

Mermaid documents text-to-SVG rendering. The graphs describe observable decisions, not private model reasoning or executable automation. Success, effort limits, and blocked progress all have explicit outcomes.

Mermaid · Usage [36: reading note]

OpenAI · Building games with Astra [04: reading note]

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

What flows between the layers

Covers: 2 · The data flow: prompts, context, skills, and tools

The documented roles distinguish conventions, optional procedures, tools, MCP, lifecycle hooks, and workflow nodes. Our graph is a simplified data model, not a claim that every host exposes every integration.

OpenAI · Custom instructions with AGENTS.md [07: reading note]

OpenAI · Build skills [08: reading note]

MCP · Architecture, specification 2026-07-28 [23: reading note]

OpenAI · Codex hooks [24: reading note]

n8n · HTTP Request node [34: reading note]

n8n · Wait node [35: reading note]

Comfy · MCP server for local and cloud workflows [37: reading note]

Choose help and improve the cause

Covers: 3 · Decide what to add · 4 · Improve the cause of the failure · A decision record you can reuse

The routing questions and decision record are our practical synthesis. Documentation-only MCP deprecation and action-capable MCP are different cases. Diagnose access, context, or implementation before adding procedures.

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

Remotion · Documentation MCP deprecation [27: reading note]

Comfy · MCP server for local and cloud workflows [37: reading note]

Comfy · Skills for agents [38: reading note]

Read the review method and corrections

CHAPTER 17

Make a video with an agent

A video workflow can contain skills, MCP, and hooks. The flow itself is a separate idea.

The exact page you saw has not been identified, so we cannot name its software with certainty. A canvas of linked model nodes is often a tool such as **ComfyUI**.

A canvas of HTTP requests, waiting steps, and conditions may be an automation tool such as **n8n**. An agent calling a media service may use MCP.

Below are verified examples and a practical way to choose among them.

A current change worth noticing

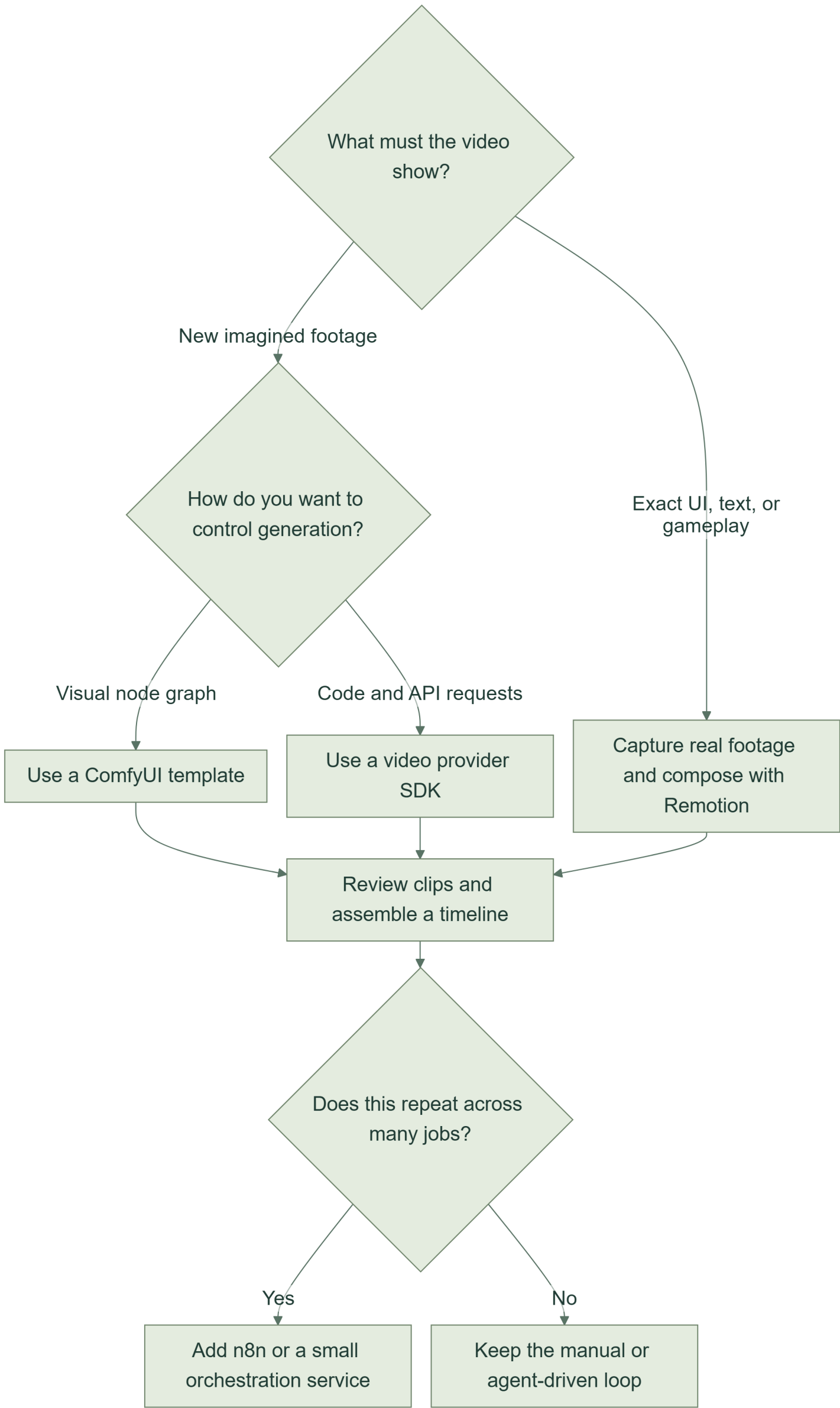
Remotion's documentation, updated **12 September 2026**, deprecates its older documentation MCP and recommends agent skills instead. This is evidence of one redundant documentation integration, not evidence that MCP or skills are universally obsolete.

Its job was to provide docs; an MCP server that actually submits video jobs has a different role. [27]

1 · Choose the route by the result you need

MERMAID · EDITABLE FLOW Choose a video workflow

Use composition and real captures for exact text and gameplay; use a generation model for imagined footage. Add orchestration when the process repeats across jobs.



Download .mmd

Route	Good fit	What you control	What still needs review
Remotion + coding agent	Precise titles, captions, UI animation, and edited gameplay	Composition code, timing, layout, and supplied media	Actual export, frame timing, typography, and sound
ComfyUI node workflow	Generated footage with visible model and input stages	A compatible template, prompts, references, and exposed settings	Model access, resource requirements, motion, and continuity
Video provider SDK / API	Programmatic generation and job tracking	Provider-supported parameters and task handling	Model variability, cost, failures, and saved outputs
n8n around an API	A repeated process spanning several jobs or services	Requests, conditions, waits, storage, and review checkpoints	Retry behavior, credentials, execution state, and duplicate jobs

This comparison is our synthesis of the current documentation, not a speed or quality benchmark. These routes can be combined: generate an atmospheric clip in ComfyUI, combine it with real game capture in Remotion, and automate a proven repeated process later. [25] [28] [31] [34]

2 · Begin with a small trailer brief

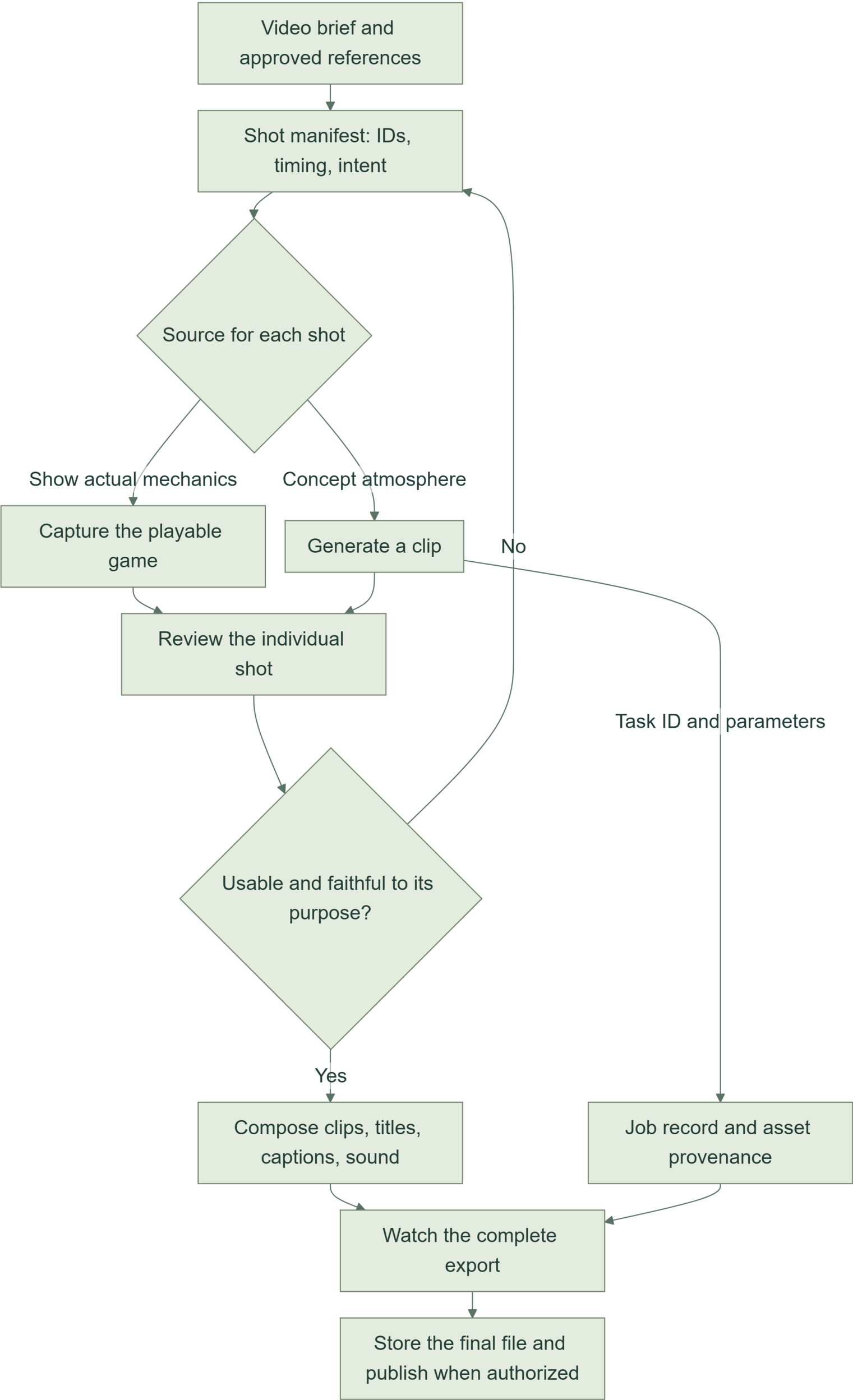
For a first **Signal Garden** teaser, propose ten seconds at 16:9 with three shots. This is an original storyboard target; no video has been generated for this book.

Use real game capture whenever the shot claims to demonstrate mechanics. Label generated concept footage so viewers can distinguish an imagined world from the implemented game.

Shot	Length	Purpose	Source and review condition
SG-01	3 seconds	Establish the rooftop and beacon	Game capture, or clearly labeled concept footage; the arena is readable.
SG-02	3 seconds	Show the collection objective	Actual gameplay capture; the cell pickup and counter agree.
SG-03	4 seconds	Show the beacon and title	Actual activation plus a composed title; spelling and timing are exact.

MERMAID · EDITABLE FLOW A trailer’s data flow

A brief becomes a shot manifest. Captured or generated clips are reviewed, assembled with titles and sound, then exported. Job records connect source assets and parameters to the final file.



Download .mmd

Make a **shot manifest**: one record per stable shot ID, with purpose, duration, source type, reference files, prompt when needed, output path, and review status. Record model/version, settings, task ID, attempt number, and actual cost where the service exposes them.

This is your project record, not a universal provider request schema. Keep credentials in the tool's credential store, outside the manifest and public repository.

For a generated establishing shot, try a focused visual prompt: “A small rooftop garden at dusk, a single teal beacon at the center, low planters, muted green concrete, a slow elevated camera push toward the beacon, stable geometry and lighting.” Supply duration, ratio, and a reference image through the service's supported controls. Add titles during composition.

Compare one relevant change at a time instead of replacing the entire prompt whenever a clip disappoints.

3 · Use Remotion for composition and exact graphics

Remotion's current coding-agent guide scaffolds a project, installs dependencies, previews the composition, and introduces its skills. The commands below create a separate video project; run them in the parent folder where you want that project. Node.js and a working coding agent are prerequisites. [25]

```
npx create-video --yes --blank signal-garden-trailer
cd signal-garden-trailer
npm install
npm run dev
```

Check whether an appropriate Remotion skill is already available. If needed, the guide provides `npx remotion skills add`.

Ask the agent to implement the storyboard using your capture files, preview it, and render with the installed project's documented command. Review the exported video itself.

The current skill collection covers composition, documentation, preview, and rendering workflows. [26]

Create motion graphics with AI - Simple tutorial for beginners is the five-minute companion linked by Remotion's official guide. **We reviewed the written guide; this video's transcript was not retrieved.** Treat it as a visual follow-along resource, not transcript evidence. Its practical sequence is summarized in reading note 25.

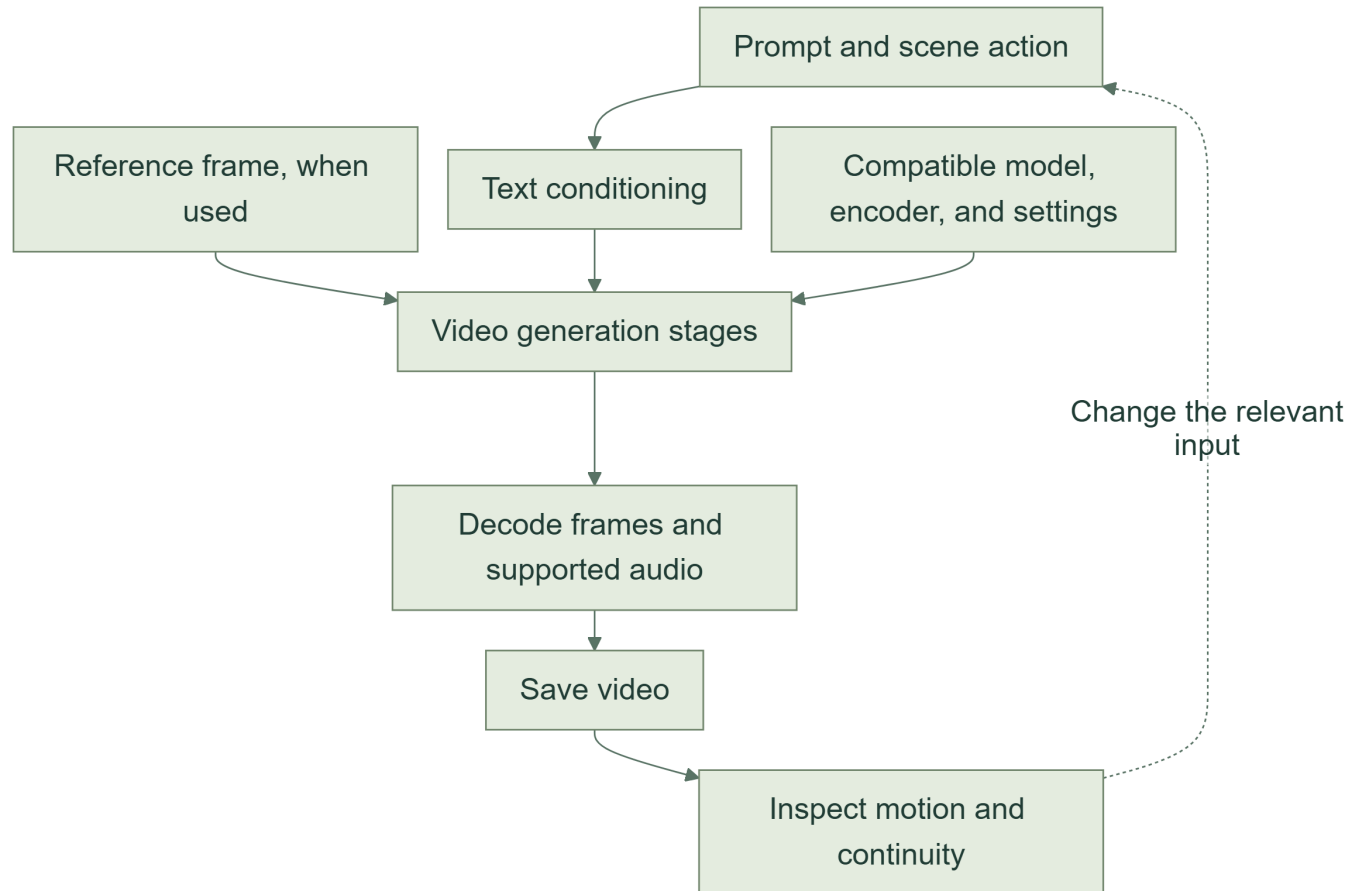
4 · Use ComfyUI when you want a visible generation graph

Start from the application's **Workflow Templates** browser and inspect the selected template's model and input requirements. Templates provide a concrete starting graph, while the reviewed LTX-2.5 guide covers text-to-video, image-to-video, and first/last-frame workflows with supported audio.

Model access and a compatible installation matter; a new model name alone does not make a graph runnable. [29] [28]

MERMAID · EDITABLE FLOW Inside a video node graph

Conceptual stages only, not literal node names or an importable ComfyUI graph. Compatible model components, prompt conditioning, and optional reference frames feed generation and decoding before saving a video.



[Download .mmd](#)

This drawing is conceptual. It is not an importable ComfyUI JSON workflow and its boxes are not exact node names. Use an official compatible template, keep its required model components together, change the prompt or reference input, run one short attempt, and inspect the saved video.

If nodes are missing, check the template's installation requirements before replacing arbitrary nodes.

LTX-2.5 is the current LTX guide reached during this review; its page does not provide a clear publication date. It is a gated model with access requirements.

Choose a model your account or machine can run, and start from its supported settings. These are operational constraints, not reasons to copy an older tutorial's model files blindly. [28]

A local LTX template does not establish that the same model is available through Comfy Cloud. For the cloud route below, discover the account's actual template catalog and inspect its schema before choosing inputs. Local installation requirements and cloud availability are separate checks. [29] [37]

Let an agent operate it through the official Comfy MCP

Comfy's **public-beta** documentation provides this Codex CLI cloud setup, separate from its local connection. Complete sign-in and check your subscription and budget before generation. These instructions have not been executed for this book. [37]

```
codex mcp add comfy-cloud --url https://cloud.comfy.org/mcp
codex mcp login comfy-cloud
```

1. Inspect the registered tools; discover a template with `search_templates` and inspect its `get_template_schema`.
2. Map the brief to supported inputs and confirm the allowed budget.
3. Use `run_template`. Custom `submit_workflow` requires API-format JSON, distinct from editor JSON.
4. Keep the job ID and wait for completion with tools such as `wait_for_job`. Use `get_output` to obtain download instructions. That tool does not write the local file itself: check the destination, perform the documented download, verify the saved file, and review the clip. [37]

Comfy also publishes skills containing workflow and model guidance. That knowledge complements the MCP's ability to act.

Reuse an available relevant skill; add one only when it fills a gap. Its documentation includes host-specific plugin examples, so do not assume a Claude Code installation command also configures Codex. [38]

Repository update: Comfy's current repository places the canonical cloud commands in `claude-code/`; its legacy `skills/` folder is frozen. The separate `comfy-cli` now bundles its own skills. Match installation to the host and discover live templates instead of copying old model lists. [55]

5 · Use an API when you want explicit job handling

Runway's multi-shot recipe, version 2026-06, supports automatic story decomposition or a custom shot list. Its shown schema generates the shots; it does not insert our existing gameplay captures.

Use it for a separately labeled **all-generated concept trailer**. For the mixed-source storyboard above, compose SG-02 and SG-03 from real capture in Remotion.

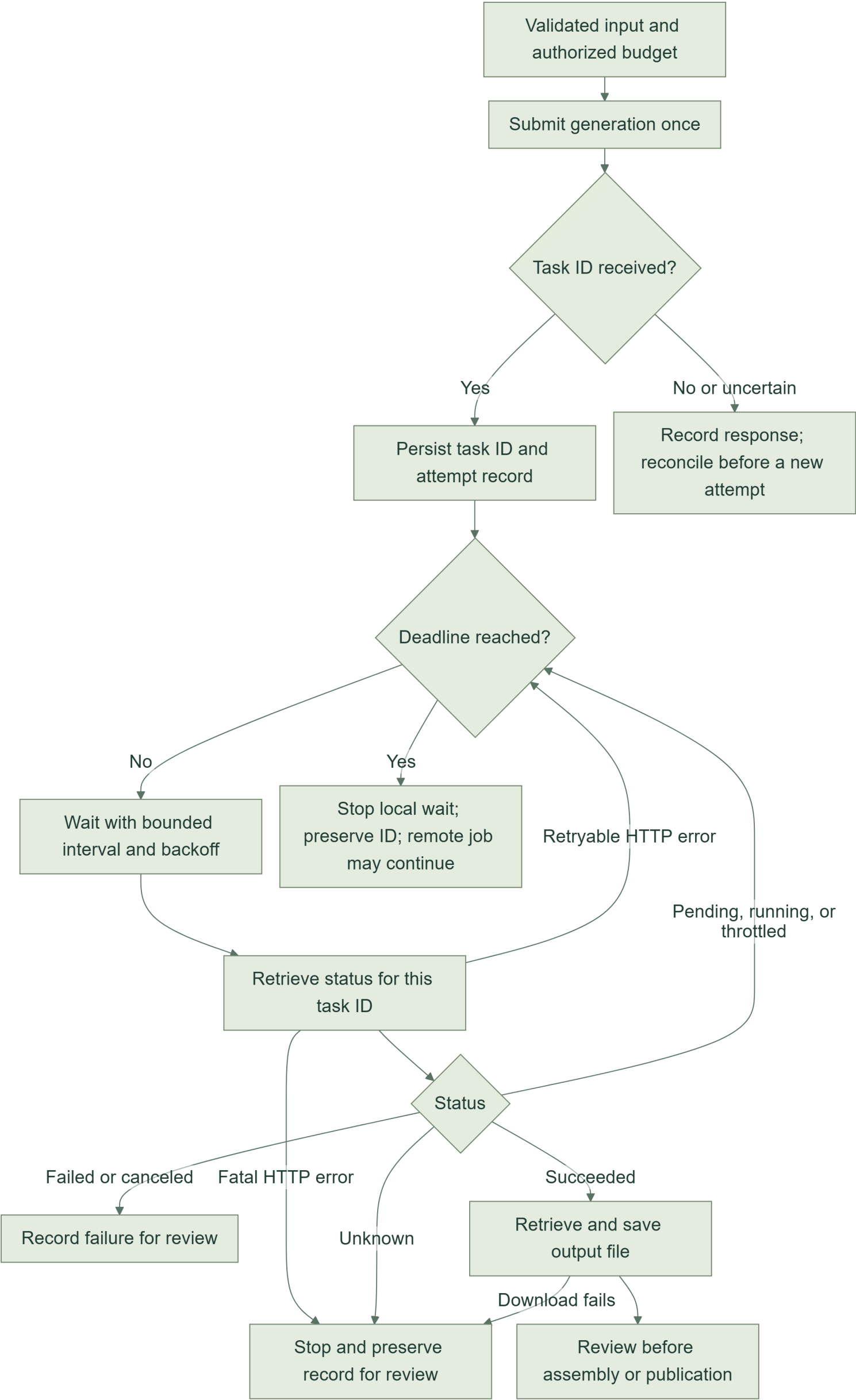
A three-shot, ten-second concept variant fits the documented timing options, but has not been run here. [32]

There is also an important MCP distinction in Runway's own documentation: <https://dev.runwayml.com/mcp> serves developer tasks and documentation, while <https://mcp.runwayml.com> is identified as the separate media-generation integration. Do not configure the developer endpoint and assume it exposes a video-generation tool.

Inspect the connected server’s actual capabilities. [30]

MERMAID · EDITABLE FLOW Submit once, track the job, inspect the output

Keep the task ID through pending, running, and throttled states. Retry transient status errors within a deadline; reconcile uncertain submissions. A local timeout does not cancel the remote job. Save and review successful output.



Download .mmd

Runway's SDK documentation describes asynchronous tasks and polling. Follow its current polling interval and backoff guidance; its manual polling example says at least five seconds with jitter.

A local waiting timeout does not prove the remote job failed or was canceled. Keep the task ID and check it before considering a new paid attempt. [31]

PENDING, RUNNING, and THROTTLED are nonterminal states. A throttled task has been accepted but is waiting for capacity; it is distinct from an HTTP 429 response.

For retryable status-request errors, use bounded backoff and jitter. Stop on a fatal error, unknown state, or deadline and preserve the task record.

If the initial submission has an uncertain result and no ID, reconcile it through provider records before submitting again. These recovery choices are our proposed orchestration policy. [47] [48]

Save accepted output to storage you control. Runway's output URLs are temporary, typically expiring within 24-48 hours of access. A task record should point to the saved file as well as the original task, so a later edit does not depend on an expired delivery URL. [33]

6 · Build the repeating flow in n8n

Once one request succeeds, reproduce its working method, URL, authentication, headers, and body in an **HTTP Request** node. That node makes API calls; the video provider performs the generation. Use n8n credentials for authentication. [34]

1. **Manual trigger:** load one approved shot and its allowed attempt budget.
2. **Submit:** send one generation request per attempt; persist the returned task ID and shot ID. An uncertain submission goes to reconciliation.
3. **Wait:** pause for the provider's required interval while within the deadline.
4. **Check:** GET that same task; route pending, running, or throttled back to Wait. Handle retryable HTTP errors with bounded backoff.
5. **Resolve:** save successful output; record failure, cancellation, unknown status, or local timeout for review. A local timeout does not cancel the remote task.
6. **Assemble:** use reviewed clips in the timeline, then watch the full export before publication.

The job diagram above is the logic to reproduce with HTTP Request, Wait, and conditional branches. The Wait node can resume after time or through a webhook.

Use a callback only when the selected provider documents one, and configure its authentication. A webhook is an incoming HTTP event; it is distinct from an agent's lifecycle hook. [35]

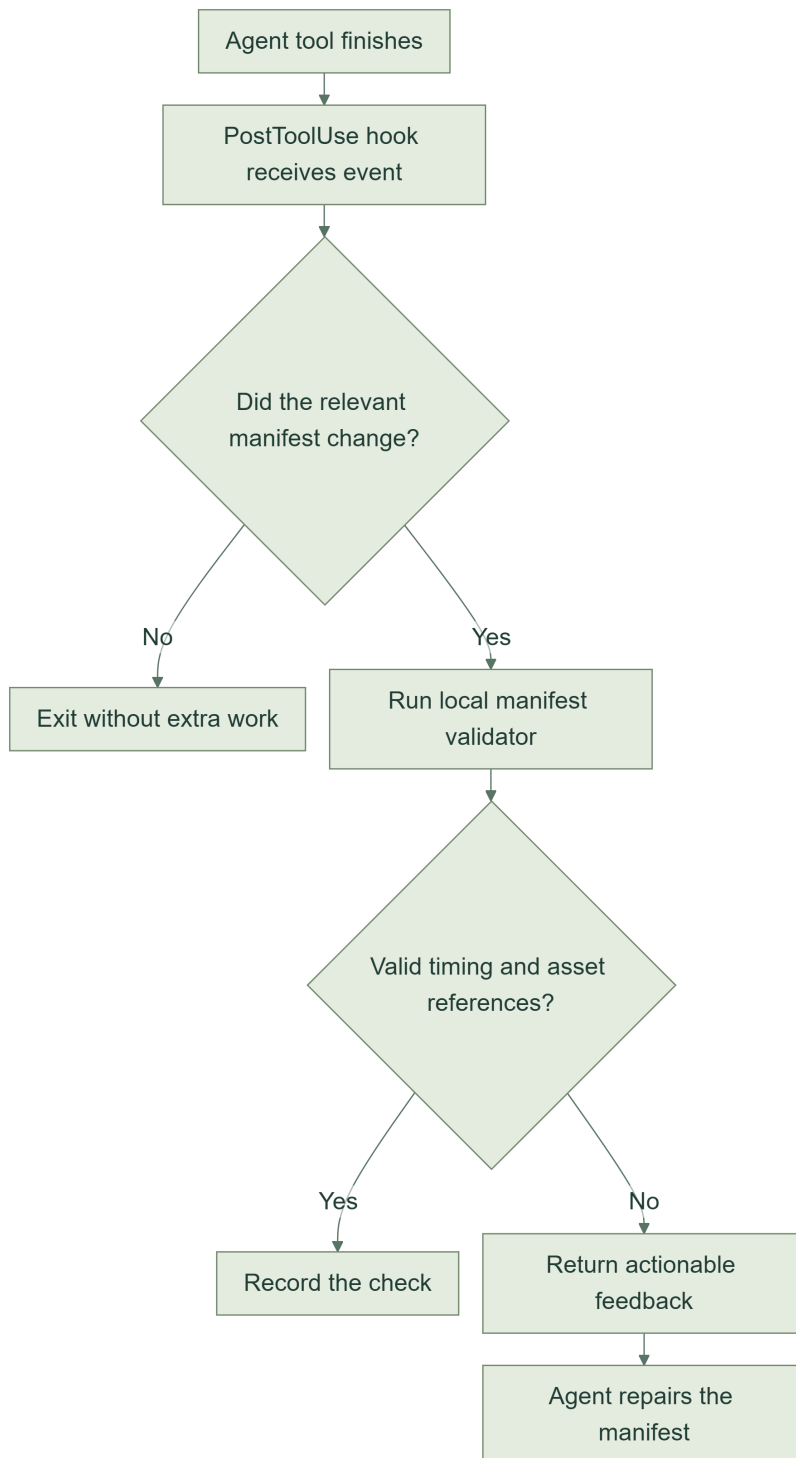
Keep generation submission outside the status-polling loop. If a request's result is uncertain, reconcile the existing task before retrying.

Workflow recovery should not silently multiply paid jobs. Automating a reviewed small pipeline is easier to diagnose than beginning with a large imported template whose endpoints and models may have changed.

7 · Add a hook only for a repeated event action

MERMAID · EDITABLE FLOW Use a hook for a narrow local check

A relevant tool event can trigger a local validator. An irrelevant event exits immediately. This proposed pattern does not generate a video or publish anything by itself.



[Download .mmd](#)

A proposed hook could run an existing local validator after the agent edits the shot manifest. That validator checks duration totals, required shot fields, and local asset references.

Filter irrelevant edits out. First run the validator directly and confirm that its feedback is useful; then wire it to your host's supported event configuration.

Current Codex hooks include PostToolUse, which runs after a tool finishes. It cannot undo that completed action.

Configuration and trust requirements belong to the host; consult the current hook guide rather than copying another application's configuration. Hooks can automate narrow checks, while the main agent or workflow coordinates the project. [24]

8 · Give Astra a brief that can become a working pipeline

This reusable prompt asks the agent to inspect capabilities, choose a justified route, and prepare an inspectable first result. Fill in actual paths and the permitted budget. It is an original template, not a measured Astra video-production benchmark.

Build a small, reviewable game trailer with GPT-6 Astra

Create a 10-second, 16:9 teaser for Signal Garden in a separate video project.

Target: three shots lasting 3, 3, and 4 seconds, with clear titles and readable gameplay.

Inputs to fill in:

- Game repository and runnable build: [path and command]
- Approved gameplay captures and references: [paths]
- Video project directory: [path]
- Available tools/services and permitted generation budget: [details]
- Publication destination and permission, if any: [details]

Inspect the actual files, installed tools, and available skills.

Verify current

official documentation where an API, model, or command may have changed.

Recommend the smallest suitable route: captured footage and Remotion composition, a compatible ComfyUI template, or a video provider API.

Add n8n only if repeated

job coordination justifies it.

Explain the choice briefly using the target result.

Prepare a shot manifest with stable IDs, intent, duration, source type, inputs, output path, and review status.

Show actual mechanics with gameplay capture.

Label generated concept footage.

Compose exact titles in the edit.

An all-generated multi-shot recipe is a concept variant; it must not replace shots that claim to demonstrate implemented mechanics.

Verify cloud template

availability separately from a local model guide.

Use existing relevant skills where helpful.

Do not create a custom skill or hook

unless a missing recurring procedure or useful event action justifies it.

An MCP connection must expose the needed tool; inspect its registered capabilities.

Build a reviewable local composition using available media.

If inputs are missing,

use clearly labeled placeholders and list exactly what is needed to replace them.

If no paid-generation budget is supplied, prepare the request but do not submit it.

For authorized generation: validate the provider's actual schema; submit once;

persist task ID, shot ID, model/version, inputs, and attempt number.

Wait and poll

within a deadline using provider guidance.

Preserve the ID after a timeout and

check the existing job before retrying.

Handle all documented nonterminal states

(including THROTTLED on Runway), retryable status-request errors with bounded

backoff, and unknown/fatal outcomes.

A local timeout does not cancel a remote job.

If submission is uncertain and no ID was returned, reconcile before resubmitting.

Save outputs to durable local/project storage; receiving download instructions

is not proof a file was saved.

Verify the file and record actual cost when

available.

Keep credentials out of GitHub.

Inspect each clip, then watch the full rendered export for timing, continuity,

legible titles, audio, and faithful gameplay.

Repair the relevant shot or layer

and compare the same section before and after.

Report the artifact paths,

what was verified, and remaining limitations.

Publish only within the supplied

destination and permission; a local preview is a useful first checkpoint.

Review the result and improve the right layer

- If a shot communicates the wrong idea, fix its intent.
- If motion is unstable, change the relevant generation input or choose a different source clip.

- If text is misspelled, compose the text in the edit.
- If files expire, fix storage.
- If duplicate jobs appear, fix orchestration.

A longer skill file does not address all of these causes.

Keep the storyboard, non-secret settings, decision notes, composition code, and appropriate asset references in GitHub. GitHub Pages can serve static public media within its hosting limits; it does not provide the worker for long generation jobs or secret-bearing API calls.

Review the complete export and confirm the publication destination. [46]

Your first useful experiment

Prepare the ten-second storyboard, capture one genuine gameplay shot, and build a composition preview. Generate one concept clip only if it adds something the capture cannot provide and its cost is authorized. Record what improved the trailer before adding another model, skill, MCP server, or automation step.

Source check · what supports this chapter?

Checked 14 September 2026. Direct links below support the stated scope; original proposals are distinguished from documented or measured findings.

Route comparison and storyboard

Covers: Introduction · 1 · Choose the route by the result you need · 2 · Begin with a small trailer brief

The unidentified page cannot be named with certainty. Product capabilities are documented; the comparison, shot manifest, and 3/3/4-second storyboard are original proposals. No video has been generated for this book.

Remotion · Create videos with coding agents [25: reading note]

Remotion · Documentation MCP deprecation [27: reading note]

ComfyUI · LTX-2.5 video workflows [28: reading note]

Runway · SDKs and task handling [31: reading note]

n8n · HTTP Request node [34: reading note]

Remotion composition and Comfy generation

Covers: 3 · Use Remotion for composition and exact graphics · 4 · Use ComfyUI when you want a visible generation graph

Checked scaffolding commands, skills, template requirements, cloud MCP setup, API-format JSON, and output download behavior. The Comfy drawing is conceptual; cloud template availability must be discovered. The Remotion video transcript was not retrieved.

Remotion · Create videos with coding agents [25: reading note]

Remotion · Agent Skills [26: reading note]

ComfyUI · LTX-2.5 video workflows [28: reading note]

ComfyUI · Workflow Templates [29: reading note]

Comfy · MCP server for local and cloud workflows [37: reading note]

Comfy · Skills for agents [38: reading note]

Comfy-Org · Current skill and plugin repository [55: reading note]

Runway jobs and n8n orchestration

Covers: 5 · Use an API when you want explicit job handling · 6 · Build the repeating flow in n8n

Checked endpoint roles, asynchronous states, polling/error guidance, output expiry, and HTTP/Wait node behavior. The bounded recovery flow is proposed, not an executed n8n export. Multi-shot generation is separate from composing real gameplay capture.

Runway · Developer MCP server [30: reading note]

Runway · SDKs and task handling [31: reading note]

Runway · Multi-shot video recipe, 2026-06 [32: reading note]

Runway · Generated asset outputs [33: reading note]

n8n · HTTP Request node [34: reading note]

n8n · Wait node [35: reading note]

Runway · Usage tiers and throttled tasks [47: reading note]

Runway · API errors [48: reading note]

Hooks, execution prompt, and final review

Covers: 7 · Add a hook only for a repeated event action · 8 · Give Astra a brief that can become a working pipeline · Review the result and improve the right layer · Your first useful experiment

PostToolUse happens after the tool and cannot reverse its action. The validator, prompt, and review exercise are original. Pages serves static results; no hook, paid job, or video pipeline was run for this book.

OpenAI · Codex hooks [24: reading note]

OpenAI · Rethinking skills and prompts for GPT-6 Astra [17: reading note]

GitHub · What is GitHub Pages? [46: reading note]

Read the review method and corrections

Review & corrections

Follow a claim back to its evidence.

Each chapter's Source check connects its sections to direct references and distinguishes documentation, reported research, interpretation, and original proposals. The reading notes summarize the checked passages and their limits.

This review clarifies Godot export naming, callback timing versus displayed frames, coordinate and distance rules, and prompt comparisons. P0-P3 now identify different instruction stages; game V2 improving does not prove prompt P2 improved. The new GitHub comparison checks scope, dependency layout, and mandatory dialogue. Comfy's frozen skill folder is distinguished from its current cloud plugin and CLI skills.

Evidence limits: 63 of 64 source entries have checked content. Source 03 remains a historical lead with unavailable transcript evidence. Remotion and GDQuest video transcripts were not retrieved; written guides support those instructions. SkillAlchemy's Table 5 has a labeling inconsistency, so it is not used as clean transfer evidence. No reviewed independent benchmark establishes Astra game-development gains.

Two original Blender concept illustrations were actually rendered. The game, prompt comparisons, skill trials, and video pipelines remain worked proposals; no Signal Garden game or paid video-generation job was run for this book. The PDF editions include the summary, full chapters, prompts, diagrams, and source notes. Website checks and PDF visual inspection validate the publication, not the proposed game.

Download the detailed review record (.md)

Sources & reading notes

Read the main ideas here. Open a source for the detail.

Each verified entry includes main items, an application, limitations, and a passage locator. Source 03 instead records an unavailable transcript and remains a historical reading lead. **Research published or updated since 1 March 2026 takes priority.** Dated sources appear first, followed by current documentation and historical exceptions. Review dates describe when we checked a page; they are not publication dates.

“For this book” is our interpretation. Benchmark findings retain their tested model and task limits; none of the independent skill benchmarks below establishes an Astra game-development result. Source 02 summarizes a selected mirrored transcript passage and links the original video. Other video retrieval limits are stated individually. Recheck changing technical documentation against your installed versions.

Source 25

Official tutorial + companion video

Remotion · Create videos with coding agents

Updated since March 2026 · Updated 12 September 2026

Main items

- Create a blank video project and install its dependencies.
- Use a coding agent with relevant Remotion guidance to build the composition.
- Preview the result and refine its timing and appearance.

For this book: Start the trailer with real capture files and an exact, editable composition.

Limits / Astra relevance: The written guide was reviewed. The five-minute companion video’s transcript was not retrieved; no video-only claims are used.

Content checked · 2026-09-14

Passage locator: Prerequisites, Start a new project, Start the preview, Start your coding agent

Written guide and commands checked; companion video transcript not retrieved.

Original / companion YouTube video · Written guide reviewed; companion video transcript not retrieved.

Source 26

Official skill documentation

Remotion · Agent Skills

Updated since March 2026 · Updated 12 September 2026

Main items

- Provides reusable guidance for coding agents working with Remotion.
- The collection covers composition, documentation, preview, and rendering workflows.
- Installation exposes that guidance to compatible agent setups.

For this book: Check available skills before adding another copy of the same procedure.

Limits / Astra relevance: Vendor workflow guidance; it does not prove a quality gain for every model or task, including Astra.

Content checked · 2026-09-14

Passage locator: Available skills: best practices, studio, render, and docs

Confirms procedures available in the collection; no measured Astra benefit.

Source 27

Official deprecation notice

Remotion · Documentation MCP deprecation

Updated since March 2026 · Updated 12 September 2026

Main items

- Deprecates the older Remotion documentation MCP.
- Directs users toward agent skills, including documentation guidance.
- Explains problems with duplicated or stale documentation and tool invocation.

For this book: A concrete current example of replacing a redundant documentation integration.

Limits / Astra relevance: The notice does not imply all MCP servers are obsolete. Its shutdown wording is not confirmation of a completed shutdown.

Content checked · 2026-09-14

Passage locator: Deprecation notice, reasons, and migration to /remotion-docs

No-earlier-than shutdown date does not prove that shutdown already happened.

Source 17

First-party Astra guidance

OpenAI · Rethinking skills and prompts for GPT-6 Astra

Since March 2026 · Published 11 September 2026

Main items

- Reassess accumulated instructions when moving to Astra; elaborate procedures can constrain a stronger model.

- Keep skill triggers precise and load supporting material only when relevant.
- Replace blanket document-reading and repeated-testing demands with context-sensitive guidance.

For this book: Audit existing instructions before adding another skill.

Limits / Astra relevance: Practical guidance from Eric Provencher, not a controlled measurement of skill gains on Astra.

Content checked · 2026-09-14

Passage locator: Instruction audit, overlapping skill triggers, task-relevant context, and targeted checks
First-party Astra advice, not an independent controlled skill-gain benchmark.

Source 44

Official technical reference

MDN · Autoplay guide for media and Web Audio APIs

Updated since March 2026 · Updated 10 September 2026

Main items

- Audible media and Web Audio may be blocked from starting automatically.
- User interaction is a common condition for permitting playback; browser policies vary.
- Handle blocked playback instead of assuming a play request always succeeds.

For this book: Start or resume game audio from a player action and keep a working mute control.

Limits / Astra relevance: A user gesture is not an unconditional guarantee under every browser preference or embedding policy.

Content checked · 2026-09-14

Passage locator: Autoplay availability; autoplay using the Web Audio API; handling play failures

Source 04

First-party game case study

OpenAI · Building games with Astra

Since March 2026 · Published 4 September 2026

Main items

- Begin with the desired player experience and visual references.
- The example uses TypeScript, Vite, and Three.js with inspectable game state.
- Separate repeatable scene setup from tests that actually operate the controls.

For this book: Ground the game brief and verification plan in a current Astra example.

Limits / Astra relevance: Thomas Ricouard's showcase is an implementation account; it does not establish typical success rates.

Content checked · 2026-09-14

Passage locator: TypeScript/Vite/Three.js setup and inspectable state / scene fixture / journey-test discussion

Confirms the implementation example, not a typical success rate.

Source 20

Research preprint · skill generation

SkillAlchemy · Open-World Agent Skill Creation

Since March 2026 · Submitted 24 August 2026 · v1 reviewed

Main items

- Discovers task requirements and supporting external evidence before compiling a reusable skill.
- Reports improvements over no-skill and automated-generation baselines on 87 SkillsBench v1.1 tasks.
- Evaluations include GPT-5.5, Opus 4.8, and DeepSeek V4 configurations; none is Astra.

For this book: Treat evidence-led skill generation as a candidate worth testing.

Limits / Astra relevance: Preprint results do not show that casually generating a skill for any task will help. Table 5 mixes PDF-redaction instruction labels with a threejs-to-obj reuse description; we do not treat that table as clean evidence of transfer.

Content checked · 2026-09-14

Passage locator: v1 full text: sections 4.1 Experimental Setup and 4.2 Main Results

Tested configurations include GPT-5.5, Opus 4.8, and DeepSeek V4; none is Astra.

Source 06

Official workflow documentation

GitHub · Spec Kit

Updated since March 2026 · Updated 21 August 2026

Main items

- Organizes work around specification, planning, tasks, and implementation.
- Makes requirements and technical decisions explicit before they spread through code.
- Provides tooling and templates for a specification-driven workflow.

For this book: Borrow the separation of decisions; one page can be enough for Signal Garden.

Limits / Astra relevance: Workflow documentation is not evidence that installing the toolkit improves every small project or Astra run.

Content checked · 2026-09-14

Passage locator: Specification → plan → tasks → implementation workflow; update date

Confirms workflow structure; using a one-page subset is our recommendation.

Source 43

Official technical reference

MDN · requestAnimationFrame

Updated since March 2026 · Updated 21 August 2026

Main items

- Requests an animation callback before the next repaint.
- Callbacks generally follow display refresh and are paused in most background tabs.
- Use the timestamp to advance animation by elapsed time rather than by callback count.

For this book: Define resume behavior and label callback-interval measurements accurately.

Limits / Astra relevance: The book's 33 ms / 60-second observation is an original target. Callback cadence is not a GPU execution-time measurement.

Content checked · 2026-09-14

Passage locator: Opening behavior description, high-refresh warning, and timestamp parameter

Source 23

Protocol architecture

MCP · Architecture, specification 2026-07-28

Version since March 2026 · Specification version 28 July 2026 · not a page publication date

Main items

- An application hosts clients that connect to MCP servers.
- Servers expose tools, resources, and prompts through a common protocol.
- The protocol exchanges context; it does not dictate the agent's planning strategy.

For this book: Separate a connection that enables actions from instructions that guide those actions.

Limits / Astra relevance: A conceptual architecture, not a video generator or an Astra performance evaluation.

Content checked · 2026-09-14

Passage locator: Architecture overview: host, client, server, and scope of context exchange

The protocol does not dictate the application's planning strategy.

Source 18

Live benchmark · broad task collection

SkillsBench · Agent leaderboard, v1.1

Updated since March 2026 · Leaderboard recomputed 16 July 2026 · underlying project began before March

Main items

- Compares skills and no-skills conditions across a broad collection of 87 tasks.
- The reviewed table contains 24 paired agent-model configurations plus one with-skills-only result.
- Curated skills often help, but the size of the benefit varies across configurations.

For this book: Useful counterevidence to the claim that all skills are redundant.

Limits / Astra relevance: The reviewed leaderboard has no Astra entry. Different harnesses and task mixes limit transfer to game development.

Content checked · 2026-09-14

Passage locator: Leaderboard methodology and v1.1 July 16 recomputation note

87 tasks, 24 paired configurations plus one skill-only run; no Astra results.

Source 45

Official technical reference

MDN · prefers-reduced-motion

Updated since March 2026 · Updated 10 June 2026

Main items

- The media feature exposes a user preference to reduce non-essential motion.
- The reduce value selects reduced-motion styling.
- The examples show replacing or minimizing animation when that preference is active.

For this book: Reduce decorative UI motion and connect the preference to game animation settings.

Limits / Astra relevance: CSS does not automatically change movement implemented inside a WebGL simulation; that integration is our implementation recommendation.

Content checked · 2026-09-14

Passage locator: Opening definition, reduce value, and Examples

Source 32

Official versioned API recipe

Runway · Multi-shot video recipe, 2026-06

Version since March 2026 · Recipe version 2026-06 · page publication date not stated

Main items

- Supports automatic story decomposition and custom shot lists.
- The custom mode accepts a small set of shots with explicit timing.
- Inputs must follow the recipe's supported total duration and format.

For this book: A possible all-generated concept variant. The mixed-source trailer instead needs composition with real gameplay captures.

Limits / Astra relevance: A recipe version is not a publication date or an assurance of consistent characters, exact gameplay, or Astra-specific gains.

Content checked · 2026-09-14

Passage locator: How it works, Inputs, custom shot example, and version 2026-06

Recipe generates shots; mixed captured/generated assembly is a separate composition task.

Source 19

Research preprint · skill selection

SkillsInjector · Dynamic Skill Context Construction for LLM Agents

Since March 2026 · Submitted 28 May 2026 · v1 reviewed

Main items

- Selects skills for the current task and adapts how much skill context to include.
- Studies interactions between descriptions, context budget, and selection; adding more skills is not consistently better.
- Reports improvements on three benchmarks using Qwen-based agents.

For this book: Evaluate which skill was selected as well as the final output.

Limits / Astra relevance: A proposed research method, not a feature claimed for Codex. The tested agents are not Astra.

Content checked · 2026-09-14

Passage locator: v1 full text: experimental setup, adaptive budget analysis, and Limitations

Qwen-family evaluation; does not establish gains on closed models such as Astra.

Source 21

First-party engineering report · updated harness

Anthropic · Harness design for long-running application development

Since March 2026 · Published 24 March 2026

Main items

- Uses explicit quality criteria and a generator/evaluator loop for application work.
- Revisits the surrounding workflow as model capabilities improve.
- Simplifies intermediate scaffolding with Opus 4.6 while retaining useful planning and evaluation roles.

For this book: Reassess the older long-running-agent recipe instead of copying it unchanged.

Limits / Astra relevance: Claude application experiments do not determine the best number of agents or milestones for an Astra game.

Content checked · 2026-09-14

Passage locator: Transition from the Opus 4.5 workflow to the simplified Opus 4.6 workflow

Removing sprint structure is a model/task-specific observation, not a universal prescription.

Source 22

Preliminary research preprint · software tasks

SWE-Skills-Bench · Do Agent Skills Actually Help in Real-World Software Engineering?

Since March 2026 · Submitted 16 March 2026 · v1 reviewed

Main items

- Evaluates 49 public software-engineering skills on roughly 565 tasks with executable acceptance checks.
- Reports no gain for 39 of 49 skills and a small average improvement of about 1.2 percentage points.
- Finds overhead and some regressions, including mismatches between advice and repository versions.

For this book: Keep a no-added-skill baseline and examine regressions, not just aggregate wins.

Limits / Astra relevance: Explicitly preliminary; uses Claude Code with Haiku 4.5, not Astra. Its setup differs from SkillsBench and Codex skill routing.

Content checked · 2026-09-14

Passage locator: v1 full text: sections 3.4 Task Formulation, 4.1 Experimental Setup, and 4.3 Results

49 skills, 565 tasks, Claude Code with Haiku 4.5; discovery setup is not identical to Codex.

Source 01

Official model guidance

OpenAI · GPT-6 Astra prompting best practices

Current reference · Live documentation · publication/update date not stated

Main items

- Makes model-specific prompting recommendations for Astra.
- Clarifies autonomy, instruction-file behavior, and boundaries for delegated work.
- Recommends verification proportionate to the task.

For this book: Inform the agent's working agreement; keep game mechanics as separate design decisions.

Limits / Astra relevance: A moving documentation URL may later describe another model. Confirm its heading before reusing the advice.

Content checked · 2026-09-14

Passage locator: GPT-6 Astra prompting best practices: initiative, instructions, delegation, and verification
Model guidance supports the working agreements; game rules remain original.

Source 08

Official skill documentation

OpenAI · Build skills

Current reference · Live documentation · publication/update date not stated

Main items

- A skill packages SKILL.md metadata and instructions, with optional scripts, references, and assets.
- Supports explicit selection and implicit matching when the skill is eligible for automatic invocation.
- The initial catalog is budgeted; selected skills then supply their full instructions.

For this book: Check available skills before authoring a duplicate. Repository skills can live in .agents/skills.

Limits / Astra relevance: allow_implicit_invocation: false disables automatic invocation. A budgeted catalog can shorten or omit entries. Availability and matching do not guarantee correct selection or internet-wide discovery.

Content checked · 2026-09-14

Passage locator: Skill discovery and progressive disclosure; explicit/implicit invocation; allow_implicit_invocation

Checked routing eligibility and catalog-budget caveats, not automatic selection accuracy.

Source 07

Official agent documentation

OpenAI · Custom instructions with AGENTS.md

Current reference · Live documentation · publication/update date not stated

Main items

- Explains how repository instructions are discovered and scoped.
- Parent instructions and more local instructions can both affect a task.
- Repository guidance supplies persistent conventions across sessions.

For this book: Record commands and local conventions in a short, accurate instruction file.

Limits / Astra relevance: This mechanism is distinct from selecting a skill. Our downloadable example is not active in this book repository.

Content checked · 2026-09-14

Passage locator: How Codex discovers guidance: global and project directory instructions

Confirms scoped repository guidance; the example file is optional project content.

Source 09

Official engine tutorial

Godot · Your first 3D game

Current reference · Stable documentation · publication/update date not stated

Main items

- Builds a small 3D game through a sequence of practical lessons.
- Introduces scene organization, movement, enemies, collisions, and interface elements.
- Shows how individual engine concepts combine into a playable result.

For this book: Use a focused lesson when an engine concept blocks progress.

Limits / Astra relevance: Match the documentation to the installed Godot version. A tutorial does not evaluate an AI workflow.

Content checked · 2026-09-14

Passage locator: Tutorial overview and lesson sequence through UI and retry

Supports the beginner learning route; it does not implement Signal Garden.

Source 11

Official export documentation

Godot · Exporting for the Web

Current reference · Stable documentation · publication/update date not stated

Main items

- Web export uses WebGL 2 and the Compatibility renderer; Godot 4 C# web export remains unavailable in the reviewed guidance.
- Threaded builds introduce hosting isolation requirements; single-thread export has different constraints.
- Browser input, audio, and hosting behavior require explicit attention.

For this book: Try a tiny export on the intended host before building the full game.

Limits / Astra relevance: Check the exact engine release. A model upgrade does not remove browser or hosting restrictions.

Content checked · 2026-09-14

Passage locator: Web export requirements, export file names, thread support, and PWA workaround
Checked C#, WebGL 2, single-thread choice, and the HTML naming exception.

Source 12

Official deployment documentation

Vite · Deploying a Static Site

Current reference · Live documentation · publication/update date not stated

Main items

- Production builds normally generate the dist directory.
- GitHub Pages project sites need a base path matching the repository name.
- The guide provides a GitHub Actions deployment example.

For this book: Build and test the game at its eventual repository subpath.

Limits / Astra relevance: Applies to the proposed Vite game. This book uses a custom static builder and a locally bundled Mermaid dependency.

Content checked · 2026-09-14

Passage locator: GitHub Pages section: base and workflow; production dist output

Applies to the proposed Vite game, not the book's build implementation.

Source 13

Official hosting documentation

GitHub · Using custom workflows with GitHub Pages

Current reference · Live documentation · publication/update date not stated

Main items

- Separates the build artifact from the deployment job.

- Describes Pages setup, artifact upload, deployment permissions, and the github-pages environment.
- Requires the deployment job to receive the appropriate artifact and authorization.

For this book: Publish only the generated dist output and inspect the completed deployment.

Limits / Astra relevance: A successful workflow does not prove correct asset paths or playable browser behavior.

Content checked · 2026-09-14

Passage locator: Configuring upload/deploy steps, required permissions, and environment

Checked the deployment pattern; current project workflow is a separate local artifact.

Source 14

Official library repository

Three.js · README

Current reference · Live repository · no single publication date used

Main items

- Provides a JavaScript library for displaying interactive 3D content.
- Links installation guidance, documentation, examples, and rendering capabilities.
- Its rendering focus leaves game rules and application structure to the developer.

For this book: A suitable starting library for a small browser game with custom mechanics.

Limits / Astra relevance: The README is a capability reference, not a performance comparison with other engines. Pin the installed version.

Content checked · 2026-09-14

Passage locator: Repository README: JavaScript 3D library and renderers

Gameplay responsibilities are our assessment of the proposed stack.

Source 15

Official engine feature reference

Babylon.js · Specifications

Current reference · Live feature reference · publication/update date not stated

Main items

- Lists the browser engine's rendering and scene capabilities.
- Covers integrated tooling and physics-related support.
- Provides a feature checklist for assessing an engine-oriented browser stack.

For this book: Consider Babylon.js when its existing systems match the game's needs.

Limits / Astra relevance: Feature availability does not establish usability or speed on your project; this book ran no engine benchmark.

Content checked · 2026-09-14

Passage locator: Specifications: physics, character controllers, and Inspector

Confirms available facilities, not performance against other engines.

Source 16

Official engine compatibility documentation

Unity · Web browser compatibility

Current reference · Live manual · Unity 6.6 page reviewed; publication/update date not stated

Main items

- Defines supported browser requirements for Unity web content.
- WebGL 2, WebAssembly, and browser/platform compatibility matter to deployment.
- Compatibility must be assessed against the Unity release and target device.

For this book: Test an export in the actual target browser before committing to this stack.

Limits / Astra relevance: Avoid applying older blanket claims about browser support to a newer Unity version.

Content checked · 2026-09-14

Passage locator: Current Unity web browser requirements and compatibility table

Check against the installed editor; page build date is not treated as article publication.

Source 24

Official agent lifecycle documentation

OpenAI · Codex hooks

Current documentation · Live documentation · publication date not stated

Main items

- Hooks run actions at supported lifecycle events such as PreToolUse and PostToolUse.
- Configuration, event matching, and trust rules govern when they execute.
- PostToolUse follows a completed action; it cannot reverse that action.

For this book: Consider a narrowly filtered local manifest check after proving the validator works directly.

Limits / Astra relevance: The illustrated hook is a proposed design, not installed configuration. An agent hook differs from an HTTP webhook.

Content checked · 2026-09-14

Passage locator: Config shape, tool coverage, and PostToolUse lifecycle semantics

An after-tool event cannot undo the completed tool action; proposed hook not installed here.

Source 28

Official model workflow tutorial

ComfyUI · LTX-2.5 video workflows

[Current documentation](#) · [Live documentation](#) · publication date not stated

Main items

- Covers text-to-video, image-to-video, and first/last-frame workflows.
- Describes supported audio and multi-shot capabilities.
- Explains model access and compatible component requirements.

For this book: Use an available supported template and inspect one short output before expanding the pipeline.

Limits / Astra relevance: Gated model access and compute requirements apply. The page is current documentation, not evidence of its publication date or an Astra benchmark.

Content checked · 2026-09-14

Passage locator: Workflow examples, model downloads/access, and text/image/first-last frame routes

Inspect compatible template and account requirements before attempting generation.

Source 29

Official interface tutorial

ComfyUI · Workflow Templates

[Current documentation](#) · [Live documentation](#) · publication date not stated

Main items

- Browse example workflows from the application's template interface.
- Templates provide starting graphs for different media tasks.
- Inspect and satisfy the selected workflow's model requirements before running it.

For this book: Start from a runnable official template instead of treating the book's conceptual Mermaid diagram as executable JSON.

Limits / Astra relevance: Availability depends on the installation and template. A tutorial does not guarantee compatibility with every version.

Content checked · 2026-09-14

Passage locator: Built-in Workflow Templates and handling required model downloads

A template is an executable starting graph only with compatible requirements.

Source 37

Official public-beta agent integration

Comfy · MCP server for local and cloud workflows

[Current documentation](#) · [Live documentation](#) · publication date not stated

Main items

- Documents separate local and cloud connections.
- Exposes template discovery, workflow execution, and job/output handling.
- `get_output` returns download instructions; the agent must save and verify the file separately.

For this book: Inspect a template schema, then submit a bounded authorized job.

Limits / Astra relevance: Public beta; cloud execution has account requirements. Connection and generation were not exercised for this book.

Content checked · 2026-09-14

Passage locator: Codex cloud setup; template discovery/schema; job tools; `get_output` download behavior

Public beta; setup was read, not executed. Cloud availability must be checked for the account.

Source 38

Official agent skill documentation

Comfy · Skills for agents

[Current documentation](#) · [Live documentation](#) · publication date not stated

Main items

- Packages guidance about Comfy workflows and model use.
- Links to the maintained Comfy skills collection.
- Includes installation examples tied to particular agent hosts.

For this book: Combine missing procedural knowledge with an existing action interface when it helps.

Limits / Astra relevance: Host-specific plugin commands are not interchangeable. No controlled Astra skill comparison is provided.

Content checked · 2026-09-14

Passage locator: Skill purpose and host-specific installation examples

Vendor guidance is not evidence that a custom skill improves every agent or task.

Source 30

Official MCP integration guide

Runway · Developer MCP server

[Current documentation](#) · [Live documentation](#) · [publication date not stated](#)

Main items

- Documents the developer MCP endpoint at dev.runwayml.com/mcp.
- Its tools support developer tasks and API documentation workflows.
- Identifies mcp.runwayml.com as a separate media-generation integration.

For this book: Choose the server by its exposed capability, then inspect the tools actually registered after connection.

Limits / Astra relevance: The developer endpoint should not be presented as the media-generation server. Generation-server setup was not tested for this book.

Content checked · 2026-09-14

Passage locator: [Connection endpoints](#) and [After you connect](#)

Developer documentation MCP is distinct from the linked generation integration.

Source 31

Official asynchronous API guide

Runway · SDKs and task handling

[Current documentation](#) · [Live documentation](#) · [publication date not stated](#)

Main items

- Generation tasks are asynchronous and expose status for later retrieval.
- SDK helpers can wait for task output.
- Manual polling guidance includes a minimum interval, jitter, and backoff.

For this book: Persist the task ID and distinguish a local wait timeout from a remote task failure.

Limits / Astra relevance: Provider-specific semantics apply. A successful task still requires visual review; the book did not submit a paid request.

Content checked · 2026-09-14

Passage locator: [Built-in task polling](#), [manual polling](#), and [timeout/cancellation behavior](#)

Checked terminal states and local timeout semantics; no paid generation run performed.

Source 33

Official output lifecycle guide

Runway · Generated asset outputs

[Current documentation](#) · [Live documentation](#) · publication date not stated

Main items

- Task results include URLs for generated assets.
- Output delivery URLs are temporary, with a documented 24-48-hour expiry window after access.
- Applications should download and store the files they need.

For this book: Keep a durable saved asset and its task record before assembling or publishing the trailer.

Limits / Astra relevance: These are Runway delivery semantics, not a universal retention rule for all video services.

Content checked · 2026-09-14

Passage locator: Generated output delivery URLs and expiration guidance

Save outputs to owned storage; no file was generated or downloaded for this book.

Source 34

[Official automation node documentation](#)

n8n · HTTP Request node

[Current documentation](#) · [Live documentation](#) · publication date not stated

Main items

- Makes HTTP calls with configurable methods, URLs, headers, and bodies.
- Supports authentication through credential configurations.
- Can be used as a regular workflow node or an agent tool.

For this book: Translate one verified provider request into a workflow step before adding orchestration.

Limits / Astra relevance: The node does not generate video itself or validate an arbitrary provider's request schema for you.

Content checked · 2026-09-14

Passage locator: Method, URL, Authentication, and Send Body node parameters

Confirms HTTP request configuration; provider API performs generation.

Source 35

[Official automation node documentation](#)

n8n · Wait node

[Current documentation](#) · [Live documentation](#) · publication date not stated

Main items

- Pauses a workflow and resumes it after an interval or another supported condition.
- Can resume on a webhook call with an execution-specific resume URL.
- Webhook resume supports authentication settings.

For this book: Wait between status requests, or use a documented provider callback when one exists.

Limits / Astra relevance: The node's webhook support does not establish callback support in a particular video API.

Content checked · 2026-09-14

Passage locator: After Time Interval, On Webhook Call, and webhook authentication

Short waits under 65 seconds do not offload execution data to the database.

Source 36

Official diagram rendering documentation

Mermaid · Usage

[Current documentation](#) · [Live documentation](#) · publication date not stated

Main items

- Renders text diagram definitions into SVG.
- Supports initialization options and programmatic rendering.
- Allows diagrams to be embedded in a web page.

For this book: The book renders nine original diagrams and exposes their editable source and SVG downloads.

Limits / Astra relevance: Mermaid depicts a flow; it does not execute a generation pipeline. The bundled renderer version is locked in package.json.

Content checked · 2026-09-14

Passage locator: Installing and hosting Mermaid, rendering definitions, and securityLevel

Confirms rendering capability; the book's diagrams are original process proposals.

Source 39

Official technical reference

Git · Staging, differences, branches, and reverting

[Current reference](#) · [Live documentation](#) · publication/update date not stated

Main items

- git add stages a snapshot of selected changes; --all includes additions, modifications, and removals within its scope.
- git diff --cached shows the staged changes to be committed; --stat is only a summary.
- The companion references document init -b, switch -c, and revert, which records an undo in a new commit.

For this book: Stage the complete new scaffold, review the staged content, and preserve unrelated work in an existing project.

Limits / Astra relevance: Commands describe Git behavior; the proposed branch names, file layout, and commit cadence are our choices.

Content checked · 2026-09-14

Passage locator: git-add Description and --all; git-diff --cached; git-init --initial-branch; git-switch -c; git-revert Description

Companion references:

git diff - staged content
git init - initial branch
git switch - create a branch
git revert - record an undo

Source 40

Official technical reference

GitHub CLI · gh repo create

Current reference · Live documentation · publication/update date not stated

Main items

- Creates a repository with explicit visibility and an optional owner/name.
- Use --source to attach an existing local project and --remote to name its remote.
- The --push option is optional; a later git push can publish the committed branch.

For this book: Create the intended public game repository after preparing and reviewing its files.

Limits / Astra relevance: Requires an installed, authenticated GitHub CLI and account permissions. The book example does not create a game repository itself.

Content checked · 2026-09-14

Passage locator: Description, Examples, and --source / --remote / --public / --push options

Source 41

Official technical reference

GitHub CLI · gh pr create

Current reference · Live documentation · publication/update date not stated

Main items

- --base selects the target branch and --head selects the source branch.
- --body-file reads the pull request description from an existing file.
- --title supplies the pull request title; flags support a reproducible CLI request.

For this book: Write the review description before invoking the example PR command.

Limits / Astra relevance: The command cannot verify gameplay quality. The camera example is a proposed change, not a PR created by this book.

Content checked · 2026-09-14

Passage locator: Options: --base, --head, --body-file, and --title

Source 42

Official technical reference

npm · npm ci

Current reference · Live documentation · publication/update date not stated

Main items

- Requires an existing package-lock.json or npm-shrinkwrap.json.
- Fails when package.json and the lockfile disagree instead of updating the lock.
- Performs a clean dependency install without rewriting package manifests or lockfiles.

For this book: Use the committed dependency graph in CI and document any project flags needed to reproduce it.

Limits / Astra relevance: A lockfile improves dependency repeatability; it does not prove browser compatibility, game correctness, or a secure dependency set.

Content checked · 2026-09-14

Passage locator: Description: differences from npm install and the note about matching install flags

Source 46

Official technical reference

GitHub · What is GitHub Pages?

Current reference · Live documentation · publication/update date not stated

Main items

- Pages publishes static HTML, CSS, and JavaScript, with an optional build process.

- Project sites normally use a repository path below the account domain.
- User or organization sites and project sites have different repository and URL conventions.

For this book: Publish built game/book files; run secret-bearing media jobs in a separate local tool or backend.

Limits / Astra relevance: Static hosting is not a private API worker. Media files remain subject to GitHub's hosting limits; the book does not benchmark media delivery.

Content checked · 2026-09-14

Passage locator: About GitHub Pages and the user/organization versus project-site table

Source 47

Official technical reference

Runway · Usage tiers and throttled tasks

Current reference · Live documentation · publication/update date not stated

Main items

- Concurrency limits constrain how many generation tasks can run at once.
- A THROTTLED task is accepted and stored while waiting to enter processing.
- Task throttling is a task state, distinct from an HTTP rate-limit response.

For this book: Continue bounded status polling for an existing throttled task instead of submitting a duplicate.

Limits / Astra relevance: Limits depend on the account and service. The book deliberately does not hard-code a tier's prices or concurrency allowance.

Content checked · 2026-09-14

Passage locator: Concurrency limits and THROTTLED task status

Source 48

Official technical reference

Runway · API errors

Current reference · Live documentation · publication/update date not stated

Main items

- The error table distinguishes non-retryable input or authentication failures from retryable conditions.
- Rate limiting and transient service failures can require retries.
- Exponential backoff with jitter reduces repeated synchronized requests.

For this book: Bound retries of status requests and reconcile uncertain generation submissions.

Limits / Astra relevance: Retry advice does not make generation submission exactly-once. Our stop-and-reconcile policy is a proposed application design.

Content checked · 2026-09-14

Passage locator: HTTP error table, retry guidance, and exponential backoff with jitter

Source 49

Primary source · workflow reference

OpenAI · Web game foundations skill

Current reference · Content checked 14 September 2026 · no publication date inferred

Main items

- Separates simulation rules, rendering, input, and browser UI.
- Routes engine selection according to the project rather than using one renderer for every game.
- References shared files outside the skill directory.

For this book: A candidate for initial browser-game architecture or a structural review.

Limits / Astra relevance: A source inspection, not an Astra benchmark. Preserve its supporting plugin files; a copied SKILL.md alone can break relative references.

Content checked · 2026-09-14

Passage locator: Workflow, engine defaults, anti-patterns, and relative reference paths

Reviewed the named passage; scope and execution limits are stated above.

Source 50

Primary source · workflow reference

Remotion · Maintained GitHub skill collection

Current reference · Content checked 14 September 2026 · no publication date inferred

Main items

- Provides separate creation, markup, preview, rendering, caption, and documentation skills.
- The README offers npx skills add remotion-dev/skills.
- The broad best-practices skill covers the collection; narrower entries address specific jobs.

For this book: Use composition and rendering guidance for a trailer assembled from real game capture.

Limits / Astra relevance: Skill availability does not install all runtime dependencies or prove the exported video is correct. Reviewed instructions; no trailer run.

Content checked · 2026-09-14

Passage locator: README: installation and Available skills

Reviewed the named passage; scope and execution limits are stated above.

Source 51

Primary source · workflow reference

Community example · Remotion Video Director

Current reference · Content checked 14 September 2026 · no publication date inferred

Main items

- Adds audience, message, and creative direction to technical Remotion guidance.
- Requires an interactive discovery phase with successive answers.
- References companion patterns and archetype files.

For this book: Consider it when the video brief is unclear.

Limits / Astra relevance: Community workflow, not a quality ranking. Its mandatory dialogue can add needless pauses to an already complete brief; inspect host/tool assumptions before adoption.

Content checked · 2026-09-14

Passage locator: Version 2.0.0: Relationship to Official Remotion Skill; Phase 1

Reviewed the named passage; scope and execution limits are stated above.

Source 52

Primary source · workflow reference

OpenAI · Evaluation best practices

Current reference · Content checked 14 September 2026 · no publication date inferred

Main items

- Use task-specific checks and representative examples.
- Record failures and combine measurable checks with human judgment.
- Compare revisions and add newly discovered cases to the evaluation set.

For this book: Keep prompt versions, outcomes, and held-out game scenarios in the repository.

Limits / Astra relevance: General evaluation guidance, not a game benchmark. The legacy hosted Evals product is being retired; the method does not require it.

Content checked · 2026-09-14

Passage locator: Define objectives, collect cases, choose metrics, compare; evaluation anti-patterns

Reviewed the named passage; scope and execution limits are stated above.

Source 53

Primary source · workflow reference

OpenAI · Prompt optimizer and retirement notice

Current reference · Content checked 14 September 2026 · no publication date inferred

Main items

- Optimization uses responses plus specific critiques or grader results.
- An optimized prompt can perform worse on some inputs; review and evaluate it.
- The page schedules legacy Evals read-only access for 31 October 2026 and shutdown for 30 November 2026.

For this book: Use the feedback principle; keep this book's comparison independent of the retiring dashboard.

Limits / Astra relevance: The dates describe the dataset-backed optimizer and Evals platform, not the end of prompt improvement or local evaluation. Recheck the notice before use.

Content checked · 2026-09-14

Passage locator: Deprecation notice; Prepare your data; Optimize your prompt

Reviewed the named passage; scope and execution limits are stated above.

Source 54

Primary source · workflow reference

Blender · Command line rendering, 4.5 manual

Current reference · Content checked 14 September 2026 · no publication date inferred

Main items

- Blender can render without opening its graphical interface.
- Command arguments are processed in order; set output options before rendering.
- A saved scene and an explicit render command support repeatable exports.

For this book: Reproduce the original concept figures from the included scene-building script.

Limits / Astra relevance: This book used the shared Blender 4.5.3 LTS executable. A rendered illustration proves neither game mechanics nor browser performance.

Content checked · 2026-09-14

Passage locator: Command Line Rendering: background mode and argument ordering

Reviewed the named passage; scope and execution limits are stated above.

Source 55

Primary source · workflow reference

Comfy-Org · Current skill and plugin repository

Current reference · Content checked 14 September 2026 · no publication date inferred

Main items

- The canonical cloud package targets Claude Code and bundles MCP with commands.
- The legacy skills folder is frozen; comfy-cli skills now ship with that CLI.
- Instructions should discover live models and templates through tools.

For this book: Use the supported host setup and discover the available video template at execution time.

Limits / Astra relevance: Do not copy the frozen folder or assume a Claude Code plugin is a Codex installer. Source inspection only; no cloud generation was run.

Content checked · 2026-09-14

Passage locator: README: Install; Repo layout; Authoring rule; Status

Reviewed the named passage; scope and execution limits are stated above.

Source 05

Historical first-party engineering report

Anthropic · Effective harnesses for long-running agents

Historical exception · Published 26 November 2025

Main items

- Separates initial project setup from subsequent incremental feature work.
- Uses progress records and Git history to help later sessions resume.
- Emphasizes browser checks for end-to-end behavior.

For this book: Retain recoverable progress and observable completion; reassess the amount of scaffolding.

Limits / Astra relevance: Read with the March 2026 follow-up [21]. Its earlier agent setup is not an Astra prescription.

Content checked · 2026-09-14

Passage locator: Initializer agent, coding agent, progress files, Git checkpoints, and browser testing

Historical long-running-app study; reconsider scaffolding using the March follow-up.

Source 02

YouTube transcript · primary speech, mirrored

Andrej Karpathy · Software Is Changing (Again)

Historical exception · Talk published 17 June 2025

Main items

- AI generation creates a corresponding human verification workload.
- Concrete requests and inspectable intermediate artifacts can make review easier.
- A large generated diff still demands substantial scrutiny; search the transcript for “verification”.

For this book: Keep observable game checks; current Astra game guidance [04] provides a newer practical example.

Limits / Astra relevance: Selected transcript passage, not a full talk synopsis or benchmark. No stable timecodes were available; captions can contain errors.

Content checked · 2026-09-14

Passage locator: Transcript passage containing “verification” and discussion of large diffs

Selected speech passage checked through a third-party mirror; no stable timecodes.

Original / companion YouTube video · Selected transcript passage checked through a mirror.

Source 03

Historical video lead · transcript unavailable in this review

Andrej Karpathy · How I use LLMs

Historical exception · Video published February 2025

Reading status

- The earlier edition included a selected transcript note about context; its passage summary is withdrawn pending fresh verification.
- The original Pickscribe mirror timed out, and an alternative transcript page exposed no speech text.
- Use this as a historical viewing lead. No current technical recommendation depends on its unavailable transcript.

For this book: For current context-loading advice, read the verified Astra guidance [17].

Limits / Astra relevance: The spoken content was not reverified on 14 September 2026. A reachable landing page did not establish transcript access.

Transcript unavailable · 2026-09-14

Passage locator: Original video linked; Pickscribe and alternative transcript retrieval attempted

Unavailable speech text is excluded from this revision’s verified recommendations.

Original / companion YouTube video · Transcript unavailable in this review; no fresh speech-content claim is made.

Source 10

Creator’s written tutorial · associated YouTube video

GDQuest · Your first 3D game with Godot 4

Historical exception · Published 3 March 2025

Main items

- The accessible written course builds a Survivor Arena FPS in Godot 4.
- Uses a concrete small game to introduce 3D development concepts.
- Connects lessons to a playable project that can be examined and adapted.

For this book: Use as a learning exercise alongside current Godot documentation [09] and export constraints [11].

Limits / Astra relevance: Video NJJNWGD25rg had no usable transcript retrieved. These notes summarize the written course, not unseen video content.

Content checked · 2026-09-14

Passage locator: Written Survivor Arena FPS introduction and course sequence; date 2025/03/03

Written course checked. Associated YouTube transcript remains unavailable.

Source 56

Official documentation

OpenAI · Desktop quickstart and Windows setup

Current documentation · Live documentation · checked 14 September 2026; page date not stated

Main items

- Desktop can work with a selected local folder.
- Sign in, then select Codex for development.
- Windows supports native PowerShell or WSL.

For this book: Connect the game folder before using the cookbook.

Limits / Astra relevance: Model access and environment permissions vary.

Content checked · 2026-09-14

Passage locator: Setup: install, sign in, select folder, choose Codex

Linked passages checked; this is setup guidance, not an executed installation or game deployment.

Companion references:

Windows installation and environment

Desktop overview

Source 57

Official documentation

Anthropic · Claude Code desktop quickstart

[Current documentation](#) · [Live documentation](#) · checked 14 September 2026; page date not stated

Main items

- Code has local project access.
- Desktop includes Claude Code; CLI installation is separate.
- Code requires eligible subscription access; Git supports worktree isolation.

For this book: Use Code → Local with the game folder.

Limits / Astra relevance: Chat, local Code and cloud environments differ.

Content checked · 2026-09-14

Passage locator: Install; Start your first session; desktop reference: Git and Git LFS errors

Linked passages checked; this is setup guidance, not an executed installation or game deployment.

Companion references:

Desktop environment and troubleshooting

Source 58

[Official documentation](#)

Git and GitHub · Installation and commit identity

[Current documentation](#) · [Live documentation](#) · checked 14 September 2026; page date not stated

Main items

- Git offers OS-specific installation routes.
- Configure the author name and commit email.
- GitHub authentication uses HTTPS or SSH separately.

For this book: Check Git before making the baseline commit.

Limits / Astra relevance: Author identity is not an account login.

Content checked · 2026-09-14

Passage locator: Git for Windows downloads; Set up Git; Setting your commit email address

Linked passages checked; this is setup guidance, not an executed installation or game deployment.

Companion references:

Set up Git

Commit email and no-reply address

Source 59

[Official documentation](#)

Node.js · Downloads

[Current documentation](#) · [Live documentation](#) · checked 14 September 2026; page date not stated

Main items

- The download selector distinguishes LTS and Current.
- Choose the installer for your OS and architecture.
- The page also links release status and verification.

For this book: Use a supported LTS compatible with the game toolchain.

Limits / Astra relevance: Check actual project requirements; a version number alone is not proof of compatibility.

Content checked · 2026-09-14

Passage locator: LTS selector; operating system and npm options

Linked passages checked; this is setup guidance, not an executed installation or game deployment.

Source 60

[Official documentation](#)

GitHub · Creating an account

[Current documentation](#) · [Live documentation](#) · checked 14 September 2026; page date not stated

Main items

- Create a personal account through signup.
- Verify the email to enable basic repository tasks.
- GitHub recommends two-factor authentication.

For this book: Complete the account steps personally before agent publication.

Limits / Astra relevance: Enterprise-managed accounts follow different setup rules.

Content checked · 2026-09-14

Passage locator: Signing up for a new personal account; Next steps

Linked passages checked; this is setup guidance, not an executed installation or game deployment.

Source 61

[Official documentation](#)

GitHub CLI · Browser login and Git authentication

[Current documentation](#) · [Live documentation](#) · checked 14 September 2026; page date not stated

Main items

- Login supports a browser flow and HTTPS or SSH selection.
- HTTPS login can also authenticate Git.
- Credential storage can fall back to a plain-text file.

For this book: Use the browser flow, then check auth status in the actual agent environment.

Limits / Astra relevance: GitHub CLI, Git transport and Codex sign-in are distinct.

Content checked · 2026-09-14

Passage locator: gh auth login; GitHub CLI quickstart: Prerequisites

Linked passages checked; this is setup guidance, not an executed installation or game deployment.

Companion references:

GitHub CLI installation

GitHub CLI quickstart

Source 62

Official documentation

GitHub · SSH key setup and connection test

Current documentation · Live documentation · checked 14 September 2026; page date not stated

Main items

- Check existing keys before generating another.
- Add the public key to GitHub; protect the private key.
- Verify the host fingerprint and the authenticated username.

For this book: Use SSH as an alternative Git transport.

Limits / Astra relevance: Agent setup is OS-specific; the successful test can exit with code 1.

Content checked · 2026-09-14

Passage locator: Generating a new SSH key; Adding a key; Testing your SSH connection

Linked passages checked; this is setup guidance, not an executed installation or game deployment.

Companion references:

Add the public key

Test connection and follow the official fingerprint link

Source 63

Official documentation

Vercel · Getting started, Git import and Vite

Current documentation · Getting-started page updated 11 August 2026

Main items

- Deployment can use the CLI or dashboard Git import.
- Connected Git changes can trigger deployments.
- Vite projects can build static production output.

For this book: Check scope, root, build output and public production URL.

Limits / Astra relevance: Dashboard use does not require an extra skill; framework and account access still matter.

Content checked · 2026-09-14

Passage locator: Deploy from the dashboard; CLI login and production deploy; Vite: Getting started

Linked passages checked; this is setup guidance, not an executed installation or game deployment.

Companion references:

Vite on Vercel

Vercel for GitHub

Source 64

Official documentation

Vercel · Hobby plan

Current documentation · Updated 31 August 2026

Main items

- Hobby is a free plan.
- It is restricted to personal, non-commercial use.
- Usage limits apply and can temporarily stop features.

For this book: Check eligibility before publishing the example on Hobby.

Limits / Astra relevance: Do not promise unlimited hosting or commercial eligibility.

Content checked · 2026-09-14

Passage locator: Hobby billing cycle; Comparing Hobby and Pro plans

Linked passages checked; this is setup guidance, not an executed installation or game deployment.

Make something small.

Learn something real.

Make it better.